FINAL TERM PREPERATION CONCEPTUAL FILE:

CS 311 : INTRODUTION TO WEB SERVICES DEVELOPMENT:

Prepared by Maira: Cilck Here To Join The Group

Lesson 28: Removing, Clearing, and Replacing XML Nodes

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum ek school ke notice board ke manager ho. Kabhi tumhe purani notice hataani hoti hai, kabhi sirf uska text badalna hota hai, aur kabhi poori notice ko naye content ke saath replace karna hota hai. XML DOM mein bhi bilkul yahi kaam hota hai — nodes ko remove karo, text saaf karo, ya poora node replace kar do.

Key Points:

1. removeChild(): Yeh method kisi bhi child node ko uske parent se hata deta hai — chahe woh element node ho ya text node.
2. nodeValue: Yeh property text node ka content directly change ya clear karne ke liye use hoti hai, bina node remove kiye.
3. removeAttribute(): Yeh method kisi element ke attribute ko uske naam se remove karta hai.
4. removeAttributeNode(): Yeh method attribute ko uske node object ke zariye remove karta hai — aksar loop mein use hota hai.
5. replaceChild(): Yeh method ek existing node ko ek naye node se replace kar deta hai.
6. createElement() / createTextNode(): Naya element ya text node banane ke liye use hote hain, aksar replaceChild() se pehle.

Example:

Roman Urdu mein sochو ke tum apne ghar ki shopping list manage kar rahe ho. Pehle tumne "chai" likha tha — ab woh khatam ho gayi toh tumne us item ko list se hata diya (removeChild). Phir tumhe laga "biscuit" ki jagah "cake" likhna chahiye, toh tumne poori line ko naye content se replace kar diya (replaceChild). Aur agar list mein koi extra column tha jaise "price" — woh tumne ek baar mein hata diya (removeAttribute). Yahi sab kaam XML DOM bhi karta hai.

Code Session

PART A — Working Code Example:

java

```java
import javax.xml.parsers.*;
import org.w3c.dom.*;
import java.io.*;

public class XMLNodeOperations {
   public static void main(String[] args) throws Exception {

      // Load and parse the XML document
      DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
      DocumentBuilder builder = factory.newDocumentBuilder();

      // Parse an XML string directly (simulating books.xml)
      String xmlContent = "<bookstore>" +
            "<book category=\"fiction\">" +
            "<title>Old Title</title>" +
            "</book>" +
            "</bookstore>";

      Document xmlDoc = builder.parse(
         new java.io.ByteArrayInputStream(xmlContent.getBytes())
      );

      // --- DEMO 1: Remove text node using removeChild() ---
      Node titleNode = xmlDoc.getElementsByTagName("title").item(0); // get first <title>
      Node textNode = titleNode.getChildNodes().item(0);          // get its text child
      titleNode.removeChild(textNode);                            // remove the text node
      System.out.println("After removeChild on text node:");
      System.out.println("Title text: '" + titleNode.getTextContent() + "'"); // should be empty

      // --- DEMO 2: Restore and clear text using nodeValue ---
      // Re-add text to demonstrate clearing
      Text restoredText = xmlDoc.createTextNode("Restored Title");
      titleNode.appendChild(restoredText);                        // add text back
      titleNode.getChildNodes().item(0).setNodeValue("");         // clear using nodeValue
      System.out.println("After clearing with nodeValue:");
      System.out.println("Title text: '" + titleNode.getTextContent() + "'");

      // --- DEMO 3: Remove attribute by name ---
      Element bookElement = (Element) xmlDoc.getElementsByTagName("book").item(0);
      System.out.println("Before removeAttribute: " + bookElement.getAttribute("category"));
      bookElement.removeAttribute("category");                    // remove "category" attribute
      System.out.println("After removeAttribute: '" + bookElement.getAttribute("category") + "'");

      // --- DEMO 4: Replace an element node ---
      Element newBook = xmlDoc.createElement("book");             // create new <book>
      Element newTitle = xmlDoc.createElement("title");           // create new <title>
      Text newText = xmlDoc.createTextNode("A Notebook");         // create text node
```

```
        newTitle.appendChild(newText);                  // add text to <title>
        newBook.appendChild(newTitle);                  // add <title> to <book>

        Node root = xmlDoc.getDocumentElement();        // get <bookstore>
        Node oldBook = xmlDoc.getElementsByTagName("book").item(0);   // get old <book>
        root.replaceChild(newBook, oldBook);            // replace old with new

        System.out.println("After replaceChild:");
        System.out.println("New title: " + xmlDoc.getElementsByTagName("title").item(0).getTextContent());
    }
}
```

Code Walkthrough (Roman Urdu):

- Pehle DocumentBuilder se XML string ko parse kiya gaya — yeh bilkul waise hai jaise `books.xml` load karna.
- DEMO 1 mein `getElementsByTagName("title")` se pehla title element liya, phir `getChildNodes().item(0)` se uska text child node liya, aur `removeChild()` se woh hata diya — ab title ka content khaali ho gaya.
- DEMO 2 mein pehle text wapis add kiya, phir `setNodeValue("")` se directly us text node ki value empty string kar di — yeh removeChild ke bajaye zyada simple tarika hai sirf text saaf karne ka.
- DEMO 3 mein `removeAttribute("category")` use karke book element ka `category` attribute hata diya. Pehle print kiya ke attribute tha, baad mein print kiya ke ab khaali string hai.
- DEMO 4 mein naya `book` element banaya, uske andar naya `title` element rakha, aur uske andar `"A Notebook"` wala text node. Phir `replaceChild(newBook, oldBook)` se purana book node naye se replace ho gaya.

Output:

```
After removeChild on text node:
Title text: "
After clearing with nodeValue:
Title text: "
Before removeAttribute: fiction
After removeAttribute: "
After replaceChild:
New title: A Notebook
```

Yeh output confirm karta hai ke `removeChild()`, `nodeValue`, `removeAttribute()`, aur `replaceChild()` sab successfully kaam kar rahe hain — text hata, attribute gaya, aur poora node replace ho gaya.

Common Mistakes — Error Finder Session

Mistake 1: Removing text node from wrong parent

Buggy Code:

java
```
// Trying to remove text node from document instead of its parent
Node titleNode = xmlDoc.getElementsByTagName("title").item(0);
Node textNode = titleNode.getChildNodes().item(0);
xmlDoc.removeChild(textNode); // WRONG: removing from document root
```

What's Wrong (Roman Urdu): Student ne `removeChild()` document root par call kar diya, jabke yeh method us node ke direct parent par call hona chahiye. Text node ka parent `titleNode` hai, `xmlDoc` nahi.

Fixed Code:

java
```
Node titleNode = xmlDoc.getElementsByTagName("title").item(0);
Node textNode = titleNode.getChildNodes().item(0);
titleNode.removeChild(textNode); // CORRECT: removing from its actual parent
```

Fix Explained (Roman Urdu): `removeChild()` hamesha us node par call karo jo remove hone wale node ka direct parent ho — is case mein `titleNode` par.

Mistake 2: Casting error when using removeAttribute()

Buggy Code:

java
```
Node bookNode = xmlDoc.getElementsByTagName("book").item(0);
bookNode.removeAttribute("category"); // WRONG: Node does not have removeAttribute()
```

What's Wrong (Roman Urdu): `getElementsByTagName() Node` type return karta hai, lekin `removeAttribute()` method sirf `Element` type par exist karta hai. Bina cast kiye call karne se compilation error aayegi.

Fixed Code:

java
```
Element bookElement = (Element) xmlDoc.getElementsByTagName("book").item(0);
bookElement.removeAttribute("category"); // CORRECT: cast to Element first
```

Fix Explained (Roman Urdu): Node ko pehle `(Element)` mein cast karo, tab hi `removeAttribute()` available hoga.

Mistake 3: replaceChild() parameters ka galat order

Buggy Code:

java
```
Node root = xmlDoc.getDocumentElement();
Node oldBook = xmlDoc.getElementsByTagName("book").item(0);
Element newBook = xmlDoc.createElement("book");
root.replaceChild(oldBook, newBook); // WRONG: parameters reversed
```

What's Wrong (Roman Urdu): `replaceChild(newChild, oldChild)` ka order hai — pehle naya node, phir purana. Yahan student ne ulta kar diya, jis se ya toh wrong node replace hoga ya exception aayega.

Fixed Code:

java
```
Node root = xmlDoc.getDocumentElement();
Node oldBook = xmlDoc.getElementsByTagName("book").item(0);
Element newBook = xmlDoc.createElement("book");
root.replaceChild(newBook, oldBook); // CORRECT: new first, old second
```

Fix Explained (Roman Urdu): `replaceChild()` ka syntax yaad rakho: `parent.replaceChild(newNode, oldNode)` — naya pehle, purana baad mein.

Mistake 4: Null pointer when text node does not exist

Buggy Code:

java
```
Node titleNode = xmlDoc.getElementsByTagName("title").item(0);
Node textNode = titleNode.getChildNodes().item(0);
textNode.setNodeValue(""); // CRASH if title has no child text node
```

What's Wrong (Roman Urdu): Agar `<title>` element empty hai aur uska koi child text node nahi, toh `item(0)` null return karega, aur null par `setNodeValue()` call karne se `NullPointerException` aayegi.

Fixed Code:

java
```
Node titleNode = xmlDoc.getElementsByTagName("title").item(0);
if (titleNode.hasChildNodes()) {                    // check first
    Node textNode = titleNode.getChildNodes().item(0);
    textNode.setNodeValue("");                      // safe to call now
}
```

Fix Explained (Roman Urdu): Pehle `hasChildNodes()` se check karo ke text node exist karta hai ya nahi — phir safely access karo.

Conceptual Questions

Q: What will happen if you call `removeChild(y)` on node `x`, but `y` is not actually a direct child of `x`?

Answer: Java DOM mein agar tum `x.removeChild(y)` call karo aur `y` asal mein `x` ka direct child nahi hai, toh program ek `DOMException` throw karta hai jiska type `NOT_FOUND_ERR` hota hai — matlab runtime par error aayegi aur node remove nahi hoga.

- Jaise agar `y` kisi aur element ka child hai aur tum galti se `x.removeChild(y)` likhte ho, toh exception aayegi. Pehle `y.getParentNode()` se confirm karo ke parent sahi hai.

Q: What is the difference in behavior between using `removeChild()` on a text node versus setting `nodeValue` to an empty string?

Answer: `removeChild()` text node ko poori tarah parent se alag kar deta hai — matlab woh node DOM tree se delete ho jaata hai. Lekin `nodeValue = ""` set karne se text node DOM mein rehta hai, sirf uska content khaali ho jaata hai. Pehle method mein node gone, doosre mein node still there.

- Agar baad mein tum `getChildNodes().item(0)` call karo `removeChild()` ke baad, woh `null` return karega. Lekin `nodeValue = ""` ke baad `item(0)` ab bhi text node return karega — sirf empty content ke saath.

Lesson Quiz

1. What method is used to remove a child node from its parent in XML DOM? (`removeChild()`)
2. Which property can be used to clear the text of a text node without removing the node itself? (`nodeValue`)
3. What type must a node be cast to before calling `removeAttribute()` on it in Java DOM? (`Element`)
4. In `replaceChild(newNode, oldNode)`, what does the first argument represent? (The new node that will replace the old one)
5. What exception is thrown if you try to use `removeChild()` on a node that is not a direct child of the parent? (`DOMException` with `NOT_FOUND_ERR`)

Lesson 29: Replacing Data in XML Text Nodes

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum apne phone mein ek message type kar rahe ho aur galti se pehle kuch words galat likh diye. Ab tum poora message delete nahi karte — sirf galat hissa select karke uski jagah sahi words likh dete ho. XML DOM ka `replaceData()` method bilkul yahi kaam karta hai — poore text node ko hataaye bina, sirf ek specific hissa replace kar deta hai.

Key Points:

1. replaceData(): Yeh method ek text node ke andar specific characters ko naye string se replace karta hai — poora node nahi badalta, sirf ek hissa.
2. offset: Yeh batata hai ke replacement kahan se shuru ho — counting zero se start hoti hai, jaise array index.
3. length: Yeh batata hai ke kitne characters replace karne hain — yani kitna hissa hatana hai.
4. string: Yeh woh naya text hai jo replace hone wali jagah insert hoga.
5. childNodes[0]: Text node usually element ka pehla child hota hai — `getElementsByTagName()` ke baad is se text node tak pahuncha jaata hai.
6. Zero-based indexing: offset mein pehla character position 0 hoti hai, 1 nahi — yeh ek common confusion point hai.

Example:

Socho tumhari teacher ne whiteboard par likha: "Mushkil Sawaal" — lekin baad mein unhon ne decide kiya ke sirf pehla word badalna hai. Unhon ne "Mushkil" ko mita ke "Aasaan" likh diya, baaki "Sawaal" waise hi raha. `replaceData()` bilkul yahi karta hai — text node mein sirf ek specific jagah ka data change karo, baaki content untouched rehta hai. Offset batata hai kahan se shuru karo, length batata hai kitna mito, aur string batata hai kya likhna hai.

Code Session

PART A — Working Code Example:

```
import javax.xml.parsers.*;
import org.w3c.dom.*;
import java.io.*;
```

```
public class ReplaceDataDemo {
    public static void main(String[] args) throws Exception {

        // Set up the XML parser
        DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();

        // Simulate books.xml as a string
        String xmlContent = "<bookstore>" +
                "<book>" +
                "<title>Everyday Italian</title>" +
                "</book>" +
                "<book>" +
                "<title>Learning XML</title>" +
                "</book>" +
                "</bookstore>";

        // Parse the XML content into a Document object
        Document xmlDoc = builder.parse(
            new ByteArrayInputStream(xmlContent.getBytes())
        );

        // --- DEMO 1: replaceData() on first title ---
        // Get the text node of the first <title> element
        Text firstTitleText = (Text) xmlDoc.getElementsByTagName("title")
                                          .item(0)
                                          .getChildNodes()
                                          .item(0);

        System.out.println("Before replaceData: " +
firstTitleText.getNodeValue());

        // Replace first 8 characters starting at offset 0 with "Easy"
        // "Everyday" has 8 characters → it will be replaced with "Easy"
        firstTitleText.replaceData(0, 8, "Easy");  // offset=0, length=8,
newString="Easy"

        System.out.println("After replaceData:  " +
firstTitleText.getNodeValue());

        // --- DEMO 2: replaceData() in the middle of a string ---
        // Get text node of second <title>: "Learning XML"
        Text secondTitleText = (Text) xmlDoc.getElementsByTagName("title")
                                            .item(1)
                                            .getChildNodes()
                                            .item(0);

        System.out.println("\nBefore replaceData: " +
secondTitleText.getNodeValue());

        // Replace "XML" (starts at offset 9, length 3) with "Java"
        secondTitleText.replaceData(9, 3, "Java"); // offset=9, length=3,
newString="Java"

        System.out.println("After replaceData:  " +
secondTitleText.getNodeValue());
```

```
        // --- DEMO 3: replaceData() replacing only part of a word ---
        // Demonstrating offset in the middle — "Easy Italian" → "Easy
Korean"
        Text demoText = (Text) xmlDoc.getElementsByTagName("title")
                                     .item(0)
                                     .getChildNodes()
                                     .item(0);

        System.out.println("\nBefore replaceData: " +
demoText.getNodeValue());

        // "Easy Italian" → replace "Italian" (offset=5, length=7) with
"Korean"
        demoText.replaceData(5, 7, "Korean"); // offset=5, length=7,
newString="Korean"

        System.out.println("After replaceData:  " + demoText.getNodeValue());
    }
}
```

Code Walkthrough (Roman Urdu):

- Pehle DocumentBuilder use karke XML string ko parse kiya — yeh hamare `books.xml` ki simulation hai jisme do book titles hain: "Everyday Italian" aur "Learning XML".
- DEMO 1 mein pehle title ka text node liya aur `replaceData(0, 8, "Easy")` call kiya. Offset 0 matlab start se shuru karo, length 8 matlab 8 characters hataao ("Everyday" = 8 chars), aur unki jagah "Easy" daal do — result bana "Easy Italian".
- DEMO 2 mein doosre title "Learning XML" mein "XML" replace kiya. "Learning " ke 9 characters hain (L-e-a-r-n-i-n-g-space), toh offset 9 hai, "XML" ke 3 characters hain toh length 3, aur naya string "Java" — result bana "Learning Java".
- DEMO 3 mein "Easy Italian" ka "Italian" replace kiya "Korean" se. "Easy " = 5 characters, isliye offset 5, "Italian" = 7 characters isliye length 7 — result "Easy Korean".
- Har demo mein pehle aur baad ka value print kiya taake change clearly nazar aaye.

Output:

```
Before replaceData: Everyday Italian
After replaceData:  Easy Italian

Before replaceData: Learning XML
After replaceData:  Learning Java

Before replaceData: Easy Italian
After replaceData:  Easy Korean
```

Yeh output confirm karta hai ke `replaceData()` sirf specified range ke characters replace karta hai — baaki text bilkul untouched rehta hai, aur offset + length ka combination exactly control karta hai ke kya badlega.

Common Mistakes — Error Finder Session

Mistake 1: Calling replaceData() on Element instead of Text node

Buggy Code:

```
// Trying to call replaceData() directly on the Element node
Element titleElement = (Element)
xmlDoc.getElementsByTagName("title").item(0);
titleElement.replaceData(0, 8, "Easy"); // WRONG: Element has no
replaceData()
```

What's Wrong (Roman Urdu): `replaceData()` method `Text` node par hota hai, `Element` node par nahi. Student ne `getElementsByTagName()` se mila hua element directly use kar liya bina uske child text node tak pahunche — yeh compilation error dega.

Fixed Code:

```
// Correctly get the Text child node first
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0)
                              .getChildNodes()
                              .item(0);    // text node is the first child
titleText.replaceData(0, 8, "Easy");       // CORRECT: called on Text node
```

Fix Explained (Roman Urdu): Pehle `getChildNodes().item(0)` se text node tak pahuncho, phir usse `Text` mein cast karo — tab `replaceData()` available hoga.

Mistake 2: offset + length exceeds the string length

Buggy Code:

```
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0).getChildNodes().item(0);
// "Everyday Italian" has 16 characters — offset 10, length 20 goes out of
bounds
titleText.replaceData(10, 20, "Cooking"); // WRONG: length too large
```

What's Wrong (Roman Urdu): Agar `offset + length` string ki actual length se zyada ho jaaye, toh `DOMException` throw ho sakti hai ya unexpected behavior aata hai. "Every day Italian" sirf 16 characters ki hai — offset 10 par 20 characters hain hi nahi.

Fixed Code:

```
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0).getChildNodes().item(0);
// "Italian" starts at index 9, has 7 characters — safe range
```

```
titleText.replaceData(9, 7, "Cooking"); // CORRECT: offset=9, length=7
```

Fix Explained (Roman Urdu): Replace karne se pehle string ki actual length count karo aur ensure karo ke `offset + length` us se zyada na ho.

Mistake 3: Treating offset as 1-based instead of 0-based

Buggy Code:

```
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0).getChildNodes().item(0);
// "Everyday Italian" — student thinks first character is at position 1
titleText.replaceData(1, 8, "Easy"); // WRONG: misses the first character 'E'
```

What's Wrong (Roman Urdu): Student ne socha ke offset 1 se start hoti hai jaise normal counting hoti hai — lekin DOM mein offset zero-based hai. Offset 1 dene se pehla character 'E' skip ho jaata hai aur "veryday" replace hoti hai.

Fixed Code:

```
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0).getChildNodes().item(0);
// Offset 0 = first character, length 8 = "Everyday" (8 chars)
titleText.replaceData(0, 8, "Easy"); // CORRECT: starts from index 0
```

Fix Explained (Roman Urdu): Hamesha yaad rakho — `replaceData()` mein offset zero se shuru hoti hai, exactly jaise Java array index karta hai.

Mistake 4: Wrong item() index to get the text node

Buggy Code:

```
// Using item(1) instead of item(0) to get the text node
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0)
                              .getChildNodes()
                              .item(1); // WRONG: text node is at index 0
titleText.replaceData(0, 8, "Easy");     // NullPointerException likely
```

What's Wrong (Roman Urdu): Simple element ke andar text node usually `item(0)` par hota hai — pehla aur akela child. `item(1)` null return karta hai agar koi doosra child nahi hai, aur phir `replaceData()` call karne par `NullPointerException` aati hai.

Fixed Code:

```
Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                              .item(0)
                              .getChildNodes()
                              .item(0);   // CORRECT: text is the first
child
titleText.replaceData(0, 8, "Easy");
```

Fix Explained (Roman Urdu): Text node hamesha `getChildNodes().item(0)` se milta hai jab element mein sirf text content ho — `item(1)` sirf tab use karo jab tumhe pata ho ke pehla child kuch aur hai.

Conceptual Questions

Q: Given the text node value `"Learning XML Today"`, what will be the result after calling `replaceData(9, 3, "Java")`?

Answer: `"Learning "` ke 9 characters hain (L-e-a-r-n-i-n-g-space), toh offset 9 par `"XML"` start hota hai. Length 3 matlab 3 characters hataao yaani `"XML"` hata, aur uski jagah `"Java"` aayega. Baaki string `" Today"` waise hi rahega — final result hoga `"Learning Java Today"`.

- Is example mein `" Today"` ka koi character affect nahi hota kyunke `replaceData()` sirf specified range ke characters ko target karta hai — us se pehle aur baad ka content protected rehta hai.

Q: What is the difference between using `nodeValue = "new text"` and `replaceData(0, oldLength, "new text")` on a text node?

Answer: `nodeValue` set karna poore text node ka content ek hi baar mein replace kar deta hai — jo bhi pehle tha woh poora chala jaata hai. Lekin `replaceData()` zyada precise hai — tum control kar sakte ho ke kahan se kitne characters replace hon, baaki text unchanged rehta hai.

- Jaise `"Hello World"` par `nodeValue = "Hi"` karne se sirf `"Hi"` reh jaayega. Lekin `replaceData(0, 5, "Hi")` karne se result hoga `"Hi World"` — sirf `"Hello"` replace hua, `" World"` intact raha.

Lesson Quiz

1. What are the three parameters of the `replaceData()` method in order? (offset, length, string)

2. In `replaceData(0, 8, "Easy")` applied to `"Everyday Italian"`, what is the resulting string? (`"Easy Italian"`)
3. What type of node must `replaceData()` be called on — `Element` or `Text`? (`Text`)
4. If a text node contains `"Hello"` and you call `replaceData(2, 2, "XY")`, what is the output? (`"HeXYo"`)
5. What will happen if you call `getChildNodes().item(1)` on a `<title>` element that only contains a text node? (`null` is returned, leading to `NullPointerException` if used)

Lesson 30: Creating, Adding, Copying Nodes and XML Node Types

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum ek school magazine bana rahe ho. Kabhi naya article add karte ho (appendChild), kabhi kisi specific page se pehle kuch insert karte ho (insertBefore), kabhi koi secret note likhte ho jo readers ke liye nahi hoti (Comment node), kabhi special raw content daalni hoti hai jise parse na kiya jaaye (CDATA), aur kabhi ek poora section copy karke doosri jagah paste karte ho (cloneNode). XML DOM mein yeh sab kaam alag methods se hote hain — aaj hum yahi seekhenge.

Key Points:

1. createCDATASection(): Yeh method ek naya CDATA section node banata hai jiska content XML parser parse nahi karta — special characters jaise `&` aur `<` safely rakhe ja sakte hain.
2. createComment(): Yeh method ek comment node banata hai — yeh sirf developers ke liye hota hai, actual XML data mein count nahi hota.
3. appendChild(): Yeh method ek naya child node existing node ke end mein add karta hai — jo bhi existing children hain un ke baad aata hai.
4. insertBefore(): Yeh method ek naya node kisi specified child node se pehle insert karta hai — position control karna ho toh yeh use hota hai.
5. setAttribute(): Yeh method kisi element par ek attribute set karta hai — agar attribute pehle se exist kare toh uski value overwrite ho jaati hai.
6. insertData(): Yeh method existing text node ke andar specific offset par naya text insert karta hai — do parameters hain: offset aur string.
7. cloneNode(true/false): Yeh method kisi node ki copy banata hai — `true` pass karne par saare children bhi copy hote hain, `false` par sirf woh node akela copy hota hai.
8. Node Types: DOM mein alag types ke nodes hote hain jaise Document, Element, Text, Comment, CDATASection — har type ke allowed children different hote hain.

Example:

Socho tum apni class ki attendance sheet manage kar rahe ho. Tum ek naya student ka naam end mein add karte ho (appendChild). Phir tumhe pata chala ek aur student pehle wale se pehle aana chahiye tha — tum usse beech mein insert karte ho (insertBefore). Ek student ka special note likhna tha jo official record mein nahi dikhna — tum comment likh dete ho (createComment). Aur ek purani sheet ki exact copy banana thi — tum clone kar lete ho (cloneNode). Yeh sab XML DOM ke methods exactly isi tarah kaam karte hain.

Code Session

PART A — Working Code Example:

```
import javax.xml.parsers.*;
import org.w3c.dom.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.*;

public class NodeOperationsDemo {
    public static void main(String[] args) throws Exception {

        // Parse an XML string simulating books.xml
        DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();

        String xmlContent = "<bookstore>" +
                "<book category=\"cooking\">" +
                "<title>Everyday Italian</title>" +
                "</book>" +
                "<book category=\"children\">" +
                "<title>Harry Potter</title>" +
                "</book>" +
                "</bookstore>";

        Document xmlDoc = builder.parse(
            new ByteArrayInputStream(xmlContent.getBytes())
        );

        // --- DEMO 1: createCDATASection() ---
        // Create a CDATA node — content will NOT be parsed by XML parser
        CDATASection newCDATA = xmlDoc.createCDATASection("Special Offer &
Book Sale");
        xmlDoc.getElementsByTagName("book").item(0).appendChild(newCDATA);
        System.out.println("CDATA added to first book.");

        // --- DEMO 2: createComment() ---
        // Create a comment node — invisible to end users, for developers
only
        Comment newComment = xmlDoc.createComment("Revised March 2015");
        xmlDoc.getElementsByTagName("book").item(0).appendChild(newComment);
```

```
        System.out.println("Comment added to first book.");

        // --- DEMO 3: appendChild() with text value ---
        // Create <edition> element, give it a text value, append to first
book
        Element newEle = xmlDoc.createElement("edition");           // create
element node
        Text newText = xmlDoc.createTextNode("first");              // create
text node
        newEle.appendChild(newText);                                // add
text inside element
        xmlDoc.getElementsByTagName("book").item(0).appendChild(newEle); //
add to first book
        System.out.println("Edition element appended.");

        // --- DEMO 4: insertBefore() ---
        // Create a new <book> and insert it before the second existing book
        Element newBook = xmlDoc.createElement("book");             // new
book element
        Element newBookTitle = xmlDoc.createElement("title");
        newBookTitle.appendChild(xmlDoc.createTextNode("New Arrival"));
        newBook.appendChild(newBookTitle);

        Node root = xmlDoc.getDocumentElement();                    //
<bookstore> root
        Node secondBook = xmlDoc.getElementsByTagName("book").item(1); //
target position
        root.insertBefore(newBook, secondBook);                     // insert
before second book
        System.out.println("New book inserted before second book.");

        // --- DEMO 5: setAttribute() ---
        // Set attribute "edition" on the first book element
        Element firstBook = (Element)
xmlDoc.getElementsByTagName("book").item(0);
        firstBook.setAttribute("edition", "first");                 //
add/overwrite attribute
        System.out.println("Attribute 'edition' set: " +
            firstBook.getAttribute("edition"));

        // --- DEMO 6: insertData() on text node ---
        // Insert "Easy " at the beginning of the first title's text
        Text titleText = (Text) xmlDoc.getElementsByTagName("title")
                                      .item(0).getChildNodes().item(0);
        System.out.println("Before insertData: " + titleText.getNodeValue());
        titleText.insertData(0, "Easy ");                           // insert
at offset 0
        System.out.println("After insertData:  " + titleText.getNodeValue());

        // --- DEMO 7: cloneNode(true) ---
        // Clone the first book including all its children
        Node oldNode = xmlDoc.getElementsByTagName("book").item(0); // node
to copy
        Node clonedNode = oldNode.cloneNode(true);                  // deep
clone
        xmlDoc.getDocumentElement().appendChild(clonedNode);        // append
clone to root
```

```
        System.out.println("Node cloned and appended.");

        // Print all title values to show final state
        System.out.println("\n--- All Titles in Final XML ---");
        NodeList titles = xmlDoc.getElementsByTagName("title");
        for (int i = 0; i < titles.getLength(); i++) {
            System.out.println("Title " + (i+1) + ": " +
                titles.item(i).getTextContent());
        }
    }
}
```

Code Walkthrough (Roman Urdu):

- DEMO 1 mein `createCDATASection()` se ek CDATA node banaya jisme `&` jaisa special character safely rakha — yeh XML parser parse nahi karega, raw text ki tarah treat hoga. Phir `appendChild()` se pehle book mein add kiya.
- DEMO 2 mein `createComment()` se ek comment node banaya — yeh node XML document mein dikhta hai lekin actual data nahi hota, sirf developer notes ke liye hota hai.
- DEMO 3 mein `createElement("edition")` se element banaya, `createTextNode("first")` se text banaya, text ko element ke andar dala `appendChild` se, phir poora element pehle book mein daal diya — yeh do-step pattern bohot common hai.
- DEMO 4 mein `insertBefore(newBook, secondBook)` ne naya book element second book se pehle insert kiya — position important thi isliye `appendChild` ki jagah yeh method use kiya.
- DEMO 5 mein `setAttribute("edition", "first")` ne pehle book par ek naya attribute add kiya — agar pehle se hota toh overwrite ho jaata.
- DEMO 6 mein `insertData(0, "Easy ")` ne text node ke bilkul start par "Easy " insert kiya — "Everyday Italian" ban gaya "Easy Everyday Italian".
- DEMO 7 mein `cloneNode(true)` ne pehle book ki deep copy banayi — `true` parameter se matlab saare child nodes bhi clone hue — phir root mein append kar diya.

Output:

```
CDATA added to first book.
Comment added to first book.
Edition element appended.
New book inserted before second book.
Attribute 'edition' set: first
Before insertData: Everyday Italian
After insertData:  Easy Everyday Italian
Node cloned and appended.

--- All Titles in Final XML ---
Title 1: Easy Everyday Italian
Title 2: New Arrival
Title 3: Harry Potter
Title 4: Easy Everyday Italian
```

Yeh output confirm karta hai ke sab methods apna kaam sahi kar rahe hain — CDATA aur comment add hue, edition element append hua, naya book beech mein insert hua, attribute set hua, text mein insert hua, aur clone duplicate ban gaya.

Common Mistakes — Error Finder Session

Mistake 1: cloneNode() without parameter — forgetting children

Buggy Code:

```
// Cloning without specifying deep copy
Node oldNode = xmlDoc.getElementsByTagName("book").item(0);
Node clonedNode = oldNode.cloneNode(); // WRONG: missing true/false parameter
xmlDoc.getDocumentElement().appendChild(clonedNode);
```

What's Wrong (Roman Urdu): `cloneNode()` ko `true` ya `false` parameter dena zaroori hai. Bina parameter ke Java mein compile error aayega kyunke yeh method ek boolean argument expect karta hai. Aksar students yeh parameter bhool jaate hain.

Fixed Code:

```
Node oldNode = xmlDoc.getElementsByTagName("book").item(0);
Node clonedNode = oldNode.cloneNode(true);  // CORRECT: true = deep clone
with children
xmlDoc.getDocumentElement().appendChild(clonedNode);
```

Fix Explained (Roman Urdu): `cloneNode(true)` pass karo agar children bhi chahiye, `cloneNode(false)` pass karo agar sirf woh ek node chahiye bina kisi child ke.

Mistake 2: insertBefore() parameters reversed

Buggy Code:

```
Node root = xmlDoc.getDocumentElement();
Node secondBook = xmlDoc.getElementsByTagName("book").item(1);
Element newBook = xmlDoc.createElement("book");
root.insertBefore(secondBook, newBook); // WRONG: parameters are reversed
```

What's Wrong (Roman Urdu): `insertBefore(newNode, referenceNode)` mein pehla parameter woh node hota hai jo insert karna hai, aur doosra woh node jiske pehle insert karna hai. Student ne dono ulte kar diye — iska result galat insertion ya DOMException hoga.

Fixed Code:

```
Node root = xmlDoc.getDocumentElement();
Node secondBook = xmlDoc.getElementsByTagName("book").item(1);
Element newBook = xmlDoc.createElement("book");
root.insertBefore(newBook, secondBook); // CORRECT: new node first, reference
second
```

Fix Explained (Roman Urdu): Yaad rakho — `insertBefore(jo_dalna_hai, jisse_pehle_dalna_hai)` — pehla argument naya node, doosra reference node.

Mistake 3: appendChild() before building the element properly

Buggy Code:

```
// Appending element to document BEFORE adding text to it
Element newEle = xmlDoc.createElement("edition");
// Forgot to create and attach text node first
xmlDoc.getElementsByTagName("book").item(0).appendChild(newEle);
Text newText = xmlDoc.createTextNode("first"); // Too late — added after
append
newEle.appendChild(newText);
```

What's Wrong (Roman Urdu): Yeh code crash nahi karega lekin logic galat hai — student ne pehle empty element append kar diya, phir text add kiya. Yeh acha practice nahi hai aur complex cases mein unexpected results de sakta hai.

Fixed Code:

```
Element newEle = xmlDoc.createElement("edition");
Text newText = xmlDoc.createTextNode("first"); // create text first
newEle.appendChild(newText);                   // add text to element
xmlDoc.getElementsByTagName("book")
      .item(0).appendChild(newEle);            // THEN append complete
element
```

Fix Explained (Roman Urdu): Hamesha pehle apna node fully build karo — text add karo, attributes set karo — phir use tree mein append karo. Sequence matter karta hai.

Mistake 4: setAttribute() called on Node type instead of Element

Buggy Code:

```
// getElementsByTagName returns NodeList — item() returns Node, not Element
Node bookNode = xmlDoc.getElementsByTagName("book").item(0);
bookNode.setAttribute("edition", "first"); // WRONG: Node has no
setAttribute()
```

What's Wrong (Roman Urdu): `setAttribute()` method `Element` interface ka part hai, `Node` ka nahi. `item()` method `Node` return karta hai — bina cast kiye directly `setAttribute()` call karna compilation error deta hai.

Fixed Code:

```
// Cast to Element before calling setAttribute()
Element bookElement = (Element) xmlDoc.getElementsByTagName("book").item(0);
bookElement.setAttribute("edition", "first"); // CORRECT: Element has
setAttribute()
```

Fix Explained (Roman Urdu): Jab bhi attribute methods use karni ho, node ko pehle `(Element)` mein cast karo — tab hi `setAttribute()` aur `getAttribute()` available honge.

Conceptual Questions

Q: What is the difference in behavior between `cloneNode(true)` and `cloneNode(false)` when copying a `<book>` element that contains `<title>` and `<author>` child elements?

Answer: `cloneNode(true)` deep copy karta hai — matlab `<book>` ke saath uske andar ke `<title>` aur `<author>` elements bhi copy hote hain. Lekin `cloneNode(false)` sirf `<book>` tag ki shallow copy karta hai — andar ke koi bhi children copy nahi hote, sirf khali element milta hai.

- Agar tum `cloneNode(false)` se mila hua node append karo aur phir `getElementsByTagName("title")` count karo, toh count same rahega. Lekin `cloneNode(true)` ke baad count badh jaayega kyunke cloned children bhi DOM mein aa jaate hain.

Q: What will happen if you call `x.insertBefore(newNode, null)` — where does the new node get inserted?

Answer: Jab `insertBefore()` ka doosra parameter `null` ho, toh naya node parent `x` ke sabse aakhri child ke baad add ho jaata hai — yeh exactly `appendChild()` jaisa behavior hai. Dono methods null reference hone par same position par insert karte hain.

- Matlab `x.insertBefore(newNode, null)` aur `x.appendChild(newNode)` dono ek hi kaam karte hain — naya node end mein jaata hai. `insertBefore` ka asli fayda tab hai jab tum specific reference node dete ho taake position control ho.

Lesson Quiz

1. Which method creates a text node that will NOT be parsed by the XML parser? (`createCDATASection()`)
2. What does `cloneNode(false)` return when called on a `<book>` element containing child elements? (A shallow copy of `<book>` only, with no children)
3. What will `x.insertBefore(newNode, null)` do to the new node? (Append it after the last existing child of `x`, same as `appendChild()`)
4. If `setAttribute("edition", "second")` is called on an element that already has `edition="first"`, what is the result? (The value is overwritten to `"second"`)
5. In `insertData(0, "Easy ")` applied to a text node with value `"Italian"`, what is the resulting text? (`"Easy Italian"`)

Lesson 31: Introduction to JAXP — Java API for XML Processing

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum ek bada parcel receive karte ho jisme bohot saari cheezein hain. Ab tumhare paas do tarike hain iske saath kaam karne ke — ya toh poora parcel ek baar mein kholo aur saari cheezein table par rakh lo taake easily dhundo (DOM), ya phir parcel ko thoda thoda kholte jao aur har cheez nikalte waqt foran koi action lo bina sab kuch yaad rakhe (SAX). JAXP ek aisi umbrella API hai jo tumhe yeh dono tarike choose karne ki azaadi deti hai apne Java program mein.

Key Points:

1. JAXP (Java API for XML Processing): Yeh ek Java API hai jo XML data process karne ke liye use hoti hai — yeh SAX, DOM, XSLT, aur StAX sab ko ek jagah support karti hai.
2. SAX (Simple API for XML): Yeh event-driven aur serial-access parsing approach hai — XML ko element by element padhta hai bina poori file memory mein rakhe — server-side applications ke liye best hai.
3. DOM (Document Object Model): Yeh poori XML file ko memory mein ek tree structure ki tarah load karta hai — use karna aasaan hai lekin zyada CPU aur memory consume karta hai.
4. XSLT (Extensible Stylesheet Language Transformations): Yeh XML data ko doosre formats mein convert karne ke liye use hota hai jaise HTML ya doosra XML document.
5. StAX (Streaming API for XML): Yeh SAX se simple programming model deta hai aur DOM se zyada efficient memory management — yeh pull-parsing approach use karta hai.
6. Pluggability Layer: JAXP ka yeh feature allow karta hai ke tum koi bhi XML-compliant parser plug in kar sako — vendor-independence milti hai.

7. javax.xml.parsers: Yeh package SAX aur DOM parsers ke liye common interface provide karta hai — SAXParserFactory aur DocumentBuilderFactory yahan se milti hain.
8. SAXParserFactory: Yeh class SAX parser ka instance banati hai — system property `javax.xml.parsers.SAXParserFactory` se determine hoti hai ke kaunsa parser use hoga.

Example:

Socho tum ek badi library ke manager ho jahan hazaron books hain. Agar koi customer aata hai aur kehta hai "mujhe sirf Science section ki books chahiye" — tum puri library scan karte jaate ho aur jab bhi Science book milti hai foran usse note kar lete ho bina baaki books yaad rakhe — yeh SAX hai. Lekin agar koi student aata hai jo poori library ki list ek baar mein dekh ke compare karna chahta hai — tum saari books ki list ek bade table par rakh dete ho — yeh DOM hai. JAXP ek aisa system hai jo tumhe dono approaches mein se choose karne deta hai library manage karne ke liye.

Code Session

PART A — Working Code Example:

```
import javax.xml.parsers.*;   // JAXP core package — SAXParserFactory,
DocumentBuilderFactory
import org.xml.sax.*;          // SAX API — Attributes, SAXException
import org.xml.sax.helpers.*;  // DefaultHandler for SAX callbacks
import org.w3c.dom.*;          // DOM API — Document, NodeList, Element
import java.io.*;

// ============================================
// PART 1: SAX Parsing using JAXP
// ============================================
class SAXDemo extends DefaultHandler {

    // Called when parser finds an opening tag like <book>
    @Override
    public void startElement(String uri, String localName,
                             String qName, Attributes attributes) {
        System.out.println("SAX — Start Element: <" + qName + ">");
    }

    // Called when parser finds text content inside an element
    @Override
    public void characters(char[] ch, int start, int length) {
        String text = new String(ch, start, length).trim();
        if (!text.isEmpty()) {
            System.out.println("SAX — Text Content: " + text);
        }
```

```
    }

    // Called when parser finds a closing tag like </book>
    @Override
    public void endElement(String uri, String localName, String qName) {
        System.out.println("SAX — End Element: </" + qName + ">");
    }
}

public class JAXPDemo {
    public static void main(String[] args) throws Exception {

        // Simulated XML content (represents books.xml)
        String xmlContent = "<bookstore>" +
                "<book>" +
                "<title>Everyday Italian</title>" +
                "<author>Giada</author>" +
                "</book>" +
                "</bookstore>";

        byte[] xmlBytes = xmlContent.getBytes();

        // ============================================
        // SAX PARSING with JAXP
        // ============================================
        System.out.println("===== SAX Parsing =====");

        // Step 1: Get a SAXParserFactory instance
        SAXParserFactory saxFactory = SAXParserFactory.newInstance();

        // Step 2: Create SAXParser from the factory
        SAXParser saxParser = saxFactory.newSAXParser();

        // Step 3: Create our custom handler that reacts to SAX events
        SAXDemo handler = new SAXDemo();

        // Step 4: Parse — parser reads XML and fires events to handler
        saxParser.parse(new ByteArrayInputStream(xmlBytes), handler);

        // ============================================
        // DOM PARSING with JAXP
        // ============================================
        System.out.println("\n===== DOM Parsing =====");

        // Step 1: Get a DocumentBuilderFactory instance
        DocumentBuilderFactory domFactory =
DocumentBuilderFactory.newInstance();

        // Step 2: Create DocumentBuilder from the factory
        DocumentBuilder domBuilder = domFactory.newDocumentBuilder();

        // Step 3: Parse XML into a Document tree — entire tree loaded in
memory
        Document document = domBuilder.parse(
            new ByteArrayInputStream(xmlBytes)
        );
```

```
        // Step 4: Navigate the tree and print title
        NodeList titles = document.getElementsByTagName("title");
        for (int i = 0; i < titles.getLength(); i++) {
            System.out.println("DOM — Title found: " +
                titles.item(i).getTextContent());
        }

        System.out.println("\nDOM — Total book elements: " +
            document.getElementsByTagName("book").getLength());
    }
}
```

Code Walkthrough (Roman Urdu):

- SAXDemo class mein `DefaultHandler` extend kiya — yeh JAXP ka built-in class hai jo SAX ke callback methods provide karta hai. Humne `startElement`, `characters`, aur `endElement` override kiye — yeh teeno tab call hote hain jab parser XML mein koi opening tag, text, ya closing tag mile.
- SAX parsing mein pehle `SAXParserFactory.newInstance()` se factory li, phir `newSAXParser()` se parser banaya — yeh JAXP ka pluggability layer hai jo actual parser implementation se independent rakhta hai.
- `saxParser.parse()` call karne par parser XML ko serial taur par padhta hai aur har event par humara handler automatically call hota hai — koi poori tree memory mein nahi banti.
- DOM parsing mein `DocumentBuilderFactory.newInstance()` se factory li, phir `DocumentBuilder` banaya, phir `parse()` se poori XML memory mein `Document` object ki form mein load ho gayi.
- DOM mein `getElementsByTagName("title")` se directly saare title elements ek baar mein mile — yeh SAX se zyada convenient hai lekin poori file memory mein thi.

Output:

```
===== SAX Parsing =====
SAX — Start Element: <bookstore>
SAX — Start Element: <book>
SAX — Start Element: <title>
SAX — Text Content: Everyday Italian
SAX — End Element: </title>
SAX — Start Element: <author>
SAX — Text Content: Giada
SAX — End Element: </author>
SAX — End Element: </book>
SAX — End Element: </bookstore>

===== DOM Parsing =====
DOM — Title found: Everyday Italian
DOM — Total book elements: 1
```

Yeh output confirm karta hai ke SAX sequential events fire karta hai jaise parser aage badhta hai — jabke DOM parsing ke baad seedha koi bhi element access kiya ja sakta hai tree se, kyunke poori structure memory mein available hai.

Common Mistakes — Error Finder Session

Mistake 1: Forgetting to call newSAXParser() — using factory directly

Buggy Code:

```
SAXParserFactory saxFactory = SAXParserFactory.newInstance();
// Student tries to call parse() directly on factory
saxFactory.parse(new ByteArrayInputStream(xmlBytes), handler); // WRONG
```

What's Wrong (Roman Urdu): `SAXParserFactory` sirf parser banane ka kaam karti hai — iske paas khud `parse()` method nahi hota. Student ne factory aur parser ko confuse kar liya — pehle `newSAXParser()` se actual parser banana zaroori hai.

Fixed Code:

```
SAXParserFactory saxFactory = SAXParserFactory.newInstance();
SAXParser saxParser = saxFactory.newSAXParser(); // get actual parser first
saxParser.parse(new ByteArrayInputStream(xmlBytes), handler); // CORRECT
```

Fix Explained (Roman Urdu): Factory se hamesha pehle parser banao `newSAXParser()` se, phir us parser par `parse()` call karo — yeh do-step process zaroori hai.

Mistake 2: Not extending DefaultHandler for SAX callback

Buggy Code:

```
// Handler class does not extend DefaultHandler
class MyHandler {
    public void startElement(String uri, String localName,
                             String qName, Attributes attributes) {
        System.out.println("Element: " + qName);
    }
}
// ...
saxParser.parse(new ByteArrayInputStream(xmlBytes), new MyHandler()); //
WRONG
```

What's Wrong (Roman Urdu): `saxParser.parse()` ek `DefaultHandler` ya `HandlerBase` type ka object expect karta hai. Agar class `DefaultHandler` extend nahi karti toh compile error aayega kyunke type match nahi karta.

Fixed Code:

```
// Must extend DefaultHandler to be accepted by SAX parser
```

```
class MyHandler extends DefaultHandler {
    @Override
    public void startElement(String uri, String localName,
                             String qName, Attributes attributes) {
        System.out.println("Element: " + qName);
    }
}
```

Fix Explained (Roman Urdu): SAX handler class ko hamesha `DefaultHandler` extend karna chahiye — tabhi parser usse apne callback events ke liye accept karega.

Mistake 3: Using DOM for large files — memory error

Buggy Code:

```
// Trying to DOM-parse a 500MB XML file — bad choice
DocumentBuilderFactory domFactory = DocumentBuilderFactory.newInstance();
DocumentBuilder domBuilder = domFactory.newDocumentBuilder();
Document document = domBuilder.parse(new File("huge_data.xml")); //
OutOfMemoryError risk
System.out.println("Parsed successfully"); // May never reach here
```

What's Wrong (Roman Urdu): DOM poori XML file ek baar mein memory mein load karta hai. 500MB jaisi badi file ke liye yeh `OutOfMemoryError` de sakta hai kyunke JVM ke paas itni heap space nahi hoti. Badi files ke liye SAX better choice hai.

Fixed Code:

```
// Use SAX for large files — processes element by element, low memory usage
SAXParserFactory saxFactory = SAXParserFactory.newInstance();
SAXParser saxParser = saxFactory.newSAXParser();
saxParser.parse(new File("huge_data.xml"), new DefaultHandler() {
    @Override
    public void startElement(String u, String l,
                             String qName, Attributes a) {
        System.out.println("Found: " + qName); // processes one at a time
    }
});
```

Fix Explained (Roman Urdu): Badi XML files ke liye hamesha SAX choose karo — woh ek waqt mein sirf ek element padhta hai aur memory efficient rehta hai.

Mistake 4: Accessing DOM tree before parsing

Buggy Code:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
```

```
DocumentBuilder builder = factory.newDocumentBuilder();
Document document = builder.newDocument(); // creates empty document

// Trying to access elements before parsing any XML
NodeList titles = document.getElementsByTagName("title"); // returns empty
list
System.out.println("Count: " + titles.getLength()); // prints 0 — confuses
student
```

What's Wrong (Roman Urdu): `builder.newDocument()` sirf ek khali Document banata hai — koi XML parse nahi kiya gaya. Student confuse ho jaata hai ke elements kyun nahi mil rahe. Actual XML parse karne ke liye `builder.parse()` call karna zaroori hai.

Fixed Code:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
// Must parse actual XML to get a populated document tree
Document document = builder.parse(
    new
ByteArrayInputStream("<bookstore><title>X</title></bookstore>".getBytes())
);
NodeList titles = document.getElementsByTagName("title");
System.out.println("Count: " + titles.getLength()); // prints 1
```

Fix Explained (Roman Urdu): DOM tree sirf tab banti hai jab `builder.parse()` call karo — `newDocument()` sirf empty document deta hai, koi parsing nahi hoti.

Conceptual Questions

Q: Given a 200MB XML file that needs to be processed on a server with limited RAM, which JAXP API should be used — SAX or DOM — and what happens differently in each case?

Answer: Is case mein SAX use karna chahiye. SAX poori file memory mein load nahi karta — woh ek ek element read karta hai aur event fire karta hai, toh RAM usage bohot kam rehti hai. DOM ke saath poori 200MB file memory mein load hogi jo `OutOfMemoryError` de sakti hai aur server slow ho jaayega.

- Practically: SAX mein `startElement()` callback tab fire hota hai jab parser ek tag padhta hai — woh data foran process hota hai aur discard. DOM mein `parse()` method return tab karta hai jab poori tree ban jaaye — tab tak saara data RAM mein hota hai.

Q: What is the role of `SAXParserFactory.newInstance()` in JAXP, and why is it used instead of directly creating a SAX parser object?

Answer: `SAXParserFactory.newInstance()` JAXP ka pluggability layer use karta hai — yeh system properties check karke decide karta hai ke kaunsa vendor ka SAX parser use karna hai. Agar tum directly parser object banao toh ek specific implementation se bound ho jaate ho — factory pattern use karne se tumhara code kisi bhi XML-compliant parser ke saath kaam karta hai bina code change kiye.

- Jaise Oracle ka parser alag hai aur Apache Xerces alag — `newInstance()` call karne par JAXP classpath ya system property se decide karta hai kaunsa use ho. Tumhara application code same rehta hai, sirf underlying parser swap ho sakta hai.

Lesson Quiz

1. Which JAXP parsing approach loads the entire XML document into memory as a tree structure? (DOM)
2. What class in JAXP is used to create a SAX parser instance? (`SAXParserFactory`)
3. Which JAXP API is preferred for high-performance server-side applications that process large XML files? (SAX)
4. What does the pluggability layer in JAXP allow developers to do? (Swap in any XML-compliant SAX or DOM parser implementation without changing application code)
5. Which package provides the `Document` class and DOM component classes used in JAXP? (`org.w3c.dom`)

Lesson 32: SAX Parser Internals and DOM API Packages

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum ek live cricket match sun rahe ho radio par. Commentator har ball par foran bolta hai — "wicket giri!", "four hua!", "over khatam!" — tum poora match yaad nahi rakhte, sirf jo abhi hua woh sun ke react karte ho. Yeh SAX parsing hai — events fire hoti hain aur tum react karte ho. Lekin DOM aise hai jaise tum match ka poora scorecard print karke table par rakh lo aur phir jab chahiye koi bhi over ya player ka score dekh lo. Aaj hum SAX ke andar ke components aur DOM ke packages detail mein samjhenge.

Key Points:

1. SAXParser: Yeh interface kaafi types ke `parse()` methods define karta hai — generally ise ek XML data source aur ek `DefaultHandler` object diya jaata hai processing ke liye.
2. SAXReader: Yeh `SAXParser` ke andar wrapped hota hai — yeh actually SAX event handlers ke saath conversation karta hai aur events fire karta hai.

3. DefaultHandler: Yeh class `ContentHandler`, `ErrorHandler`, `DTDHandler`, aur `EntityResolver` sab implement karti hai null methods ke saath — tum sirf woh methods override karo jo tumhe chahiye.
4. ContentHandler: Isme `startDocument`, `endDocument`, `startElement`, `endElement`, `characters()`, aur `processingInstruction()` methods hote hain — yeh tab invoke hote hain jab parser XML tags ya text recognize kare.
5. ErrorHandler: Isme `error()`, `fatalError()`, aur `warning()` methods hain — default implementation fatal errors par exception throw karta hai baaki errors ignore karta hai.
6. DTDHandler: Yeh DTD processing ke liye hota hai — specifically unparsed entity declarations ke liye — general applications mein rarely use hota hai.
7. EntityResolver: Iska `resolveEntity()` method tab invoke hota hai jab parser URI se koi document identify kare — URL ya URN dono handle kar sakta hai, local copy bhi use kar sakta hai.
8. DOM Packages: `org.w3c.dom` DOM programming interfaces define karta hai — `javax.xml.parsers` mein `DocumentBuilderFactory` aur `DocumentBuilder` hain jo `Document` object banate hain.

Example:

Socho tum ek security guard ho university gate par. Jab koi student aata hai — tum uska ID check karte ho (startElement). Jab woh kuch bolta hai — tum sun te ho (characters). Jab woh andar chala jaata hai — tum note karte ho (endElement). Agar koi problem hoti hai — tum apne supervisor ko alert karte ho (ErrorHandler). Aur agar koi visitor bahar se aata hai jise identify karna ho — tum central office se confirm karte ho (EntityResolver). SAX parser bilkul isi tarah kaam karta hai — har event par specific callback method call hota hai.

Code Session

PART A — Working Code Example:

java

```java
import javax.xml.parsers.*;    // SAXParserFactory, SAXParser
import org.xml.sax.*;          // SAXException, Attributes
import org.xml.sax.helpers.*;  // DefaultHandler
import java.io.*;

// Custom handler extending DefaultHandler
// We only override the methods we care about — rest have null implementations
class BookSAXHandler extends DefaultHandler {

    private String currentElement = ""; // tracks which element we are inside

    // ---- ContentHandler methods ----
```

```
// Called once when parser starts reading the document
@Override
public void startDocument() throws SAXException {
    System.out.println("[ContentHandler] Document parsing started.");
}

// Called when an opening tag is found e.g. <title>
@Override
public void startElement(String uri, String localName,
                         String qName, Attributes attributes) throws SAXException {
    currentElement = qName; // remember which element we are inside
    System.out.println("[ContentHandler] Start Element: <" + qName + ">");

    // Check for attributes on the element
    if (attributes.getLength() > 0) {
        System.out.println("  Attribute: " +
            attributes.getQName(0) + "=" + attributes.getValue(0));
    }
}

// Called when text content is found between tags e.g. "Everyday Italian"
@Override
public void characters(char[] ch, int start, int length) throws SAXException {
    String text = new String(ch, start, length).trim();
    if (!text.isEmpty()) {
        System.out.println("[ContentHandler] Text in <" +
            currentElement + ">: " + text);
    }
}

// Called when a closing tag is found e.g. </title>
@Override
public void endElement(String uri, String localName,
                       String qName) throws SAXException {
    System.out.println("[ContentHandler] End Element: </" + qName + ">");
    currentElement = ""; // reset current element tracker
}

// Called once when parser finishes reading the entire document
@Override
public void endDocument() throws SAXException {
    System.out.println("[ContentHandler] Document parsing complete.");
}

// ---- ErrorHandler methods ----

// Called for recoverable errors
@Override
public void error(SAXParseException e) throws SAXException {
    System.out.println("[ErrorHandler] Error: " + e.getMessage());
}

// Called for non-recoverable fatal errors — must throw exception
@Override
public void fatalError(SAXParseException e) throws SAXException {
    System.out.println("[ErrorHandler] Fatal Error: " + e.getMessage());
```

```
        throw e; // fatal errors must stop parsing
    }

    // Called for warnings — usually just log them
    @Override
    public void warning(SAXParseException e) throws SAXException {
        System.out.println("[ErrorHandler] Warning: " + e.getMessage());
    }
}

public class SAXParserDemo {
    public static void main(String[] args) throws Exception {

        // Simulated XML (books.xml)
        String xmlContent =
            "<bookstore>" +
            "<book category=\"cooking\">" +
            "<title>Everyday Italian</title>" +
            "<author>Giada</author>" +
            "</book>" +
            "<book category=\"children\">" +
            "<title>Harry Potter</title>" +
            "<author>Rowling</author>" +
            "</book>" +
            "</bookstore>";

        // Step 1: Create SAXParserFactory — uses pluggability layer
        SAXParserFactory factory = SAXParserFactory.newInstance();

        // Step 2: Create SAXParser from the factory
        SAXParser saxParser = factory.newSAXParser();

        // Step 3: Instantiate our custom handler
        BookSAXHandler handler = new BookSAXHandler();

        // Step 4: Parse XML — SAXReader fires events to our handler
        saxParser.parse(
            new ByteArrayInputStream(xmlContent.getBytes()),
            handler
        );
    }
}
```

Code Walkthrough (Roman Urdu):

- `BookSAXHandler` class `DefaultHandler` extend karti hai — iska matlab yeh hai ke humne sirf woh methods override kiye jo humein chahiye the (`startElement`, `characters`, `endElement`, `startDocument`, `endDocument`, `error`, `fatalError`, `warning`) — baaki sab null implementations automatically inherited hain.
- `currentElement` variable track karta hai ke abhi parser kis element ke andar hai — yeh `characters()` mein kaam aata hai taake pata chale ke yeh text kis tag ka content hai.
- `startElement()` mein `Attributes` object se attribute name aur value bhi print ki — yeh `category="cooking"` jaisi information deta hai.

- `ErrorHandler` methods mein `fatalError()` ne exception rethrow ki — yeh zaroori hai kyunke fatal errors se parsing continue nahi ho sakti. `error()` aur `warning()` ko sirf log kiya.
- `SAXParserFactory.newInstance()` JAXP ka pluggability layer hai — yeh system property `javax.xml.parsers.SAXParserFactory` dekh ke decide karta hai kaunsa vendor implementation use ho.

Output:

```
[ContentHandler] Document parsing started.
[ContentHandler] Start Element: <bookstore>
[ContentHandler] Start Element: <book>
  Attribute: category=cooking
[ContentHandler] Start Element: <title>
[ContentHandler] Text in <title>: Everyday Italian
[ContentHandler] End Element: </title>
[ContentHandler] Start Element: <author>
[ContentHandler] Text in <author>: Giada
[ContentHandler] End Element: </author>
[ContentHandler] End Element: </book>
[ContentHandler] Start Element: <book>
  Attribute: category=children
[ContentHandler] Start Element: <title>
[ContentHandler] Text in <title>: Harry Potter
[ContentHandler] End Element: </title>
[ContentHandler] Start Element: <author>
[ContentHandler] Text in <author>: Rowling
[ContentHandler] End Element: </author>
[ContentHandler] End Element: </book>
[ContentHandler] End Element: </bookstore>
[ContentHandler] Document parsing complete.
```

Yeh output confirm karta hai ke SAX parser sequentially events fire karta hai — `startDocument` se shuru, phir har element aur text ke events, aur `endDocument` par khatam — poori tree kabhi memory mein nahi aati.

Common Mistakes — Error Finder Session

Mistake 1: Overriding characters() but ignoring whitespace — confusing output

Buggy Code:

java

```
@Override
public void characters(char[] ch, int start, int length) {
    // Printing ALL characters including whitespace/newlines
    String text = new String(ch, start, length);
    System.out.println("Text: " + text); // prints blank lines too
}
```

What's Wrong (Roman Urdu): `characters()` method whitespace aur newlines ke liye bhi call hota hai jo XML formatting mein hote hain. Bina `.trim()` aur empty check ke output mein bahut saari blank lines aati hain jo confusing lagti hain.

Fixed Code:

java
```
@Override
public void characters(char[] ch, int start, int length) {
    String text = new String(ch, start, length).trim(); // remove whitespace
    if (!text.isEmpty()) {            // only print non-empty content
        System.out.println("Text: " + text);
    }
}
```

Fix Explained (Roman Urdu): Hamesha `.trim()` karo aur `isEmpty()` check lagao — tab sirf meaningful text content print hoga, formatting whitespace nahi.

Mistake 2: Not throwing exception in fatalError()

Buggy Code:

java
```
@Override
public void fatalError(SAXParseException e) throws SAXException {
    // Student just logs it — does NOT rethrow
    System.out.println("Fatal error: " + e.getMessage());
    // Missing: throw e;
}
```

What's Wrong (Roman Urdu): Fatal errors aise errors hain jinke baad parsing continue karna possible nahi — agar tum exception rethrow nahi karte, toh parser kisi undefined state mein reh sakta hai aur galat results de sakta hai. Yeh serious bug hai.

Fixed Code:

java
```
@Override
public void fatalError(SAXParseException e) throws SAXException {
    System.out.println("Fatal error: " + e.getMessage());
    throw e; // MUST rethrow — parsing cannot continue after fatal error
}
```

Fix Explained (Roman Urdu): `fatalError()` mein hamesha exception rethrow karo — yeh parser ko batata hai ke ab kaam band karo, aage mat badho.

Mistake 3: Using SAXParser directly without factory

Buggy Code:

java
```
// Student tries to directly instantiate SAXParser
SAXParser saxParser = new SAXParser(); // WRONG: SAXParser is an abstract class/interface
saxParser.parse(new ByteArrayInputStream(xml.getBytes()), handler);
```

What's Wrong (Roman Urdu): `SAXParser` directly instantiate nahi ho sakta kyunke yeh ek abstract class hai. Isko hamesha `SAXParserFactory` ke zariye banana padta hai — yeh JAXP ka design hai taake implementation vendor-independent rahe.

Fixed Code:

java
```
// Always use factory to create SAXParser
SAXParserFactory factory = SAXParserFactory.newInstance(); // factory first
SAXParser saxParser = factory.newSAXParser();              // then parser
saxParser.parse(new ByteArrayInputStream(xml.getBytes()), handler);
```

Fix Explained (Roman Urdu): Factory pattern follow karo — `SAXParserFactory.newInstance()` se factory lo, `newSAXParser()` se parser lo — kabhi directly instantiate mat karo.

Mistake 4: Confusing DocumentBuilder.newDocument() with parse()

Buggy Code:

java
```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.newDocument(); // creates EMPTY document

// Student thinks elements from some XML are now available
NodeList list = doc.getElementsByTagName("title");
System.out.println("Found: " + list.getLength()); // prints 0 — confuses student
```

What's Wrong (Roman Urdu): `newDocument()` sirf ek bilkul khaali `Document` object banata hai — koi XML parse nahi hota. Student confuse ho jaata hai ke elements kyun nahi mil rahe. Elements tabhi milenge jab `builder.parse()` se actual XML load karo.

Fixed Code:

java
```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
```

```
// Use parse() to load actual XML into the Document
Document doc = builder.parse(
   new ByteArrayInputStream("<root><title>Hello</title></root>".getBytes())
);
NodeList list = doc.getElementsByTagName("title");
System.out.println("Found: " + list.getLength()); // prints 1
```

Fix Explained (Roman Urdu): `newDocument()` aur `parse()` mein fark yaad rakho — `newDocument()` khaali document, `parse()` actual XML se filled document. Elements dhundne hain toh `parse()` use karo.

Conceptual Questions

Q: A student overrides only `startElement()` and `endElement()` in their `DefaultHandler` subclass but never sees any text content printed. What is the reason and which method should they add?

Answer: `startElement()` aur `endElement()` sirf opening aur closing tags ke liye fire hote hain — text content ke liye `characters()` method alag se override karna padta hai. `DefaultHandler` mein `characters()` ki default implementation null hai — kuch nahi karta. Isi liye text kabhi print nahi hua.

- Solution yeh hai ke class mein `characters(char[] ch, int start, int length)` method add karo aur `new String(ch, start, length).trim()` se text extract karo. Tabhi `<title>Everyday Italian</title>` jaisi values milenge.

Q: What is the difference between the `error()` and `fatalError()` methods in SAX `ErrorHandler`, and how should each be handled differently in code?

Answer: `error()` recoverable parsing error ke liye call hota hai — jaise validation error — jahan se parser kabhi kabhi aage bhi chal sakta hai. `fatalError()` non-recoverable error ke liye call hota hai — parser is ke baad kaam nahi kar sakta. Dono ka handling different hona chahiye.

- `error()` mein tum sirf log kar sakte ho ya apna custom logic laga sakte ho — exception throw karna optional hai. Lekin `fatalError()` mein `throw e` karna zaroori hai, warna parser undefined state mein chala jaata hai aur galat results deta hai.

Lesson Quiz

1. Which SAX interface provides the `startElement()`, `endElement()`, and `characters()` callback methods? (`ContentHandler`)
2. What does `DefaultHandler` provide that makes it easier to implement a SAX handler class? (Null implementations of all SAX interfaces so you only override the methods you need)
3. Which SAX package defines the core SAX interfaces including `ContentHandler` and `ErrorHandler`? (`org.xml.sax`)
4. What is the role of `EntityResolver` in SAX parsing? (It resolves external entities identified by URI — it can substitute a local copy instead of fetching from the web)
5. Which DOM class is used to create an empty `Document` object or parse existing XML into a `Document`? (`DocumentBuilder`)

Lesson 32: SAX Parser Internals and DOM API Packages

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum ek live cricket match sun rahe ho radio par. Commentator har ball par foran bolta hai — "wicket giri!", "four hua!", "over khatam!" — tum poora match yaad nahi rakhte, sirf jo abhi hua woh sun ke react karte ho. Yeh SAX parsing hai — events fire hoti hain aur tum react karte ho. Lekin DOM aise hai jaise tum match ka poora scorecard print karke table par rakh lo aur phir jab chahiye koi bhi over ya player ka score dekh lo. Aaj hum SAX ke andar ke components aur DOM ke packages detail mein samjhenge.

Key Points:

1. SAXParser: Yeh interface kaafi types ke `parse()` methods define karta hai — generally ise ek XML data source aur ek `DefaultHandler` object diya jaata hai processing ke liye.
2. SAXReader: Yeh `SAXParser` ke andar wrapped hota hai — yeh actually SAX event handlers ke saath conversation karta hai aur events fire karta hai.
3. DefaultHandler: Yeh class `ContentHandler`, `ErrorHandler`, `DTDHandler`, aur `EntityResolver` sab implement karti hai null methods ke saath — tum sirf woh methods override karo jo tumhe chahiye.
4. ContentHandler: Isme `startDocument`, `endDocument`, `startElement`, `endElement`, `characters()`, aur `processingInstruction()` methods hote hain — yeh tab invoke hote hain jab parser XML tags ya text recognize kare.
5. ErrorHandler: Isme `error()`, `fatalError()`, aur `warning()` methods hain — default implementation fatal errors par exception throw karta hai baaki errors ignore karta hai.
6. DTDHandler: Yeh DTD processing ke liye hota hai — specifically unparsed entity declarations ke liye — general applications mein rarely use hota hai.

7. EntityResolver: Iska `resolveEntity()` method tab invoke hota hai jab parser URI se koi document identify kare — URL ya URN dono handle kar sakta hai, local copy bhi use kar sakta hai.
8. DOM Packages: `org.w3c.dom` DOM programming interfaces define karta hai — `javax.xml.parsers` mein `DocumentBuilderFactory` aur `DocumentBuilder` hain jo `Document` object banate hain.

Example:

Socho tum ek security guard ho university gate par. Jab koi student aata hai — tum uska ID check karte ho (startElement). Jab woh kuch bolta hai — tum sun te ho (characters). Jab woh andar chala jaata hai — tum note karte ho (endElement). Agar koi problem hoti hai — tum apne supervisor ko alert karte ho (ErrorHandler). Aur agar koi visitor bahar se aata hai jise identify karna ho — tum central office se confirm karte ho (EntityResolver). SAX parser bilkul isi tarah kaam karta hai — har event par specific callback method call hota hai.

Code Session

PART A — Working Code Example:

java

```java
import javax.xml.parsers.*;    // SAXParserFactory, SAXParser
import org.xml.sax.*;          // SAXException, Attributes
import org.xml.sax.helpers.*;  // DefaultHandler
import java.io.*;

// Custom handler extending DefaultHandler
// We only override the methods we care about — rest have null implementations
class BookSAXHandler extends DefaultHandler {

    private String currentElement = ""; // tracks which element we are inside

    // ---- ContentHandler methods ----

    // Called once when parser starts reading the document
    @Override
    public void startDocument() throws SAXException {
        System.out.println("[ContentHandler] Document parsing started.");
    }

    // Called when an opening tag is found e.g. <title>
    @Override
    public void startElement(String uri, String localName,
                             String qName, Attributes attributes) throws SAXException {
        currentElement = qName; // remember which element we are inside
        System.out.println("[ContentHandler] Start Element: <" + qName + ">");
```

```
        // Check for attributes on the element
        if (attributes.getLength() > 0) {
            System.out.println("  Attribute: " +
                attributes.getQName(0) + "=" + attributes.getValue(0));
        }
    }

    // Called when text content is found between tags e.g. "Everyday Italian"
    @Override
    public void characters(char[] ch, int start, int length) throws SAXException {
        String text = new String(ch, start, length).trim();
        if (!text.isEmpty()) {
            System.out.println("[ContentHandler] Text in <" +
                currentElement + ">: " + text);
        }
    }

    // Called when a closing tag is found e.g. </title>
    @Override
    public void endElement(String uri, String localName,
                           String qName) throws SAXException {
        System.out.println("[ContentHandler] End Element: </" + qName + ">");
        currentElement = ""; // reset current element tracker
    }

    // Called once when parser finishes reading the entire document
    @Override
    public void endDocument() throws SAXException {
        System.out.println("[ContentHandler] Document parsing complete.");
    }

    // ---- ErrorHandler methods ----

    // Called for recoverable errors
    @Override
    public void error(SAXParseException e) throws SAXException {
        System.out.println("[ErrorHandler] Error: " + e.getMessage());
    }

    // Called for non-recoverable fatal errors — must throw exception
    @Override
    public void fatalError(SAXParseException e) throws SAXException {
        System.out.println("[ErrorHandler] Fatal Error: " + e.getMessage());
        throw e; // fatal errors must stop parsing
    }

    // Called for warnings — usually just log them
    @Override
    public void warning(SAXParseException e) throws SAXException {
        System.out.println("[ErrorHandler] Warning: " + e.getMessage());
    }
}

public class SAXParserDemo {
    public static void main(String[] args) throws Exception {
```

```
        // Simulated XML (books.xml)
        String xmlContent =
            "<bookstore>" +
            "<book category=\"cooking\">" +
            "<title>Everyday Italian</title>" +
            "<author>Giada</author>" +
            "</book>" +
            "<book category=\"children\">" +
            "<title>Harry Potter</title>" +
            "<author>Rowling</author>" +
            "</book>" +
            "</bookstore>";

        // Step 1: Create SAXParserFactory — uses pluggability layer
        SAXParserFactory factory = SAXParserFactory.newInstance();

        // Step 2: Create SAXParser from the factory
        SAXParser saxParser = factory.newSAXParser();

        // Step 3: Instantiate our custom handler
        BookSAXHandler handler = new BookSAXHandler();

        // Step 4: Parse XML — SAXReader fires events to our handler
        saxParser.parse(
            new ByteArrayInputStream(xmlContent.getBytes()),
            handler
        );
    }
}
```

Code Walkthrough (Roman Urdu):

- `BookSAXHandler` class `DefaultHandler` extend karti hai — iska matlab yeh hai ke humne sirf woh methods override kiye jo humein chahiye the (`startElement`, `characters`, `endElement`, `startDocument`, `endDocument`, `error`, `fatalError`, `warning`) — baaki sab null implementations automatically inherited hain.
- `currentElement` variable track karta hai ke abhi parser kis element ke andar hai — yeh `characters()` mein kaam aata hai taake pata chale ke yeh text kis tag ka content hai.
- `startElement()` mein `Attributes` object se attribute name aur value bhi print ki — yeh `category="cooking"` jaisi information deta hai.
- `ErrorHandler` methods mein `fatalError()` ne exception rethrow ki — yeh zaroori hai kyunke fatal errors se parsing continue nahi ho sakti. `error()` aur `warning()` ko sirf log kiya.
- `SAXParserFactory.newInstance()` JAXP ka pluggability layer hai — yeh system property `javax.xml.parsers.SAXParserFactory` dekh ke decide karta hai kaunsa vendor implementation use ho.

Output:

```
[ContentHandler] Document parsing started.
```

```
[ContentHandler] Start Element: <bookstore>
[ContentHandler] Start Element: <book>
  Attribute: category=cooking
[ContentHandler] Start Element: <title>
[ContentHandler] Text in <title>: Everyday Italian
[ContentHandler] End Element: </title>
[ContentHandler] Start Element: <author>
[ContentHandler] Text in <author>: Giada
[ContentHandler] End Element: </author>
[ContentHandler] End Element: </book>
[ContentHandler] Start Element: <book>
  Attribute: category=children
[ContentHandler] Start Element: <title>
[ContentHandler] Text in <title>: Harry Potter
[ContentHandler] End Element: </title>
[ContentHandler] Start Element: <author>
[ContentHandler] Text in <author>: Rowling
[ContentHandler] End Element: </author>
[ContentHandler] End Element: </book>
[ContentHandler] End Element: </bookstore>
[ContentHandler] Document parsing complete.
```

Yeh output confirm karta hai ke SAX parser sequentially events fire karta hai — `startDocument` se shuru, phir har element aur text ke events, aur `endDocument` par khatam — poori tree kabhi memory mein nahi aati.

Common Mistakes — Error Finder Session

Mistake 1: Overriding characters() but ignoring whitespace — confusing output

Buggy Code:

java
```
@Override
public void characters(char[] ch, int start, int length) {
    // Printing ALL characters including whitespace/newlines
    String text = new String(ch, start, length);
    System.out.println("Text: " + text); // prints blank lines too
}
```

What's Wrong (Roman Urdu): `characters()` method whitespace aur newlines ke liye bhi call hota hai jo XML formatting mein hote hain. Bina `.trim()` aur empty check ke output mein bahut saari blank lines aati hain jo confusing lagti hain.

Fixed Code:

java
```
@Override
public void characters(char[] ch, int start, int length) {
```

```
    String text = new String(ch, start, length).trim(); // remove whitespace
    if (!text.isEmpty()) {            // only print non-empty content
        System.out.println("Text: " + text);
    }
}
```

Fix Explained (Roman Urdu): Hamesha `.trim()` karo aur `isEmpty()` check lagao — tab sirf meaningful text content print hoga, formatting whitespace nahi.

Mistake 2: Not throwing exception in fatalError()

Buggy Code:

java
```
@Override
public void fatalError(SAXParseException e) throws SAXException {
    // Student just logs it — does NOT rethrow
    System.out.println("Fatal error: " + e.getMessage());
    // Missing: throw e;
}
```

What's Wrong (Roman Urdu): Fatal errors aise errors hain jinke baad parsing continue karna possible nahi — agar tum exception rethrow nahi karte, toh parser kisi undefined state mein reh sakta hai aur galat results de sakta hai. Yeh serious bug hai.

Fixed Code:

java
```
@Override
public void fatalError(SAXParseException e) throws SAXException {
    System.out.println("Fatal error: " + e.getMessage());
    throw e; // MUST rethrow — parsing cannot continue after fatal error
}
```

Fix Explained (Roman Urdu): `fatalError()` mein hamesha exception rethrow karo — yeh parser ko batata hai ke ab kaam band karo, aage mat badho.

Mistake 3: Using SAXParser directly without factory

Buggy Code:

java
```
// Student tries to directly instantiate SAXParser
SAXParser saxParser = new SAXParser(); // WRONG: SAXParser is an abstract class/interface
saxParser.parse(new ByteArrayInputStream(xml.getBytes()), handler);
```

What's Wrong (Roman Urdu): `SAXParser` directly instantiate nahi ho sakta kyunke yeh ek abstract class hai. Isko hamesha `SAXParserFactory` ke zariye banana padta hai — yeh JAXP ka design hai taake implementation vendor-independent rahe.

Fixed Code:

java
```
// Always use factory to create SAXParser
SAXParserFactory factory = SAXParserFactory.newInstance(); // factory first
SAXParser saxParser = factory.newSAXParser();              // then parser
saxParser.parse(new ByteArrayInputStream(xml.getBytes()), handler);
```

Fix Explained (Roman Urdu): Factory pattern follow karo — `SAXParserFactory.newInstance()` se factory lo, `newSAXParser()` se parser lo — kabhi directly instantiate mat karo.

Mistake 4: Confusing DocumentBuilder.newDocument() with parse()

Buggy Code:

java
```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.newDocument(); // creates EMPTY document

// Student thinks elements from some XML are now available
NodeList list = doc.getElementsByTagName("title");
System.out.println("Found: " + list.getLength()); // prints 0 — confuses student
```

What's Wrong (Roman Urdu): `newDocument()` sirf ek bilkul khaali `Document` object banata hai — koi XML parse nahi hota. Student confuse ho jaata hai ke elements kyun nahi mil rahe. Elements tabhi milenge jab `builder.parse()` se actual XML load karo.

Fixed Code:

java
```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
// Use parse() to load actual XML into the Document
Document doc = builder.parse(
    new ByteArrayInputStream("<root><title>Hello</title></root>".getBytes())
);
NodeList list = doc.getElementsByTagName("title");
System.out.println("Found: " + list.getLength()); // prints 1
```

Fix Explained (Roman Urdu): `newDocument()` aur `parse()` mein fark yaad rakho — `newDocument()` khaali document, `parse()` actual XML se filled document. Elements dhundne hain toh `parse()` use karo.

Conceptual Questions

Q: A student overrides only `startElement()` and `endElement()` in their `DefaultHandler` subclass but never sees any text content printed. What is the reason and which method should they add?

Answer: `startElement()` aur `endElement()` sirf opening aur closing tags ke liye fire hote hain — text content ke liye `characters()` method alag se override karna padta hai. `DefaultHandler` mein `characters()` ki default implementation null hai — kuch nahi karta. Isi liye text kabhi print nahi hua.

- Solution yeh hai ke class mein `characters(char[] ch, int start, int length)` method add karo aur `new String(ch, start, length).trim()` se text extract karo. Tabhi `<title>Everyday Italian</title>` jaisi values milenge.

Q: What is the difference between the `error()` and `fatalError()` methods in SAX `ErrorHandler`, and how should each be handled differently in code?

Answer: `error()` recoverable parsing error ke liye call hota hai — jaise validation error — jahan se parser kabhi kabhi aage bhi chal sakta hai. `fatalError()` non-recoverable error ke liye call hota hai — parser is ke baad kaam nahi kar sakta. Dono ka handling different hona chahiye.

- `error()` mein tum sirf log kar sakte ho ya apna custom logic laga sakte ho — exception throw karna optional hai. Lekin `fatalError()` mein `throw e` karna zaroori hai, warna parser undefined state mein chala jaata hai aur galat results deta hai.

Lesson Quiz

1. Which SAX interface provides the `startElement()`, `endElement()`, and `characters()` callback methods? (`ContentHandler`)
2. What does `DefaultHandler` provide that makes it easier to implement a SAX handler class? (Null implementations of all SAX interfaces so you only override the methods you need)
3. Which SAX package defines the core SAX interfaces including `ContentHandler` and `ErrorHandler`? (`org.xml.sax`)

4. What is the role of `EntityResolver` in SAX parsing? (It resolves external entities identified by URI — it can substitute a local copy instead of fetching from the web)
5. Which DOM class is used to create an empty `Document` object or parse existing XML into a `Document`? (`DocumentBuilder`

SAX Packages The SAX parser is defined in the packages listed in the following Table. Packages Description org.xml.sax Defines the SAX interfaces. The name org.xml is the package prefix that was settled on by the group that defined the SAX API. org.xml.sax.ext Defines SAX extensions that are us

pasted

Lesson 33: SAX ContentHandler Interface and Introduction to JDOM

Lesson 33: SAX ContentHandler Interface and Introduction to JDOM

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tum ek live sports commentary sun rahe ho — commentator har waqiah ka real time mein announcement karta hai: "match shuru!", "player maidan mein aaya!", "ball pheenki gayi!", "over khatam!", "match khatam!". Tumhara kaam hai in announcements ko sun ke react karna. SAX ka ContentHandler bilkul yahi hai — har XML event par specific method call hota hai. JDOM ek alag cheez hai — yeh ek Java-friendly library hai jo SAX aur DOM dono ki achi baatein combine kar ke developers ko simple aur fast XML parsing deti hai.

Key Points:

1. SAX Event-Based Parsing: SAX koi parse tree nahi banata — woh XML ko upar se neeche sequential order mein padhta hai aur events fire karta hai — application ko sirf event handler register karna hota hai.
2. startDocument() / endDocument(): Yeh methods document ki shuruwaat aur end par call hote hain — initialization aur cleanup logic yahan likhte hain.
3. startElement(): XML mein opening tag milne par call hota hai — element name, namespace URI, aur attributes milte hain parameter mein.
4. endElement(): Closing tag milne par call hota hai — element ka kaam khatam hone ka signal.
5. characters(): Text content milne par call hota hai — ek `char[]` array milta hai jisme start index aur length se actual text nikalna padta hai.
6. ignorableWhitespace(): DTD present hone par formatting whitespace ke liye call hota hai — actual data nahi hota.
7. processingInstruction(): XML mein koi inline processing instruction milne par call hota hai.

8. JDOM: Yeh ek open-source Java library hai jo XML parsing ke liye Java Collections (List, Arrays) use karti hai — SAX se zyada convenient aur DOM se zyada memory efficient hai.
9. JDOM Classes: Document, Element, Attribute, Text, aur Comment — yeh sab Java objects ki tarah kaam karte hain aur directly manipulate ho sakte hain.

Example:

Socho tum ek wedding mein gate par khadhe ho aur mehman aane ka record rakh rahe ho. Jab shaadi shuru hoti hai — tum note karte ho (startDocument). Jab koi mehman aata hai — tum uska naam aur card check karte ho (startElement + attributes). Jab woh kuch kehta hai — tum sun te ho (characters). Jab woh andar chala jaata hai — tum tick mark karte ho (endElement). Jab sab mehman aa jaate hain aur shaadi khatam — tum register band karte ho (endDocument). SAX ContentHandler bilkul yahi kaam karta hai har XML element ke saath.

Code Session

PART A — Working Code Example:

java

```java
import javax.xml.parsers.*;
import org.xml.sax.*;
import org.xml.sax.helpers.*;
import java.io.*;

// Full ContentHandler implementation showing all major callback methods
class FullContentHandler extends DefaultHandler {

    private String currentElement = "";  // track which element we are inside
    private int depth = 0;               // track nesting depth for indentation

    // Called once at the very start of XML document
    @Override
    public void startDocument() throws SAXException {
        System.out.println(">>> startDocument() called — parsing begins");
    }

    // Called once at the very end of XML document
    @Override
    public void endDocument() throws SAXException {
        System.out.println(">>> endDocument() called — parsing complete");
    }

    // Called every time an opening tag is encountered
    @Override
    public void startElement(String uri, String localName,
                             String qName, Attributes atts) throws SAXException {
```

```
    currentElement = qName;
    depth++;

    // Build indentation based on nesting depth
    String indent = "  ".repeat(depth);
    System.out.println(indent + "startElement(): <" + qName + ">");

    // Print all attributes of this element if any exist
    for (int i = 0; i < atts.getLength(); i++) {
        System.out.println(indent + "  @" + atts.getQName(i)
                + " = " + atts.getValue(i));
    }
}

// Called every time a closing tag is encountered
@Override
public void endElement(String uri, String localName,
                String qName) throws SAXException {
    String indent = "  ".repeat(depth);
    System.out.println(indent + "endElement(): </" + qName + ">");
    depth--;                    // decrease depth as we exit element
    currentElement = "";
}

// Called when text content is found between tags
@Override
public void characters(char[] ch, int start,
                int length) throws SAXException {
    String text = new String(ch, start, length).trim(); // extract text safely
    if (!text.isEmpty()) {
        String indent = "  ".repeat(depth + 1);
        System.out.println(indent + "characters() in <"
                + currentElement + ">: " + text);
    }
}

// Called when ignorable whitespace is encountered (only with DTD)
@Override
public void ignorableWhitespace(char[] ch, int start,
                        int length) throws SAXException {
    // Usually ignored — just logging for demo
    System.out.println("  [ignorableWhitespace called]");
}

// Called when a processing instruction is encountered in XML
@Override
public void processingInstruction(String target,
                          String data) throws SAXException {
    System.out.println("processingInstruction(): target=" +
                target + ", data=" + data);
}

// Called when namespace mapping starts
@Override
public void startPrefixMapping(String prefix,
                        String uri) throws SAXException {
```

```
        System.out.println("startPrefixMapping(): prefix=" +
                           prefix + ", uri=" + uri);
    }

    // Called when namespace mapping ends
    @Override
    public void endPrefixMapping(String prefix) throws SAXException {
        System.out.println("endPrefixMapping(): prefix=" + prefix);
    }
}

public class ContentHandlerDemo {
    public static void main(String[] args) throws Exception {

        // XML with processing instruction, attributes, and nested elements
        String xmlContent =
            "<?xml version=\"1.0\"?>" +
            "<?display table-view?>" +          // processing instruction
            "<bookstore>" +
            "<book category=\"cooking\">" +
            "<title lang=\"en\">Everyday Italian</title>" +
            "<price>30.00</price>" +
            "</book>" +
            "<book category=\"children\">" +
            "<title lang=\"en\">Harry Potter</title>" +
            "<price>29.99</price>" +
            "</book>" +
            "</bookstore>";

        // Create factory and parser using JAXP
        SAXParserFactory factory = SAXParserFactory.newInstance();
        SAXParser saxParser = factory.newSAXParser();

        // Use our full content handler
        FullContentHandler handler = new FullContentHandler();

        // Begin parsing — events will fire into handler callbacks
        saxParser.parse(
            new ByteArrayInputStream(xmlContent.getBytes()),
            handler
        );
    }
}
```

Code Walkthrough (Roman Urdu):

- `depth` variable ek counter hai jo track karta hai ke hum XML mein kitne levels andar hain — har `startElement` par badhta hai, har `endElement` par ghatata hai — yeh indented output ke liye use kiya taake structure clearly dikh sake.
- `startElement()` mein `Attributes atts` parameter se sab attributes loop mein print kiye — `atts.getLength()` se count mili, `getQName(i)` se naam aur `getValue(i)` se value. Yeh `category="cooking"` aur `lang="en"` dono pakdta hai.

- `characters()` mein `new String(ch, start, length)` zaroor likhna padta hai — seedha `new String(ch)` galat hoga kyunke `ch` array mein extra data bhi ho sakta hai, sirf `start` se `length` tak ka hissa hamare element ka text hai.
- `processingInstruction()` tab call hua jab parser ne `<?display table-view?>` dekha — `target` = "display", `data` = "table-view".
- `startDocument()` aur `endDocument()` sirf ek ek baar call hue — poori parsing ke shuru aur khatam mein.

Output:

```
>>> startDocument() called — parsing begins
processingInstruction(): target=display, data=table-view
  startElement(): <bookstore>
    startElement(): <book>
      @category = cooking
      startElement(): <title>
        @lang = en
        characters() in <title>: Everyday Italian
      endElement(): </title>
      startElement(): <price>
        characters() in <price>: 30.00
      endElement(): </price>
    endElement(): </book>
    startElement(): <book>
      @category = children
      startElement(): <title>
        @lang = en
        characters() in <title>: Harry Potter
      endElement(): </title>
      startElement(): <price>
        characters() in <price>: 29.99
      endElement(): </price>
    endElement(): </book>
  endElement(): </bookstore>
>>> endDocument() called — parsing complete
```

Yeh output confirm karta hai ke SAX parser top-to-bottom sequential events fire karta hai — processing instruction pehle, phir har element ka startElement, characters, aur endElement apni respective depth par — koi tree memory mein nahi bani.

Common Mistakes — Error Finder Session

Mistake 1: Extracting text with wrong String constructor in characters()

Buggy Code:

java
```
@Override
public void characters(char[] ch, int start, int length)
```

```
        throws SAXException {
    // WRONG: using entire char array instead of start+length slice
    String text = new String(ch).trim();
    System.out.println("Text: " + text);
}
```

What's Wrong (Roman Urdu): `ch` array mein sirf humara text nahi hota — parser ka internal buffer hota hai jisme alag alag content hota hai. `new String(ch)` poora buffer convert karta hai toh galat aur extra text print hota hai. `start` aur `length` zaroor use karna chahiye.

Fixed Code:

java

```
@Override
public void characters(char[] ch, int start, int length)
        throws SAXException {
    // CORRECT: extract only the relevant slice of the buffer
    String text = new String(ch, start, length).trim();
    if (!text.isEmpty()) {
        System.out.println("Text: " + text);
    }
}
```

Fix Explained (Roman Urdu): Hamesha `new String(ch, start, length)` use karo — yeh sirf woh hissa leta hai jo actually is element ka text hai, poora buffer nahi.

Mistake 2: Registering no handler and expecting output

Buggy Code:

java

```
SAXParserFactory factory = SAXParserFactory.newInstance();
SAXParser saxParser = factory.newSAXParser();

// Parsing without providing any handler
saxParser.parse(
    new ByteArrayInputStream(xml.getBytes()),
    (DefaultHandler) null  // WRONG: null handler passed
);
System.out.println("Done"); // runs but nothing was processed
```

What's Wrong (Roman Urdu): Agar handler null diya jaaye toh `NullPointerException` aayegi ya parser koi event process nahi karega. SAX parser ka poora kaam handler ke callbacks par depend karta hai — bina handler ke parsing karne ka koi faida nahi.

Fixed Code:

java

```
SAXParserFactory factory = SAXParserFactory.newInstance();
SAXParser saxParser = factory.newSAXParser();
FullContentHandler handler = new FullContentHandler(); // create handler
saxParser.parse(
   new ByteArrayInputStream(xml.getBytes()),
   handler  // CORRECT: pass actual handler object
);
```

Fix Explained (Roman Urdu): SAX parsing hamesha ek valid handler object ke saath karo — wahi actual processing karta hai events receive karke.

Mistake 3: Using depth counter but forgetting to reset on endElement

Buggy Code:

java
```
int depth = 0;

@Override
public void startElement(String uri, String localName,
                         String qName, Attributes atts) {
   depth++;
   System.out.println("  ".repeat(depth) + "<" + qName + ">");
}

@Override
public void endElement(String uri, String localName,
                       String qName) {
   // WRONG: forgot to decrease depth
   System.out.println("  ".repeat(depth) + "</" + qName + ">");
   // Missing: depth--;
}
```

What's Wrong (Roman Urdu): `depth` counter sirf badh raha hai, kum nahi ho raha. Jaise jaise elements close hote hain, indentation badh ti jaati hai galat tarike se — output bilkul bے ترتیب dikhega.

Fixed Code:

java
```
@Override
public void endElement(String uri, String localName,
                       String qName) {
   System.out.println("  ".repeat(depth) + "</" + qName + ">");
   depth--; // CORRECT: decrease after printing closing tag
}
```

Fix Explained (Roman Urdu): `endElement()` mein pehle current depth par closing tag print karo, phir `depth--` karo — yahi sequence sahi indentation deta hai.

Mistake 4: Confusing JDOM with standard Java DOM

Buggy Code:

java
```
// Student thinks JDOM is built into Java — tries to import without adding library
import org.jdom2.Document;   // WRONG if JDOM jar not in classpath
import org.jdom2.Element;

public class JDOMTest {
    public static void main(String[] args) {
        Document doc = new Document(); // compile error if JDOM not added
    }
}
```

What's Wrong (Roman Urdu): JDOM Java ka built-in part nahi hai — yeh ek alag open-source library hai jise project mein manually add karna padta hai (Maven dependency ya jar file). Bina library add kiye `import org.jdom2.*` compile error deta hai.

Fixed Code:

xml
```
<!-- Add this to pom.xml if using Maven -->
<dependency>
    <groupId>org.jdom</groupId>
    <artifactId>jdom2</artifactId>
    <version>2.0.6</version>
</dependency>
```
java
```
// After adding the dependency, imports will resolve correctly
import org.jdom2.Document;
import org.jdom2.Element;
// Now JDOM classes are available
```

Fix Explained (Roman Urdu): JDOM use karne se pehle uski library project mein add karo — Maven mein dependency daalo ya manually jdom2.jar classpath mein rakho, tab imports kaam karenge.

Conceptual Questions

Q: In a SAX `ContentHandler`, the `characters()` method is called with `char[] ch, int start, int length`. What will go wrong if a student writes `new String(ch)` instead of `new String(ch, start, length)`?

Answer: `new String(ch)` poora internal parser buffer convert karta hai — isme sirf hamare element ka text nahi hota, baaki content bhi hota hai. Iska result yeh hota hai ke output mein galat aur extra characters print honge. `new String(ch, start, length)` sirf woh slice leta hai jo actually is callback call ka text hai.

- Jaise agar `ch` buffer mein `"Everyday ItalianHarry Potter"` store hai aur hamara element sirf `"Everyday Italian"` hai — `start=0, length=16` batata hai ke sirf pehle 16 characters lo. `new String(ch)` se poora string milega jo wrong hai.

Q: What is the key difference between JDOM and the standard Java DOM API in terms of how they handle XML elements in code?

Answer: Standard Java DOM (`org.w3c.dom`) W3C specification ke according low-level nodes use karta hai — `Node`, `Element`, `Text` sab alag alag type-cast karke access karne padte hain aur code verbose hota hai. JDOM Java developers ke liye optimize hai — woh Java Collections jaise `List` use karta hai aur directly Java objects ki tarah feel hota hai.

- Jaise DOM mein kisi element ke children get karne ke liye `getChildNodes()` se `NodeList` milti hai jise loop mein cast karna padta hai. JDOM mein `element.getChildren("title")` seedha `List<Element>` return karta hai jo foreach loop mein directly use hoti hai — code bohot saaf aur kam hota hai.

Lesson Quiz

1. Which `ContentHandler` method is called when text content like `"Everyday Italian"` is encountered between XML tags? (`characters()`)
2. What is the correct way to extract text from the `characters()` callback parameters `ch`, `start`, and `length`? (`new String(ch, start, length)`)
3. Which `ContentHandler` method is called when the parser encounters `<?display table-view?>` in the XML? (`processingInstruction()`)
4. What is the main advantage JDOM has over standard Java DOM in terms of data structures? (JDOM uses Java Collections like `List` and `Arrays` making it more Java-friendly and less verbose)
5. In SAX parsing, which method would you override to capture the `category` attribute value from `<book category="cooking">`? (`startElement()` — using the `Attributes atts` parameter)

Lesson 34: JDOM XML Parsing and Introduction to JAXB

Deep Explanation

Opening Analogy (Roman Urdu):

Socho tumhare paas ek register hai jisme school ke students ka data likha hai. JDOM ek aisi library hai jo yeh register read karne, usme naya entry add karne, purani entry update karne, aur koi entry delete karne ka kaam bohot aasaan tarike se karne deti hai — bilkul jaise tum notebook mein pencil se changes karte ho. JAXB ek aur level aage hai — yeh Java objects aur XML ke darmiyan automatic translator ki tarah kaam karta hai — Java object likho, woh khud XML bana deta hai, aur XML do toh khud Java object bana deta hai.

Key Points:

1. SAXBuilder: Yeh JDOM ka main class hai jo XML file ya stream ko parse karke `Document` object banata hai — standard SAX parser ka wrapper hai.
2. document.getRootElement(): Yeh method XML ka root element return karta hai jisse poori tree navigate ki ja sakti hai.
3. getAttribute("name"): Yeh kisi element ka specific attribute return karta hai — jaise `id` ya `category`.
4. getChildren("name"): Yeh ek specific naam ke saare child elements ki `List` return karta hai — Java for-each loop mein directly use hoti hai.
5. getChild("name"): Yeh specified naam ka pehla child element return karta hai — single element access ke liye.
6. JDOM Modification: JDOM se existing XML mein elements add, update aur delete kiye ja sakte hain — `addContent()`, `setAttribute()`, `setText()`, aur `detach()` methods use hoti hain.
7. JAXB (Java Architecture for XML Binding): Yeh Java objects aur XML ke darmiyan two-way binding provide karta hai — marshalling (Java to XML) aur unmarshalling (XML to Java) dono support karta hai.
8. Unmarshalling: XML document ko Java content tree mein convert karna — XML read karna aur Java objects banana.
9. Marshalling: Java content tree ko XML document mein convert karna — Java objects ko XML mein likhna.
10. Schema Compiler / Generator: JAXB ka schema compiler XML schema se Java classes banata hai, schema generator Java classes se XML schema banata hai.

Example:

Socho tumhare office mein employees ka ek XML file hai. JDOM se tum woh file khol sakte ho, naya employee add kar sakte ho (addContent), kisi ka salary update kar sakte ho (setText), kisi ka ID badal sakte ho (setAttribute), aur kisi employee ko hata sakte ho (detach). JAXB ek step aur aage hai — tum seedha Java mein `Employee` class banao, uska object banao, salary set karo — JAXB khud usse XML mein convert kar deta hai. Aur agar XML do toh JAXB seedha Java `Employee` object bana deta hai — beech mein parsing ki zaroorat nahi.

Code Session

PART A — Working Code Example:

java

```java
import java.io.*;
import java.util.*;
import org.jdom2.*;          // Document, Element, Attribute
import org.jdom2.input.*;    // SAXBuilder
import org.jdom2.output.*;   // XMLOutputter for printing result

public class JDOMDemo {
    public static void main(String[] args) throws Exception {

        // ================================================
        // PART 1: Build Document from XML string
        // ================================================

        // Simulated XML content (normally would be a file)
        String xmlContent =
            "<company>" +
            "<staff id=\"1\">" +
            "<firstname>Ali</firstname>" +
            "<lastname>Khan</lastname>" +
            "<salary>5000</salary>" +
            "</staff>" +
            "<staff id=\"2\">" +
            "<firstname>Sara</firstname>" +
            "<lastname>Ahmed</lastname>" +
            "<salary>6000</salary>" +
            "</staff>" +
            "</company>";

        // Step 1: Create SAXBuilder — JDOM's XML parser
        SAXBuilder saxBuilder = new SAXBuilder();

        // Step 2: Build Document from stream
        Document document = saxBuilder.build(
            new ByteArrayInputStream(xmlContent.getBytes())
        );

        // Step 3: Get root element <company>
        Element rootElement = document.getRootElement();
        System.out.println("Root element: " + rootElement.getName());

        // Step 4: Get all <staff> children
        List<Element> staffList = rootElement.getChildren("staff");
        System.out.println("Number of staff: " + staffList.size());

        // Step 5: Examine attributes and sub-elements
        for (Element staff : staffList) {
            // getAttribute() returns the Attribute object
            String id = staff.getAttributeValue("id");
            // getChild() returns first child element by name
            String firstName = staff.getChildText("firstname");
```

```
            String salary = staff.getChildText("salary");
            System.out.println("ID=" + id +
                               ", Name=" + firstName +
                               ", Salary=" + salary);
        }

        // ================================================
        // PART 2: MODIFY the XML document
        // ================================================

        // --- ADD: Add new <age> element under first staff ---
        Element firstStaff = staffList.get(0);          // get first staff
        Element ageElement = new Element("age");        // create new element
        ageElement.setText("30");                       // set its text value
        firstStaff.addContent(ageElement);              // add to first staff
        System.out.println("\nAdded <age> to first staff.");

        // --- UPDATE ATTRIBUTE: Change id of second staff to "99" ---
        Element secondStaff = staffList.get(1);
        secondStaff.setAttribute("id", "99");           // update attribute value
        System.out.println("Updated second staff id to: " +
            secondStaff.getAttributeValue("id"));

        // --- UPDATE VALUE: Change salary of second staff to 7000 ---
        secondStaff.getChild("salary").setText("7000");  // update text content
        System.out.println("Updated second staff salary to: " +
            secondStaff.getChildText("salary"));

        // --- DELETE: Remove <firstname> from second staff ---
        Element firstNameToDelete = secondStaff.getChild("firstname");
        firstNameToDelete.detach();                     // detach removes from tree
        System.out.println("Deleted <firstname> from second staff.");

        // ================================================
        // PART 3: Print final modified XML
        // ================================================
        XMLOutputter xmlOutputter = new XMLOutputter();
        xmlOutputter.setFormat(Format.getPrettyFormat()); // readable formatting
        System.out.println("\n--- Final Modified XML ---");
        xmlOutputter.output(document, System.out);      // print to console
    }
}
```

Code Walkthrough (Roman Urdu):

- `SAXBuilder saxBuilder = new SAXBuilder()` se JDOM ka parser banaya — yeh internally SAX use karta hai lekin humein directly SAX events handle nahi karne padte — JDOM sab kuch apne aap manage karta hai.
- `saxBuilder.build()` se XML stream parse hoki aur `Document` object ban gaya — poori tree memory mein available ho gayi just jaise Java DOM mein hoti hai.
- `rootElement.getChildren("staff")` ne seedha `List<Element>` return ki — yeh Java Collections ka faida hai jo standard DOM mein nahi milta — for-each loop directly chalaya.

- `getAttributeValue("id")` ne attribute ki string value di — `getAttribute()` pura `Attribute` object deta hai jabke `getAttributeValue()` seedha string deta hai — practical use mein `getAttributeValue()` zyada common hai.
- Modification mein `new Element("age")`, `setText()`, `addContent()` se naya node banaya aur tree mein daala — `setAttribute()` se attribute update hua — `getChild("salary").setText("7000")` se value change hui — `detach()` se node tree se hata diya.

Output:

```
Root element: company
Number of staff: 2
ID=1, Name=Ali, Salary=5000
ID=2, Name=Sara, Salary=6000

Added <age> to first staff.
Updated second staff id to: 99
Updated second staff salary to: 7000
Deleted <firstname> from second staff.

--- Final Modified XML ---
<company>
  <staff id="1">
    <firstname>Ali</firstname>
    <lastname>Khan</lastname>
    <salary>5000</salary>
    <age>30</age>
  </staff>
  <staff id="99">
    <lastname>Ahmed</lastname>
    <salary>7000</salary>
  </staff>
</company>
```

Yeh output confirm karta hai ke JDOM se XML modify karna bohot straightforward hai — `<age>` add hua, id update hua, salary change hui, aur `<firstname>` delete ho gaya — sab Java objects ki tarah treat hue.

Common Mistakes — Error Finder Session

Mistake 1: Using getChildren() when only one element expected — wrong return type

Buggy Code:

java
```
Element staff = rootElement.getChildren("staff"); // WRONG: type mismatch
// getChildren() returns List<Element>, not Element
System.out.println(staff.getName());
```

What's Wrong (Roman Urdu): `getChildren()` hamesha `List<Element>` return karta hai — ek akela `Element` nahi. Agar student directly `Element` variable mein assign kare toh compile error aayega kyunke types match nahi karte.

Fixed Code:

java
```
// Use getChildren() with List, or getChild() for single element
List<Element> staffList = rootElement.getChildren("staff"); // for multiple
Element firstStaff = rootElement.getChild("staff");        // for first one only
System.out.println(firstStaff.getName());
```

Fix Explained (Roman Urdu): Multiple children chahiye toh `getChildren()` use karo `List<Element>` mein, sirf pehla element chahiye toh `getChild()` use karo seedha `Element` mein — dono ka return type different hai.

Mistake 2: Calling detach() but still trying to use the detached element

Buggy Code:

java
```
Element firstName = secondStaff.getChild("firstname");
firstName.detach();    // removes element from tree

// WRONG: trying to read value after detaching
System.out.println("Deleted: " + secondStaff.getChildText("firstname"));
// returns null — element is gone
```

What's Wrong (Roman Urdu): `detach()` element ko XML tree se hata deta hai. Us ke baad `getChildText("firstname")` null return karta hai kyunke woh element ab tree mein exist nahi karta — student null output dekh ke confuse ho jaata hai.

Fixed Code:

java
```
// Read the value BEFORE detaching if you need it
Element firstName = secondStaff.getChild("firstname");
String deletedName = firstName.getText(); // save value before detach
firstName.detach();                       // now detach safely
System.out.println("Deleted element had value: " + deletedName); // uses saved value
```

Fix Explained (Roman Urdu): Agar delete hone wale element ki value baad mein chahiye toh pehle `getText()` se value save karo, phir `detach()` karo — baad mein woh value accessible nahi hogi.

Mistake 3: Confusing setText() with addContent() for updating values

Buggy Code:

java
```
Element salary = secondStaff.getChild("salary");
// WRONG: trying to add new text as content instead of replacing
salary.addContent("7000"); // addContent() expects Element or other Node, not String
```

What's Wrong (Roman Urdu): `addContent()` ek `Content` object ya `Element` expect karta hai — plain `String` pass karne par compile error aata hai. Text value update karne ke liye `setText()` method hoti hai, `addContent()` nahi.

Fixed Code:

java
```
Element salary = secondStaff.getChild("salary");
salary.setText("7000"); // CORRECT: setText() replaces the text content
System.out.println("Updated salary: " + salary.getText());
```

Fix Explained (Roman Urdu): Existing element ka text update karna ho toh `setText()` use karo — `addContent()` sirf nayi child nodes add karne ke liye hai, string values ke liye nahi.

Mistake 4: Forgetting JDOM jar dependency — import fails silently

Buggy Code:

java
```
import org.jdom2.*;       // compile error if jar not added
import org.jdom2.input.*;

public class JDOMTest {
    public static void main(String[] args) throws Exception {
        SAXBuilder builder = new SAXBuilder(); // cannot resolve symbol
    }
}
```

What's Wrong (Roman Urdu): JDOM Java Standard Library ka part nahi hai — agar `jdom2.jar` ya Maven dependency project mein add nahi ki toh `import org.jdom2.*` compile time par "cannot resolve symbol" error deta hai. Yeh ek common beginner mistake hai.

Fixed Code:

xml
```
<!-- Add to pom.xml for Maven projects -->
<dependency>
    <groupId>org.jdom</groupId>
```

```
    <artifactId>jdom2</artifactId>
    <version>2.0.6</version>
</dependency>
```

java

```
// After adding dependency, imports work correctly
import org.jdom2.*;
import org.jdom2.input.*;
import org.jdom2.output.*;
// Now SAXBuilder, Document, Element all resolve properly
```

Fix Explained (Roman Urdu): JDOM use karne se pehle uski library project mein add karna zaroori hai — Maven users `pom.xml` mein dependency daalen, otherwise manually `jdom2.jar` ko classpath mein add karen.

Conceptual Questions

Q: What is the difference between `getChild("salary")` and `getChildren("salary")` in JDOM, and what happens if you assign the result of `getChildren()` to an `Element` variable?

Answer: `getChild("salary")` specified naam ka pehla child `Element` directly return karta hai — return type `Element` hai. `getChildren("salary")` us naam ke saare child elements ki `List<Element>` return karta hai. Agar `getChildren()` ka result `Element` variable mein assign karo toh compile-time type mismatch error aayega.

- Practically: agar XML mein sirf ek `<salary>` hai toh `getChild("salary").getText()` seedha value deta hai. Lekin agar multiple `<item>` elements hain toh `getChildren("item")` se `List` lo aur for-each se sab iterate karo — dono ka use case different hai.

Q: In JAXB, what is the difference between marshalling and unmarshalling, and in which real-world scenario would you use each?

Answer: Marshalling ka matlab hai Java object ko XML mein convert karna — jaise ek `Employee` object liya aur usse XML file mein likh diya. Unmarshalling iska ulta hai — XML file ya stream liya aur usse Java `Employee` object mein convert kar diya. Dono operations binding runtime framework handle karta hai.

- Real-world mein: agar tumhara server kisi client ko data XML format mein bhejta hai toh Java object marshal karoge. Agar client ne XML data bheja aur tum use Java mein process karna chahte ho toh unmarshal karoge — jaise REST/SOAP web services mein request aur response handle karna.

Lesson Quiz

1. Which JDOM class is used to parse an XML file and create a `Document` object? (`SAXBuilder`)
2. What does `rootElement.getChildren("staff")` return in JDOM? (A `List<Element>` containing all `<staff>` child elements)
3. Which JDOM method is used to remove an element from the XML tree? (`detach()`)
4. In JAXB, what is the process of converting an XML document into a Java content tree called? (Unmarshalling)
5. Which JAXB component is responsible for generating Java classes from an XML schema? (Schema compiler)

SECTION 1 — LESSON HEADING

Lesson 2: The JAXB Binding Process

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:

Yeh bilkul waisi baat hai jaise tum Urdu ka koi paragraph likhna chahte ho aur phir usay English mein translate karna chahte ho. Pehle tum Urdu words ko samajhte ho (XML), phir unka English translation (Java objects) banate ho, phir agar zaroorat ho toh wapas Urdu mein convert karte ho. JAXB exactly yahi karta hai — XML aur Java objects ke darmiyaan translation ka kaam.

PART B — Key Content List:

Steps:

1. **Generate classes**: XML schema ko JAXB compiler mein daal kar JAXB classes generate karte hain jo schema ke hisaab se banti hain.
2. **Compile classes**: Generate hui classes, source files, aur application code ko compile karna padta hai taake program run ho sake.

3. **Unmarshal**: XML document ko JAXB framework ke zariye Java objects mein convert karte hain — jaise Urdu se English translation karna.
4. **Generate content tree**: Unmarshalling ke baad Java objects ka ek tree structure banta hai jo XML document ki structure aur content ko represent karta hai.
5. **Validate (optional)**: XML document ko validate kar sakte hain yeh check karne ke liye ke woh schema ke mutabiq hai ya nahi.
6. **Process content**: Client application Java content tree mein changes kar sakti hai — data modify karna, add karna, ya delete karna.
7. **Marshal**: Processed Java objects ko wapas XML document mein convert karte hain — jaise English se wapas Urdu translation.

Key Terms:

1. **Unmarshalling**: XML data ko JAXB-derived Java objects mein convert karne ka process hai — XML se Java ki taraf.
2. **Marshalling**: JAXB-derived Java objects ko wapas XML data mein convert karne ka process hai — Java se XML ki taraf.
3. **Validation**: Yeh check karne ka process hai ke XML document schema ki tamam constraints ko follow karta hai ya nahi.
4. **JAXB Annotations**: Yeh special tags hain jo Java classes mein lagaye jaate hain taake JAXB runtime samajh sake ke XML ke saath mapping kaise karni hai.
5. **Content Tree**: Yeh Java objects ka ek tree-like structure hai jo XML document ki complete representation hoti hai memory mein.
6. **XML Schema**: Yeh ek blueprint hai jo batata hai ke XML document mein kaun se elements allowed hain aur unka structure kya hai.

SECTION 3 — EXAMPLE

Example:

Socho tumhe ek library mein books ki list XML file mein di gayi hai aur tumhe usay Java program mein read karna hai. Pehle tum XML schema ke mutabiq JAXB classes generate

karte ho (jaise Book class), phir XML file ko unmarshal karke Book objects ki list banate ho, phir us list mein nayi book add karte ho, aur akhir mein wapas marshal karke nayi XML file bana dete ho. Yahi JAXB binding process hai — XML se Java, phir Java se XML.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
import javax.xml.bind.annotation.*;
import javax.xml.bind.*;
import java.io.File;
import java.util.ArrayList;
import java.util.List;

// Step 1: JAXB annotated class representing XML element
@XmlRootElement(name = "bookstore")  // Root element in XML
@XmlAccessorType(XmlAccessType.FIELD)  // Access fields directly
class Bookstore {
    @XmlElement(name = "book")  // Each book element
    private List<Book> books = new ArrayList<>();

    public List<Book> getBooks() { return books; }
    public void setBooks(List<Book> books) { this.books = books; }
}

// JAXB annotated class for Book
@XmlAccessorType(XmlAccessType.FIELD)
class Book {
    @XmlAttribute(name = "id")  // Book ID as attribute
    private int id;

    @XmlElement(name = "title")  // Title element
    private String title;
```

```
    @XmlElement(name = "author")  // Author element
    private String author;

    // Default constructor required for JAXB
    public Book() {}

    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    // Getters and setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
}

public class JAXBDemo {
    public static void main(String[] args) throws JAXBException {
        // Step 2: Create JAXB context and marshaller/unmarshaller
        JAXBContext context = JAXBContext.newInstance(Bookstore.class);
        Marshaller marshaller = context.createMarshaller();
        Unmarshaller unmarshaller = context.createUnmarshaller();

        // Step 3: Unmarshal - Convert XML to Java objects
        System.out.println("=== UNMARSHALLING: XML to Java ===");
        File xmlFile = new File("books.xml");  // Input XML file
        Bookstore bookstore = (Bookstore) unmarshaller.unmarshal(xmlFile);

        // Step 4: Process content - Display and modify data
        System.out.println("Books in XML:");
        for (Book book : bookstore.getBooks()) {
            System.out.println("ID: " + book.getId() +
                               ", Title: " + book.getTitle() +
                               ", Author: " + book.getAuthor());
```

```
        }

        // Add a new book to content tree
        bookstore.getBooks().add(new Book(3, "Java Programming", "John Doe"));
        System.out.println("\nAdded new book to Java objects");

        // Step 5: Marshal - Convert Java objects back to XML
        System.out.println("\n=== MARSHALLING: Java to XML ===");
        marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        marshaller.marshal(bookstore, System.out);
        System.out.println("\nXML created successfully with new book!");
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** Pehle humne `Bookstore` aur `Book` classes banayi hain jin par JAXB annotations lagaye hain — `@XmlRootElement` root element ko specify karta hai aur `@XmlElement` child elements ko.
- **Step 2:** `JAXBContext` create kiya hai jo JAXB runtime ko batata hai ke kaun si classes ko handle karna hai — yeh JAXB ka entry point hai.
- **Step 3:** `Unmarshaller` use karke hum `books.xml` file ko read karte hain aur usay Java object tree mein convert karte hain — yahi unmarshalling hai.
- **Step 4:** Content tree mein se books ko print karte hain aur phir ek nayi book add karte hain — yeh processing step hai jahan hum data modify kar sakte hain.
- **Step 5:** `Marshaller` use karke hum modified Java objects ko wapas XML mein convert karte hain aur console par print karte hain — yahi marshalling hai.

PART B — Expected Output:

```
text
=== UNMARSHALLING: XML to Java ===
Books in XML:
ID: 1, Title: Web Development, Author: Ahmed Khan
ID: 2, Title: Database Systems, Author: Sara Ali
```

```
Added new book to Java objects

=== MARSHALLING: Java to XML ===
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<bookstore>
    <book id="1">
        <title>Web Development</title>
        <author>Ahmed Khan</author>
    </book>
    <book id="2">
        <title>Database Systems</title>
        <author>Sara Ali</author>
    </book>
    <book id="3">
        <title>Java Programming</title>
        <author>John Doe</author>
    </book>
</bookstore>

XML created successfully with new book!
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke XML file successfully Java objects mein convert hui (unmarshalling), humne nayi book add ki, aur wapas XML mein convert kar ke (marshalling) nayi book ke saath complete XML generate ho gayi — yahi JAXB binding process ka complete cycle hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Missing Default Constructor

Buggy Code:

```
java
import javax.xml.bind.annotation.*;
```

```
@XmlRootElement
class Student {
    private String name;

    // No default constructor - only parameterized constructor
    public Student(String name) {
        this.name = name;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Student.class);
        Unmarshaller unmarshaller = context.createUnmarshaller();
        Student s = (Student) unmarshaller.unmarshal(new File("student.xml"));
    }
}
```

What's Wrong (Roman Urdu): Student class mein default constructor nahi hai, sirf parameterized constructor hai. JAXB unmarshalling ke waqt object create karne ke liye default constructor call karta hai, agar nahi hai toh exception aati hai.

Fixed Code:

```
java
import javax.xml.bind.annotation.*;

@XmlRootElement
class Student {
    private String name;

    // Default constructor required for JAXB
    public Student() {}

    public Student(String name) {
        this.name = name;
```

```
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Student.class);
        Unmarshaller unmarshaller = context.createUnmarshaller();
        Student s = (Student) unmarshaller.unmarshal(new File("student.xml"));
    }
}
```

Fix Explained (Roman Urdu): Default constructor add kar diya taake JAXB unmarshalling ke waqt object create kar sake aur runtime exception se bach sake.

Mistake 2: Forgetting @XmlAccessorType Annotation

Buggy Code:

```
java
import javax.xml.bind.annotation.*;

@XmlRootElement
class Person {
    private String name;  // Private field without getter/setter
    private int age;
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Person.class);
        Marshaller m = context.createMarshaller();
        Person p = new Person();
```

```
        p.name = "Ali";  // Error - name has private access
        m.marshal(p, System.out);
    }
}
```

What's Wrong (Roman Urdu): Student ne private fields ko access karne ki koshish ki hai lekin getter/setter nahi banaye aur na hi `@XmlAccessorType` annotation lagayi, is liye JAXB fields ko access nahi kar paata.

Fixed Code:

```
java
import javax.xml.bind.annotation.*;

@XmlRootElement
@XmlAccessorType(XmlAccessType.FIELD)  // Allow JAXB to access private fields
class Person {
    private String name;
    private int age;

    // Getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Person.class);
        Marshaller m = context.createMarshaller();
        Person p = new Person();
        p.setName("Ali");  // Using setter method
        m.marshal(p, System.out);
    }
}
```

Fix Explained (Roman Urdu): `@XmlAccessorType(XmlAccessType.FIELD)` annotation lagaytaake JAXB private fields ko bhi access kar sake, aur getter/setter bana kar proper encapsulation follow ki.

Mistake 3: Incorrect JAXBContext Class Name

Buggy Code:

java

```
import javax.xml.bind.*;

@XmlRootElement
class Product {
    private String name;
    // getters and setters
}

public class Test {
    public static void main(String[] args) throws Exception {
        // Wrong class passed - JAXBContext expects the root class
        JAXBContext context = JAXBContext.newInstance("Product");
        Marshaller m = context.createMarshaller();
        Product p = new Product();
        p.setName("Laptop");
        m.marshal(p, System.out);
    }
}
```

What's Wrong (Roman Urdu): Student ne `JAXBContext.newInstance()` mein string "Product" pass kiya hai lekin yeh method class object expect karta hai, string nahi. Isliye compilation error aayegi.

Fixed Code:

java

```java
import javax.xml.bind.*;

@XmlRootElement
class Product {
    private String name;
    // getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        // Pass the class object correctly
        JAXBContext context = JAXBContext.newInstance(Product.class);
        Marshaller m = context.createMarshaller();
        Product p = new Product();
        p.setName("Laptop");
        m.marshal(p, System.out);
    }
}
```

Fix Explained (Roman Urdu): `JAXBContext.newInstance()` mein `Product.class` pass kiya (class object) taake JAXB runtime ko pata chal sake ke kaun si class ko handle karna hai.

Mistake 4: Trying to Marshal Without Setting Property

Buggy Code:

java

```java
import javax.xml.bind.*;

@XmlRootElement
class Customer {
    private String name;
    // getters and setters
```

```
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Customer.class);
        Marshaller m = context.createMarshaller();
        Customer c = new Customer();
        c.setName("Sara");
        // Forgot to set JAXB_FORMATTED_OUTPUT property
        m.marshal(c, System.out);
    }
}
```

What's Wrong (Roman Urdu): Student ne marshaller par `JAXB_FORMATTED_OUTPUT` property set nahi ki, is liye XML ek single line mein aayegi jo read karna mushkil ho jayega.

Fixed Code:

```
java
import javax.xml.bind.*;

@XmlRootElement
class Customer {
    private String name;
    // getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Customer.class);
        Marshaller m = context.createMarshaller();
        // Set formatted output property for readable XML
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        Customer c = new Customer();
        c.setName("Sara");
        m.marshal(c, System.out);
```

```
    }
}
```

Fix Explained (Roman Urdu): `m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true)` add kiya taake XML formatted aur readable form mein print ho.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen when the following code is executed and why?

```
java
@XmlRootElement
class Employee {
    private String name;

    // No default constructor
    public Employee(String name) {
        this.name = name;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        JAXBContext context = JAXBContext.newInstance(Employee.class);
        Unmarshaller unmarshaller = context.createUnmarshaller();
        Employee emp = (Employee) unmarshaller.unmarshal(new File("emp.xml"));
        System.out.println(emp.getName());
    }
}
```

Answer: Is code mein JAXBException aayegi kyunke `Employee` class mein default constructor nahi hai. JAXB unmarshalling ke waqt object create karne ke liye default constructor call karta hai. Agar default constructor nahi hai toh JAXB object nahi bana paata aur exception throw karta hai. JAXB ke liye har JAXB-annotated class mein default constructor hona zaroori hai.

Q: What is the difference between marshalling and unmarshalling in terms of data flow and when would you use each in a Java web service application?

Answer: Unmarshalling mein data XML se Java objects ki taraf flow karta hai — iska use tab hota hai jab humein XML data ko read karna ho aur usay Java program mein process karna ho, jaise web service se XML response aata hai toh usay Java objects mein convert karte hain. Marshalling mein data Java objects se XML ki taraf flow karta hai — iska use tab hota hai jab humein Java objects ko XML mein convert karna ho aur client ko bhejna ho, jaise web service client ko response bhejne se pehle Java object ko XML mein convert karte hain.

Web service mein typically unmarshalling request data ke liye aur marshalling response data ke liye use hota hai.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. What happens if you try to unmarshal an XML that doesn't match the schema structure? (JAXBException is thrown due to validation failure)

2. Which annotation must be added to a Java class to make it the root element for JAXB? (@XmlRootElement)
3. What will this code output?

```java
Marshaller m = context.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(obj, System.out);
```

(XML output will be formatted with proper indentation and line breaks)

4. What is missing in this JAXB code that would cause a runtime error?

```java
@XmlRootElement
class Book {
    @XmlElement private String title;
    public Book(String title) { this.title = title; }
}
```

(Missing default constructor - JAXB needs it for unmarshalling)

5. Which JAXB operation would you use to convert an XML file into Java objects? (Unmarshalling)

SECTION 1 — LESSON HEADING

Lesson 3: Marshalling and Unmarshalling in JAXB

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum apni dairy mein Urdu mein likhte ho (Java objects), aur phir usay apne dost ko English mein translate karke bhejte ho (XML). Jab dost wapas Urdu mein convert karta hai (unmarshalling), toh woh tumhari original cheez samajh leta

hai. Marshalling Java objects ko XML mein convert karta hai, aur unmarshalling XML ko wapas Java objects mein.

PART B — Key Content List:

Key Terms:

1. **Marshalling**: Java object ki memory representation ko XML format mein convert karne ka process hai, taake data store ya transmit ho sake.
2. **Unmarshalling**: XML data ko wapas Java objects mein convert karne ka process hai, taake program us data ko process kar sake.
3. **JAXBContext**: Yeh JAXB API ka entry point hai jo XML aur Java ke darmiyaan binding information maintain karta hai.
4. **Unmarshaller**: Yeh JAXB object hai jo XML text ko process karta hai aur Java objects ka equivalent set create karta hai.
5. **Marshaller**: Yeh JAXB object hai jo Java objects ko XML text mein convert karta hai.
6. **JAXBElement<T>**: Yeh wrapper class hai jo parsed XML document ke root node ko represent karti hai.
7. **Package Name**: JAXB context create karte waqt hum packages ka naam dete hain jo JAXB schema-derived classes contain karte hain.

Steps:

1. **Create JAXBContext**: Static method `newInstance()` call karte hain with package names containing JAXB classes.
2. **Create Unmarshaller**: JAXBContext se Unmarshaller object create karte hain.
3. **Call unmarshal()**: Unmarshaller ki `unmarshal()` method call karte hain with XML source (file, input stream, etc.).
4. **Get JAXBElement**: Unmarshalling se `JAXBElement<T>` object milta hai jo root node represent karta hai.
5. **Extract Value**: `getValue()` method call karke actual Java object extract karte hain.

6. **Create Marshaller**: JAXBContext se Marshaller object create karte hain.
7. **Call marshal()**: Marshaller ki `marshal()` method call karte hain with Java object and output destination.

SECTION 3 — EXAMPLE

Example:

Socho tumhe university ki student list XML file mein mili hai aur tum usay Java program mein read karna chahte ho. Tum JAXBContext banate ho, Unmarshaller use karte ho XML ko Java objects mein convert karne ke liye (unmarshalling). Phir tum us list mein naye student add karte ho, aur Marshaller use karte ho updated list ko wapas XML mein convert karne ke liye (marshalling) taake woh file mein save ho jaye. Yahi marshalling aur unmarshalling ka practical use hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
import javax.xml.bind.*;
import javax.xml.bind.annotation.*;
import java.io.*;
import java.util.*;

// JAXB annotated class
@XmlRootElement(name = "student")
@XmlAccessorType(XmlAccessType.FIELD)
class Student {
    @XmlAttribute(name = "id")
    private int id;

    @XmlElement(name = "name")
```

```
    private String name;

    @XmlElement(name = "age")
    private int age;

    // Default constructor required for JAXB
    public Student() {}

    public Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    // Getters and setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "', age=" + age + "}";
    }
}

// Generic helper class for marshalling/unmarshalling
class JAXBHelper {
    // Generic unmarshalling method
    public static <T> T unmarshal(Class<T> docClass, InputStream inputStream)
            throws JAXBException {
        // Get package name from class
        String packageName = docClass.getPackage().getName();

        // Create JAXB context with package name
        JAXBContext jc = JAXBContext.newInstance(packageName);
```

```
        // Create unmarshaller
        Unmarshaller u = jc.createUnmarshaller();

        // Unmarshal XML to JAXBElement
        JAXBElement<T> doc = (JAXBElement<T>) u.unmarshal(inputStream);

        // Return the extracted content value
        return doc.getValue();
    }

    // Generic marshalling method
    public static <T> void marshal(T object, OutputStream outputStream)
            throws JAXBException {
        // Get package name from object's class
        String packageName = object.getClass().getPackage().getName();

        // Create JAXB context
        JAXBContext jc = JAXBContext.newInstance(packageName);

        // Create marshaller
        Marshaller m = jc.createMarshaller();

        // Set formatted output
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);

        // Marshal object to XML
        m.marshal(object, outputStream);
    }
}

public class MarshallingDemo {
    public static void main(String[] args) {
        try {
            // 1. UNMARSHALLING: XML to Java Object
            System.out.println("=== UNMARSHALLING DEMO ===");

            // Create XML string as input
            String xmlData = "<?xml version=\"1.0\" encoding=\"UTF-8\"?>" +
                             "<student id=\"101\">" +
```

```
                "<name>Ahmed Ali</name>" +
                "<age>22</age>" +
                "</student>";

// Convert string to InputStream
InputStream inputStream = new ByteArrayInputStream(xmlData.getBytes());

// Unmarshal XML to Student object
Student student = JAXBHelper.unmarshal(Student.class, inputStream);
System.out.println("Unmarshalled Student: " + student);

// 2. MARSHALLING: Java Object to XML
System.out.println("\n=== MARSHALLING DEMO ===");

// Create a new Student object
Student newStudent = new Student(102, "Fatima Noor", 23);

// Marshal to XML and print to console
System.out.println("Marshalling Student object to XML:");
ByteArrayOutputStream baos = new ByteArrayOutputStream();
JAXBHelper.marshal(newStudent, baos);
System.out.println(baos.toString());

// 3. Working with multiple objects
System.out.println("\n=== MARSHALLING LIST DEMO ===");
List<Student> students = new ArrayList<>();
students.add(new Student(101, "Ahmed Ali", 22));
students.add(new Student(102, "Fatima Noor", 23));
students.add(new Student(103, "Usman Khan", 21));

// Wrap list in a root element
StudentListWrapper wrapper = new StudentListWrapper(students);

// Marshal the list
ByteArrayOutputStream listOutput = new ByteArrayOutputStream();
JAXBHelper.marshal(wrapper, listOutput);
System.out.println("Student List as XML:");
System.out.println(listOutput.toString());
```

```
        } catch (JAXBException e) {
            System.err.println("JAXB Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

// Wrapper class for list of students
@XmlRootElement(name = "students")
@XmlAccessorType(XmlAccessType.FIELD)
class StudentListWrapper {
    @XmlElement(name = "student")
    private List<Student> students;

    public StudentListWrapper() {}

    public StudentListWrapper(List<Student> students) {
        this.students = students;
    }

    public List<Student> getStudents() { return students; }
    public void setStudents(List<Student> students) { this.students = students; }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** Pehle humne `Student` class banayi hai jis par JAXB annotations lagaye hain — `@XmlRootElement` root element ke liye, `@XmlAttribute` ID ke liye, aur `@XmlElement` name aur age ke liye. Default constructor bhi add kiya hai jo JAXB ke liye zaroori hai.
- **Step 2:** `JAXBHelper` class mein generic `unmarshal()` method banayi hai — yeh kisi bhi class ke liye kaam karti hai. Yeh pehle package name extract karti hai, phir JAXBContext create karti hai, phir Unmarshaller banati hai, aur akhir mein XML ko Java object mein convert karti hai.

- **Step 3:** `unmarshal()` method mein `JAXBElement<T>` use kiya hai kyunke unmarshalling root node ko `JAXBElement` return karta hai. Phir `getValue()` call karke actual object extract karte hain.
- **Step 4:** `JAXBHelper` mein `marshal()` method banayi hai jo generic hai — yeh kisi bhi object ko XML mein convert karti hai. `setProperty()` se formatted output enable kiya hai taake XML readable ho.
- **Step 5:** `main` method mein pehle hum XML string se Student object banate hain (unmarshalling), phir ek naya Student object XML mein convert karte hain (marshalling), aur akhir mein students ki list ko XML mein convert karte hain — yeh complete cycle dikhata hai.

PART B — Expected Output:

```
text
=== UNMARSHALLING DEMO ===
Unmarshalled Student: Student{id=101, name='Ahmed Ali', age=22}

=== MARSHALLING DEMO ===
Marshalling Student object to XML:
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<student id="102">
    <name>Fatima Noor</name>
    <age>23</age>
</student>

=== MARSHALLING LIST DEMO ===
Student List as XML:
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<students>
    <student id="101">
        <name>Ahmed Ali</name>
        <age>22</age>
    </student>
    <student id="102">
        <name>Fatima Noor</name>
        <age>23</age>
```

```
    </student>
    <student id="103">
        <name>Usman Khan</name>
        <age>21</age>
    </student>
</students>
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke unmarshalling ne XML string ko successfully Student object mein convert kiya, marshalling ne Student object ko formatted XML mein convert kiya, aur list marshalling ne multiple students ko ek proper XML structure mein convert kiya — yeh demonstrate karta hai ke JAXB dono directions mein kaam karta hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Passing Class Object Instead of Package Name

Buggy Code:

```
java
public static <T> T unmarshal(Class<T> docClass, InputStream inputStream)
        throws JAXBException {
    // Wrong: Passing class object instead of package name
    JAXBContext jc = JAXBContext.newInstance(docClass);
    Unmarshaller u = jc.createUnmarshaller();
    JAXBElement<T> doc = (JAXBElement<T>) u.unmarshal(inputStream);
    return doc.getValue();
}
```

What's Wrong (Roman Urdu): Student ne `JAXBContext.newInstance()` mein class object pass kiya hai lekin method expects package name string ya multiple classes. Is tarah context sahi se initialize nahi hoga aur runtime exception aayegi.

Fixed Code:

```java
public static <T> T unmarshal(Class<T> docClass, InputStream inputStream)
        throws JAXBException {
    // Correct: Get package name from class
    String packageName = docClass.getPackage().getName();
    JAXBContext jc = JAXBContext.newInstance(packageName);
    Unmarshaller u = jc.createUnmarshaller();
    JAXBElement<T> doc = (JAXBElement<T>) u.unmarshal(inputStream);
    return doc.getValue();
}
```

Fix Explained (Roman Urdu): Class se package name extract karke `newInstance()` mein pass kiya, kyunke JAXBContext multiple packages ki classes ko handle karta hai aur package name string leta hai.

Mistake 2: Forgetting to Handle JAXBElement Return Type

Buggy Code:

```java
public static <T> T unmarshal(Class<T> docClass, InputStream inputStream)
        throws JAXBException {
    String packageName = docClass.getPackage().getName();
    JAXBContext jc = JAXBContext.newInstance(packageName);
    Unmarshaller u = jc.createUnmarshaller();
    // Wrong: Directly casting to T without JAXBElement wrapper
    T doc = (T) u.unmarshal(inputStream);
    return doc;
}
```

What's Wrong (Roman Urdu): Unmarshaller ki `unmarshal()` method `JAXBElement<T>` return karti hai, lekin student direct cast kar raha hai. Is se ClassCastException aayegi kyunke return type JAXBElement hai, T nahi.

Fixed Code:

```java
public static <T> T unmarshal(Class<T> docClass, InputStream inputStream)
        throws JAXBException {
    String packageName = docClass.getPackage().getName();
    JAXBContext jc = JAXBContext.newInstance(packageName);
    Unmarshaller u = jc.createUnmarshaller();
    // Correct: Unmarshal to JAXBElement then extract value
    JAXBElement<T> doc = (JAXBElement<T>) u.unmarshal(inputStream);
    return doc.getValue();
}
```

Fix Explained (Roman Urdu): Pehle `JAXBElement<T>` mein cast kiya, phir `getValue()` call karke actual object extract kiya — yahi JAXB ka standard pattern hai.

Mistake 3: Not Setting JAXB_FORMATTED_OUTPUT Property

Buggy Code:

```java
public static <T> void marshal(T object, OutputStream outputStream)
        throws JAXBException {
    String packageName = object.getClass().getPackage().getName();
    JAXBContext jc = JAXBContext.newInstance(packageName);
    Marshaller m = jc.createMarshaller();
    // Forgot to set formatted output property
    m.marshal(object, outputStream);
}
```

What's Wrong (Roman Urdu): Student ne marshaller par `JAXB_FORMATTED_OUTPUT` property set nahi ki, is liye XML ek single line mein aayegi jo human-readable nahi hogi aur debugging mushkil ho jayegi.

Fixed Code:

```java
```

```
public static <T> void marshal(T object, OutputStream outputStream)
        throws JAXBException {
    String packageName = object.getClass().getPackage().getName();
    JAXBContext jc = JAXBContext.newInstance(packageName);
    Marshaller m = jc.createMarshaller();
    // Set formatted output for readability
    m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
    m.marshal(object, outputStream);
}
```

Fix Explained (Roman Urdu): `setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true)` add kiya taake XML formatted ho aur proper indentation ke saath print ho.

Mistake 4: Missing @XmlRootElement Annotation

Buggy Code:

```
java
import javax.xml.bind.annotation.*;

// Missing @XmlRootElement annotation
@XmlAccessorType(XmlAccessType.FIELD)
class Product {
    @XmlElement private String name;
    @XmlElement private double price;

    public Product() {}
    // getters and setters
}

public class Test {
    public static void main(String[] args) throws Exception {
        Product p = new Product();
        p.setName("Laptop");
        p.setPrice(50000);
```

```
        JAXBContext context = JAXBContext.newInstance(Product.class);
        Marshaller m = context.createMarshaller();
        m.marshal(p, System.out);  // This will cause error
    }
}
```

What's Wrong (Roman Urdu): `Product` class par `@XmlRootElement` annotation nahi hai, is liye JAXB ko nahi pata ke root element ka naam kya hai. Marshalling ke waqt error aayegi kyunke root element specify nahi hai.

Fixed Code:

```
java
import javax.xml.bind.annotation.*;

@XmlRootElement(name = "product")  // Added root element annotation
@XmlAccessorType(XmlAccessType.FIELD)
class Product {
    @XmlElement private String name;
    @XmlElement private double price;

    public Product() {}
    // getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        Product p = new Product();
        p.setName("Laptop");
        p.setPrice(50000);

        JAXBContext context = JAXBContext.newInstance(Product.class);
        Marshaller m = context.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
```

```
        m.marshal(p, System.out);  // Now works correctly
    }
}
```

Fix Explained (Roman Urdu): `@XmlRootElement(name = "product")` annotation add ki taake JAXB ko root element ka naam pata chal sake aur marshalling successfully ho jaye.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen when the following code is executed and why?

```
java
@XmlRootElement
class Book {
    private String title;
    public Book(String title) { this.title = title; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
}

public class Test {
    public static void main(String[] args) throws Exception {
        Book book = new Book("Java Basics");
        JAXBContext context = JAXBContext.newInstance(Book.class);
        Marshaller m = context.createMarshaller();
        m.marshal(book, System.out);
    }
}
```

Answer: Is code mein JAXBException aayegi kyunke `Book` class mein default constructor nahi hai. JAXB marshalling ke waqt bhi object ko inspect karta hai aur agar default constructor nahi hai toh framework object create nahi kar paata. JAXB ke liye har annotated class mein default constructor hona zaroori hai, chahe marshalling ho ya unmarshalling.

Q: Why do we need to wrap the result of `unmarshal()` in `JAXBElement<T>` and then call `getValue()` instead of directly casting to the target type?

Answer: JAXB ka `unmarshal()` method root element ko `JAXBElement<T>` wrapper mein return karta hai kyunke yeh root element ki metadata (jaise namespace aur tag name) ko preserve karta hai. Agar hum direct cast karein toh ClassCastException aayegi kyunke return type `JAXBElement` hai, target class nahi. `getValue()` call karke hum actual object extract karte hain jo humein chahiye. Agar humein root element ki metadata ki zaroorat nahi hai toh sirf value extract kar sakte hain.

Jaise agar XML mein `<book id="101">` hai toh JAXBElement book ki metadata (id attribute) ke saath return karega, aur `getValue()` sirf Book object degi.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method call is used to convert a Java object to XML in JAXB? (marshaller.marshal(object, outputStream))
2. What will this code print?

```
java
JAXBElement<Student> doc = (JAXBElement<Student>) u.unmarshal(inputStream);
System.out.println(doc.getValue().getName());
```

(Prints the name of the Student object extracted from XML)

3. What is the purpose of `JAXBContext.newInstance(packageName)` in the unmarshalling process? (Creates JAXB context with binding information for all classes in that package)
4. What annotation is missing if marshalling fails with "unexpected element" error? (@XmlRootElement on the class)

5. What happens if you forget to call `m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true)` before marshalling? (XML output appears as a single unformatted line without indentation)

SECTION 1 — LESSON HEADING

Lesson 4: Introduction to Servlets and HTTP Methods

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:

Yeh bilkul waisi baat hai jaise tum kisi restaurant mein jaate ho aur waiter tumhara order leta hai, kitchen mein chef us order ko prepare karta hai, aur phir waiter tumhe khana serve karta hai. Servlet bhi yahi karta hai — web browser (client) se request aati hai, server par servlet us request ko process karta hai, aur dynamic web page generate karke response bhejta hai.

PART B — Key Content List:

Key Terms:

1. **Servlet**: Ek Java technology hai jo server-side par web applications banane ke liye use hoti hai aur dynamic web pages generate karti hai.
2. **Web Application**: Ek application jo web ke zariye accessible hoti hai aur web components (Servlet, JSP, Filter) aur HTML etc. se bani hoti hai.
3. **CGI (Common Gateway Interface)**: Purani technology hai jo web server ko external program call karne deti thi, lekin har request ke liye naya process start karti thi — yeh slow thi.
4. **HTTP (Hyper Text Transfer Protocol)**: Yeh protocol hai jo web servers aur browsers ko internet par data exchange karne deta hai — request-response model follow karta hai.
5. **Container**: Yeh servlet ka environment hai jo servlet lifecycle manage karta hai, requests handle karta hai, aur resources provide karta hai.
6. **Stateless**: HTTP ka yeh property hai ke har request nayi maani jaati hai — server user ko default tor par recognize nahi karta.
7. **Deployment**: Servlet ko web server par install aur configure karne ka process hai taake woh client requests ko handle kar sake.

Key Points:

1. **Performance**: Servlet har request ke liye naya thread create karta hai (process nahi), is liye CGI se zyada fast aur efficient hai.
2. **Portability**: Java mein likha honay ki wajah se servlet kisi bhi platform par run ho sakta hai.
3. **Robust**: JVM servlets ko manage karta hai, is liye memory leak aur garbage collection ki tension nahi hoti.
4. **Secure**: Java ki security features ki wajah se servlets safe hain.

Types:

1. **GenericServlet**: Protocol-independent servlet hai jo kisi bhi protocol ke saath kaam kar sakta hai.
2. **HttpServlet**: HTTP protocol ke liye specific servlet hai — web applications mein sab se zyada use hota hai.

Superstructure:

1. **Servlet Interface**: Sab se basic interface hai jise har servlet implement karna zaroori hai.
2. **ServletRequest**: Client request ki information contain karta hai.
3. **ServletResponse**: Server response ki information contain karta hai.
4. **ServletConfig**: Servlet ke initialization parameters provide karta hai.
5. **ServletContext**: Web application ke global information provide karta hai.

SECTION 3 — EXAMPLE

Example:

Socho tum university ki website par admission form fill kar rahe ho. Tum form submit karte ho (HTTP request), server par servlet us data ko process karta hai (validate karna, database mein save karna), aur phir tumhe confirmation page dikhata hai (dynamic response). Agar tum form mein koi mistake karte ho toh servlet error message ke saath wapas form bhej deta hai. Yahi servlet ka kaam hai — request process karna aur dynamic response generate karna.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

// Basic Servlet example - extends HttpServlet
public class HelloWorldServlet extends HttpServlet {

    // init() method - called once when servlet is loaded
    @Override
    public void init() throws ServletException {
        System.out.println("Servlet initialized!");
    }

    // doGet() method - handles HTTP GET requests
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // Set response content type
        response.setContentType("text/html");

        // Get PrintWriter to write response
        PrintWriter out = response.getWriter();

        // Generate dynamic HTML page
        out.println("<html>");
        out.println("<head><title>Hello Servlet</title></head>");
        out.println("<body>");
        out.println("<h1>Hello from Servlet!</h1>");
        out.println("<p>This is a dynamic web page generated by servlet</p>");

        // Get parameter from request
        String name = request.getParameter("name");
        if (name != null && !name.isEmpty()) {
            out.println("<p>Welcome, " + name + "!</p>");
        } else {
```

```
            out.println("<p>No name provided. Use ?name=YourName</p>");
        }

        out.println("</body></html>");
    }

    // doPost() method - handles HTTP POST requests
    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // For POST, we can process form data
        String name = request.getParameter("name");
        String email = request.getParameter("email");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h1>Form Submission Received</h1>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Email: " + email + "</p>");
        out.println("</body></html>");
    }

    // destroy() method - called when servlet is unloaded
    @Override
    public void destroy() {
        System.out.println("Servlet destroyed!");
    }
}

// Example servlet showing GET vs POST difference
class UserLoginServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

```
        // GET request shows the login form
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Login Form</h2>");
        out.println("<form method='POST' action='login'>");
        out.println("Username: <input type='text' name='username'><br>");
        out.println("Password: <input type='password' name='password'><br>");
        out.println("<input type='submit' value='Login'>");
        out.println("</form>");
        out.println("</body></html>");
    }

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // POST request processes login credentials
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        if ("admin".equals(username) && "123".equals(password)) {
            out.println("<h2>Login Successful!</h2>");
            out.println("<p>Welcome " + username + "</p>");
        } else {
            out.println("<h2>Login Failed!</h2>");
            out.println("<p>Invalid credentials. <a href='login'>Try again</a></p>");
        }
        out.println("</body></html>");
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** Humne `HelloWorldServlet` banayi hai jo `HttpServlet` class extend karti hai — yeh HTTP requests handle karne ke liye base class hai. `doGet` aur `doPost` methods override kiye hain.
- **Step 2:** `init()` method sirf ek baar call hoti hai jab servlet pehli baar load hoti hai — isme hum initialization code likhte hain jaise database connection.
- **Step 3:** `doGet()` method GET requests handle karti hai — isme `request.getParameter()` se hum URL parameters read karte hain. `response.setContentType()` se response ka type set karte hain aur `getWriter()` se HTML likhte hain.
- **Step 4:** `doPost()` method POST requests handle karti hai — isme hum form data process karte hain. POST mein data URL mein nahi dikhta, body mein bheja jata hai.
- **Step 5:** `destroy()` method tab call hoti hai jab servlet server se unload hoti hai — cleanup code yahan likhte hain.
- **Step 6:** `UserLoginServlet` example mein GET login form dikhata hai aur POST credentials process karta hai — yeh GET aur POST ka practical difference dikhata hai.

PART B — Expected Output:

```
text
// When accessing HelloWorldServlet with GET request
// Browser displays:

Hello from Servlet!
This is a dynamic web page generated by servlet
Welcome, Ahmed!

// When accessing login page with GET request
// Browser displays login form

// When submitting login form with POST request
// Browser displays:

Login Successful!
Welcome admin

// OR

Login Failed!
Invalid credentials. Try again
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke servlet dynamic HTML generate kar raha hai — GET request par welcome message ke saath page aaya, aur POST request par login credentials process ho kar success ya failure message show ho raha hai. Yeh servlet ka real-world behavior hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Missing doGet/doPost Method Override

Buggy Code:

```
java
import javax.servlet.http.*;

public class TestServlet extends HttpServlet {
    // Missing doGet method
    public void doPost(HttpServletRequest request, HttpServletResponse response) {
        // Some code
    }
}
```

What's Wrong (Roman Urdu): Student ne `doGet()` method override nahi ki, agar client GET request bhejta hai toh servlet response nahi de payega aur HTTP 405 error aayegi. Har servlet mein kam se kam ek HTTP method override hona chahiye.

Fixed Code:

```
java
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;

public class TestServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
        out.println("<h1>GET Request Handled</h1>");
    }

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>POST Request Handled</h1>");
    }
}
```

Fix Explained (Roman Urdu): `doGet()` method add kar di taake servlet dono GET aur POST requests handle kar sake aur HTTP 405 error na aaye.

Mistake 2: Forgetting to Set Content Type

Buggy Code:

```
java
public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    PrintWriter out = response.getWriter();
    out.println("<html><body>Hello</body></html>");
    // Missing: response.setContentType("text/html");
}
```

What's Wrong (Roman Urdu): Student ne content type set nahi kiya, is liye browser HTML ko plain text samajh kar display karega, render nahi karega. Content type batata hai ke response mein kis type ka data hai.

Fixed Code:

```
java
public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    // Set content type before writing response
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<html><body><h1>Hello</h1></body></html>");
```

```
}
```

Fix Explained (Roman Urdu): `setContentType("text/html")` pehle call kiya taake browser ko pata chal sake ke response HTML hai, is liye woh properly render karega.

Mistake 3: Using Wrong Import Statements

Buggy Code:

```
java
// Wrong imports
import java.servlet.*;
import java.servlet.http.*;

public class MyServlet extends HttpServlet {
    // Code
}
```

What's Wrong (Roman Urdu): Student ne `javax.servlet` ki jagah `java.servlet` import kiya hai — Java mein servlet packages `javax` se start hotay hain, `java` se nahi. Is liye compilation error aayegi.

Fixed Code:

```
java
// Correct imports
import javax.servlet.*;
import javax.servlet.http.*;

public class MyServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Code
    }
}
```

Fix Explained (Roman Urdu): Sahi import statements `javax.servlet.*` aur `javax.servlet.http.*` use kiya, kyunke servlet API `javax` package mein available hai.

Mistake 4: Not Handling IOException/UnsupportedEncodingException

Buggy Code:

```
java
public class TestServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response) {
        // Missing throws declaration or try-catch
        PrintWriter out = response.getWriter();
        out.println("Hello");
    }
}
```

What's Wrong (Roman Urdu): Student ne `getWriter()` ke exception `IOException` ko handle nahi kiya, na hi method signature mein throws clause add kiya. Is liye compilation error aayegi.

Fixed Code:

```
java
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;

public class TestServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {  // Added throws
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>Hello</h1>");
    }
}
```

Fix Explained (Roman Urdu): Method signature mein `throws ServletException, IOException` add kiya taake exceptions properly propagate ho sakein aur code compile ho jaye.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if a client sends a POST request to a servlet that only has a doGet() method implementation? Explain with code behavior.

Answer: Agar servlet mein sirf `doGet()` method hai aur client POST request bhejta hai, toh server HTTP 405 "Method Not Allowed" error return karega. Kyunke servlet POST method handle nahi kar rahi, container default implementation call karega jo error generate karti hai. Is problem ko solve karne ke liye hum `doPost()` method override karein, ya agar dono methods same kaam karte hain toh `doPost()` ke andar `doGet()` call kar sakte hain.

Jaise:

```
java
public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    doGet(request, response);  // Reuse GET logic
}
```

Q: What is the difference between GET and POST methods in terms of how data is transmitted and when would you use each in a web application?

Answer: GET method mein data URL ke saath query string mein bheja jata hai (`?name=Ali&age=22` ki tarah), is liye data URL mein visible hota hai aur browser history mein save ho jata hai. POST method mein data HTTP request body mein bheja jata hai, is liye URL mein dikhta nahi aur zyada secure hai.

GET use karte hain jab data sensitive nahi hai aur hum bookmark/share karna chahte hain, jaise search queries. POST use karte hain jab sensitive data ho (passwords, credit card info) ya jab data modify ho raha ho (form submission, database update). GET ki length limit hoti hai (2048 characters) jabke POST ki koi limit nahi hoti.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method is called when a servlet receives an HTTP GET request? (doGet(HttpServletRequest, HttpServletResponse))
2. What happens if a servlet doesn't override doPost() and receives a POST request? (HTTP 405 Method Not Allowed error is returned)
3. What will this code output if accessed with URL "?name=Ali"?

   ```java
   String name = request.getParameter("name");
   if(name == null) out.print("No name");
   else out.print("Hello " + name);
   ```

   (Prints "Hello Ali" because parameter is present)

4. Which import statement is required to use HttpServlet class? (import javax.servlet.http.HttpServlet)
5. What is the difference between CGI and Servlet in terms of request handling? (CGI creates new process for each request, Servlet creates new thread which is more efficient)

SECTION 1 — LESSON HEADING

Lesson 5: Servlet Lifecycle and HTTP Methods

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum ek restaurant kholte ho. Pehle tum kitchen set up karte ho (init), phir customers aate hain aur tum unka order letay ho aur khana serve karte ho (service), aur akhir mein restaurant band karte ho aur saaf safai karte ho (destroy). Servlet ka lifecycle bhi yahi hai — creation, request processing, aur destruction.

PART B — Key Content List:

Steps:

1. **Initialization (init)**: Servlet pehli baar create hoti hai aur `init()` method call hota hai — yeh sirf ek baar hota hai, one-time setup ke liye.
2. **Request Processing (service)**: Har client request ke liye container `service()` method call karta hai — yeh request type check karta hai aur appropriate doGet/doPost method call karta hai.
3. **Destruction (destroy)**: Servlet lifecycle ke end mein `destroy()` method call hota hai — isme cleanup code likhte hain, jaise connections close karna.
4. **Garbage Collection**: destroy ke baad servlet object JVM ke garbage collector ke liye mark ho jata hai aur memory free ho jati hai.

Key Terms:

1. **Init Method**: Sirf ek baar call hoti hai jab servlet pehli baar create hoti hai — isme database connections establish karna, configuration load karna, etc. karte hain.
2. **Service Method**: Har request ke liye call hoti hai — yeh request type detect karti hai aur doGet, doPost, doPut, doDelete etc. call karti hai.
3. **doGet Method**: GET requests handle karta hai — URL se data read karta hai, query string mein data aata hai.
4. **doPost Method**: POST requests handle karta hai — request body se data read karta hai, zyada secure aur unlimited data transfer.
5. **Destroy Method**: Sirf ek baar call hoti hai jab servlet unload hoti hai — resources release karte hain, connections close karte hain.
6. **Servlet Container**: Web server ka component hai jo servlet lifecycle manage karta hai, threads create karta hai, aur requests handle karta hai.

Key Points:

1. **Single Instance**: Har servlet ka sirf ek instance create hota hai, jo sabhi clients ke requests handle karta hai.
2. **Thread Per Request**: Har request ke liye container naya thread create karta hai, is liye servlet fast aur efficient hai.
3. **Lifecycle Methods**: Container hi `init()`, `service()`, aur `destroy()` methods call karta hai — hum sirf override karte hain.
4. **GET vs POST**: GET data URL mein bhejta hai (visible, limited), POST data body mein bhejta hai (hidden, unlimited).

SECTION 3 — EXAMPLE

Example:

Socho tum online shopping website bana rahe ho. Jab koi pehli baar website open karta hai, servlet ka `init()` method database connection establish karta hai (ek baar). Phir jab koi product search karta hai, `doGet()` method query parameters read karta hai aur products display karta hai. Jab koi order place karta hai, `doPost()` method cart data body se read karta hai aur order save karta hai. Server band hote waqt `destroy()` method database connection close karta hai. Yahi servlet lifecycle hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.sql.*;
import java.util.*;

// Servlet demonstrating complete lifecycle and GET/POST methods
public class LifecycleDemoServlet extends HttpServlet {

    // Class-level variables for shared resources
    private Connection dbConnection;
    private int requestCount = 0;
    private List<String> logMessages = new ArrayList<>();

    // 1. INIT METHOD - Called once when servlet is first created
    @Override
    public void init() throws ServletException {
        try {
            System.out.println("=== INIT METHOD CALLED ===");
            System.out.println("Servlet instance created at: " + new Date());

            // Simulate database connection
            System.out.println("Establishing database connection...");
            // In real application, use DriverManager.getConnection()
```

```
            dbConnection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb",
                                                        "root", "password");
            System.out.println("Database connected successfully!");

            // Load configuration data
            ServletConfig config = getServletConfig();
            String adminEmail = config.getInitParameter("adminEmail");
            System.out.println("Admin Email from config: " + adminEmail);

            logMessages.add("Servlet initialized at: " + new Date());

        } catch (SQLException e) {
            throw new ServletException("Failed to initialize database connection",
e);
        }
    }

    // 2. SERVICE METHOD - Called for each request (rarely overridden directly)
    @Override
    protected void service(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        requestCount++;
        System.out.println("Request #" + requestCount + " received at: " + new
Date());
        logMessages.add("Request #" + requestCount + " - " + request.getMethod());

        // Call parent service which will route to doGet/doPost
        super.service(request, response);
    }

    // 3. doGET METHOD - Handles GET requests
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        System.out.println("doGET() called for request #" + requestCount);
```

```
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get parameters from URL (query string)
        String name = request.getParameter("name");
        String action = request.getParameter("action");

        out.println("<html><body>");
        out.println("<h1>Servlet Lifecycle Demo - GET Request</h1>");
        out.println("<h3>Request Count: " + requestCount + "</h3>");

        if (action != null && action.equals("showLog")) {
            // Show log messages
            out.println("<h3>Log Messages:</h3>");
            out.println("<ul>");
            for (String msg : logMessages) {
                out.println("<li>" + msg + "</li>");
            }
            out.println("</ul>");
        }

        if (name != null && !name.isEmpty()) {
            out.println("<p>Hello, " + name + "! (GET request with name
parameter)</p>");
        } else {
            // Show a form for GET request
            out.println("<p>No name provided. Try: ?name=YourName</p>");
            out.println("<h3>GET Form:</h3>");
            out.println("<form method='GET' action=''>");
            out.println("Name: <input type='text' name='name'><br>");
            out.println("<input type='submit' value='Send GET Request'>");
            out.println("</form>");
        }

        // Show servlet info
        out.println("<hr>");
        out.println("<p>Servlet Info: " + getServletInfo() + "</p>");
        out.println("<p>Memory Address: " + this.toString() + "</p>");
        out.println("</body></html>");
```

```
    }

    // 4. doPOST METHOD - Handles POST requests
    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        System.out.println("doPOST() called for request #" + requestCount);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get parameters from request body
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String message = request.getParameter("message");

        out.println("<html><body>");
        out.println("<h1>Servlet Lifecycle Demo - POST Request</h1>");
        out.println("<h3>Request Count: " + requestCount + "</h3>");

        if (name != null && email != null) {
            out.println("<h3>Form Data Received:</h3>");
            out.println("<ul>");
            out.println("<li><strong>Name:</strong> " + name + "</li>");
            out.println("<li><strong>Email:</strong> " + email + "</li>");
            if (message != null && !message.isEmpty()) {
                out.println("<li><strong>Message:</strong> " + message + "</li>");
            }
            out.println("</ul>");

            // Simulate saving to database
            try {
                // In real app, use PreparedStatement
                System.out.println("Saving data to database: " + name + ", " +
email);
                logMessages.add("POST data saved - " + name);
            } catch (Exception e) {
                logMessages.add("Error saving POST data: " + e.getMessage());
```

```
            }

            out.println("<p style='color:green;'><strong>Data saved
successfully!</strong></p>");

        } else {
            // Show POST form
            out.println("<h3>POST Form:</h3>");
            out.println("<form method='POST' action=''>");
            out.println("Name: <input type='text' name='name'><br>");
            out.println("Email: <input type='email' name='email'><br>");
            out.println("Message: <textarea name='message'
rows='3'></textarea><br>");
            out.println("<input type='submit' value='Send POST Request'>");
            out.println("</form>");
        }

        out.println("<hr>");
        out.println("<a href='?action=showLog'>View Log Messages</a><br>");
        out.println("<a href=''>Back to Main</a>");
        out.println("</body></html>");
    }

    // 5. DESTROY METHOD - Called once when servlet is unloaded
    @Override
    public void destroy() {
        System.out.println("=== DESTROY METHOD CALLED ===");
        System.out.println("Servlet being destroyed at: " + new Date());
        logMessages.add("Servlet destroyed at: " + new Date());

        // Close database connections
        try {
            if (dbConnection != null && !dbConnection.isClosed()) {
                dbConnection.close();
                System.out.println("Database connection closed successfully!");
            }
        } catch (SQLException e) {
            System.err.println("Error closing database connection: " +
e.getMessage());
```

```
        }

        // Clear log
        logMessages.clear();
        System.out.println("Cleanup completed!");
    }

    // Get servlet info
    @Override
    public String getServletInfo() {
        return "LifecycleDemoServlet version 1.0 - Demonstrates full servlet
lifecycle";
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `init()` method mein database connection establish kiya hai — yeh sirf ek baar call hota hai jab servlet pehli baar load hoti hai. `getServletConfig()` se initialization parameters read kiye hain. Exception handling bhi ki hai agar connection fail ho toh.
- **Step 2:** `service()` method override ki hai — har request par call hoti hai. Isme hum request count increment karte hain aur log add karte hain. Phir `super.service()` call karte hain jo request type detect karke doGet/doPost route karta hai.
- **Step 3:** `doGet()` method mein hum URL parameters read karte hain (`request.getParameter()`) — yeh query string se data leta hai. Agar `action=showLog` hai toh log messages display karte hain. Form bhi display karte hain taake user GET request bhej sake.
- **Step 4:** `doPost()` method mein hum request body se parameters read karte hain — POST data yahan se aata hai. Database save simulate karte hain aur confirmation message show karte hain. Agar data nahi aaya toh POST form display karte hain.
- **Step 5:** `destroy()` method mein database connection close karte hain — yeh tab call hoti hai jab servlet unload ho rahi hai (server shutdown ya servlet reload). Log clear karte hain aur cleanup complete message print karte hain.

PART B — Expected Output:

```
text
// Console output when servlet first loads:
=== INIT METHOD CALLED ===
Servlet instance created at: Wed Jun 28 14:30:00 PKT 2026
```

```
Establishing database connection...
Database connected successfully!
Admin Email from config: admin@example.com

// Console output for each request:
Request #1 received at: Wed Jun 28 14:31:00 PKT 2026
doGET() called for request #1

Request #2 received at: Wed Jun 28 14:32:00 PKT 2026
doPOST() called for request #2
Saving data to database: Ahmed, ahmed@email.com

// Console output when servlet is destroyed:
=== DESTROY METHOD CALLED ===
Servlet being destroyed at: Wed Jun 28 15:00:00 PKT 2026
Database connection closed successfully!
Cleanup completed!
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke init sirf ek baar call hui, har request ke liye service method call hui aur request count increment hua, GET aur POST alag-alag handle hue, aur destroy call hui jab servlet unload hui. Yeh complete lifecycle demonstrate karta hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Forgetting to Call super.service() in Overridden service()

Buggy Code:

```java
public class TestServlet extends HttpServlet {
    @Override
    protected void service(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Custom logic
        System.out.println("Service called");
        // Missing super.service() call
    }
```

```
    public void doGet(HttpServletRequest request, HttpServletResponse response) {
        // This will never be called
    }
}
```

What's Wrong (Roman Urdu): Student ne `service()` method override ki hai lekin `super.service()` call nahi ki, is liye request doGet/doPost tak nahi pahunchayegi. Service method khud request routing handle karti hai, agar hum super call nahi karte toh routing break ho jati hai.

Fixed Code:

```
java
public class TestServlet extends HttpServlet {
    @Override
    protected void service(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Custom logic before processing
        System.out.println("Service called - Request: " + request.getMethod());

        // IMPORTANT: Call super to route to doGet/doPost
        super.service(request, response);

        // Custom logic after processing
        System.out.println("Service completed");
    }

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // This will now be called correctly
    }
}
```

Fix Explained (Roman Urdu): `super.service()` call add ki taake parent class ka routing mechanism kaam kare aur request sahi doGet/doPost method mein pahunchaye.

Mistake 2: Using Wrong Method Signature for doGet/doPost

Buggy Code:

```
java
public class TestServlet extends HttpServlet {
    // Wrong signature - missing HttpServletRequest and HttpServletResponse
    public void doGet(HttpServletRequest request, HttpServletResponse response) {
        // This won't be called by container
    }

    // Wrong signature - missing throws declaration
    public void doPost(HttpServletRequest request, HttpServletResponse response) {
        // IOException may be thrown but not declared
    }
}
```

What's Wrong (Roman Urdu): `doGet` aur `doPost` methods ki signatures exactly match honi chahiye — `throws ServletException, IOException` zaroori hai. Agar throws clause missing hai toh compilation error aayegi kyunke PrintWriter operations IOException throw karte hain.

Fixed Code:

```
java
public class TestServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {  // Correct signature
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>GET Request</h1>");
    }

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {  // Correct signature
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>POST Request</h1>");
    }
```

```
}
```

Fix Explained (Roman Urdu): Sahi method signatures use ki — `throws ServletException, IOException` add kiya taake exceptions properly handle ho sakein.

Mistake 3: Trying to Read POST Data in doGet Method

Buggy Code:

```
java
public class TestServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Trying to get POST data in GET method - won't work
        String email = request.getParameter("email");  // Returns null for POST
        String password = request.getParameter("password");
        // Process login...
    }
}
```

What's Wrong (Roman Urdu): Student doGet mein POST data read kar raha hai, lekin GET requests mein data URL query string mein aata hai, body mein nahi. Agar client POST bhejta hai toh doPost call hogi, doGet nahi. Isliye parameters null aayenge.

Fixed Code:

```
java
public class TestServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // GET: Show login form with action="POST"
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<form method='POST' action='login'>");
        out.println("Email: <input type='email' name='email'><br>");
```

```
        out.println("Password: <input type='password' name='password'><br>");
        out.println("<input type='submit' value='Login'>");
        out.println("</form>");
        out.println("</body></html>");
    }

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // POST: Process login credentials from body
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        // Process login...
    }
}
```

Fix Explained (Roman Urdu): doGet mein login form show kiya, doPost mein credentials process kiye — taake GET aur POST dono apni sahi jagah use ho sakein.

Mistake 4: Not Handling Init Method Exceptions Properly

Buggy Code:

```
java
public class TestServlet extends HttpServlet {
    @Override
    public void init() {
        // No throws clause, but code might throw exception
        Connection conn = DriverManager.getConnection("jdbc:mysql://...");
        // SQLException will cause servlet to fail silently
    }
}
```

What's Wrong (Roman Urdu): init method mein throws ServletException nahi hai, lekin code SQLException throw kar sakta hai. Agar exception aati hai toh servlet initialize nahi hogi aur container ko pata nahi chalega ke kya hua, debugging mushkil ho jayegi.

Fixed Code:

```java
public class TestServlet extends HttpServlet {
    private Connection dbConnection;

    @Override
    public void init() throws ServletException {  // Added throws
        try {
            System.out.println("Initializing servlet...");
            // Proper exception handling
            dbConnection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb",
                                                        "root", "password");
            System.out.println("Database connected!");

        } catch (SQLException e) {
            // Convert to ServletException for container to handle
            throw new ServletException("Failed to initialize database connection",
e);
        }
    }

    @Override
    public void destroy() {
        try {
            if (dbConnection != null) dbConnection.close();
        } catch (SQLException e) {
            System.err.println("Error closing connection: " + e.getMessage());
        }
    }
}
```

Fix Explained (Roman Urdu): Exception handling add ki — SQLException catch karke ServletException throw kiya taake container ko pata chal sake ke initialization fail hui. Destroy mein bhi connection close kiya.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if a servlet's init() method throws a ServletException and why is this important for web application deployment?

Answer: Agar servlet ki init() method ServletException throw karti hai, toh servlet container servlet ko unavailable mark kar dega aur all subsequent requests to this servlet will fail with HTTP 503 or 500 error. Is liye init mein proper exception handling karna bohot zaroori hai — agar database connection fail ho ya configuration missing ho toh hum exception throw kar sakte hain taake container ko pata chal sake aur servlet properly initialize na ho.

Jaise agar database down hai aur init mein connection establish nahi ho raha, toh hum ServletException throw karte hain jisse container log karega aur servlet unavailable ho jayegi.

Q: In a high-traffic web application with 1000 simultaneous users, how many instances of a servlet will be created and how many threads will be created? Explain with reasoning.

Answer: Sirf ek instance of each servlet create hoga, chahe 1000 users simultaneously request karein. Container ek hi servlet instance maintain karta hai aur har request ke liye ek naya thread create karta hai (ya thread pool se reuse karta hai). Isliye 1000 users ke liye 1000 threads honge lekin sirf ek servlet object. Yeh design bohot efficient hai kyunke memory save hoti hai (ek object) aur request processing fast hoti hai (threads lightweight hain).

Isliye servlets mein instance variables use karte waqt careful hona chahiye kyunke multiple threads ek hi object ki variables ko access kar rahe hain — synchronization zaroori hai.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which lifecycle method is called only once when the servlet is first loaded? (init())
2. What happens to the servlet object after destroy() method is called? (It is marked for garbage collection by JVM)
3. What will this code print if a POST request is received?

```
java
```

```
protected void service(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    System.out.println("Method: " + req.getMethod());
    super.service(req, res);
}
```

(Prints "Method: POST" and then calls doPost())

4. What is the difference between init() and service() methods in terms of how many times they are called? (init() called once, service() called for each request)
5. Which method would handle a URL request with query parameters like "?id=5&action=delete"? (doGet() because URL query string is used for GET requests)

SECTION 1 — LESSON HEADING

Lesson 6: Servlet Architecture and Deployment Steps

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:

Yeh bilkul waisi baat hai jaise tum ek naya ghar banate ho. Pehle tum blueprint banate ho (directory structure), phir ghar ki deewarein banate ho (servlet class), phir us mein darwaze aur khirkiyan lagate ho (web.xml mapping), aur akhir mein us mein rehna shuru karte ho (deploy and access). Servlet deployment bhi yahi hai — structure create karo, servlet likho, configure karo, aur server par deploy karo.

PART B — Key Content List:

Steps:

1. **Create Directory Structure**: Standard folders banate hain — WEB-INF, classes, lib — taake container ko files ka pata chal sake.

2. **Create Servlet**: Servlet class likhte hain — teen tareekay hain: Servlet interface implement karna, GenericServlet extend karna, ya HttpServlet extend karna.
3. **Compile Servlet**: Servlet ko compile karte hain — is ke liye servlet-api.jar file classpath mein honi chahiye.
4. **Create Deployment Descriptor**: web.xml file banate hain — is mein servlet ka naam aur URL mapping dete hain.
5. **Start Server and Deploy**: Tomcat server start karte hain aur project deploy karte hain.
6. **Access Servlet**: Browser mein URL daal kar servlet ko access karte hain.

Key Terms:

1. **Directory Structure**: Standard folder hierarchy hai jo Sun Microsystem ne define ki hai — jis mein classes, web.xml, aur libraries ki specific locations hoti hain.
2. **WEB-INF**: Yeh folder hai jo web application ki private files contain karta hai — client directly access nahi kar sakta.
3. **classes Folder**: WEB-INF ke andar yeh folder servlet ke compiled .class files contain karta hai.
4. **web.xml**: Deployment descriptor file hai jo container ko batati hai ke kaun si servlet kaun se URL ko handle karegi.
5. **Servlet Mapping**: web.xml mein servlet-name aur url-pattern ko map karna — yeh batata hai ke client kis URL par servlet access karega.
6. **Tomcat Server**: Apache Tomcat ek popular servlet container hai jo Java web applications run karta hai.
7. **JAVA_HOME**: Environment variable hai jo Tomcat ko JVM ka path batata hai taake server start ho sake.

Types:

1. **Implementing Servlet Interface**: Sab se basic tareeqa — saare lifecycle methods khud implement karne padte hain.

2. **Inheriting GenericServlet**: Protocol-independent servlet — service method sirf override karna padta hai.
3. **Inheriting HttpServlet**: Sab se common tareeqa — HTTP-specific methods (doGet, doPost, etc.) override kar sakte hain.

SECTION 3 — EXAMPLE

Example:

Socho tum school mein annual day function organize kar rahe ho. Tumhe pehle venue set karna hai (directory structure), phir program ka schedule banana hai (servlet class), phir speakers ko timeline assign karna hai (web.xml mapping), aur akhir mein schedule announce karna hai (deploy and access). Jab koi website par click karega toh schedule show ho jayega. Yahi servlet deployment ka process hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
// 1. The Servlet class - DemoWebServlet.java
// This file should be placed in the Java source folder

import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;
import java.util.Date;

// Approach: Extending HttpServlet (most common way)
public class DemoWebServlet extends HttpServlet {

    // Handler for GET requests (default request type)
    @Override
```

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    // Set content type so browser knows it's HTML
    response.setContentType("text/html");

    // Get PrintWriter to write response
    PrintWriter out = response.getWriter();

    // Write HTML response
    out.println("<html>");
    out.println("<head><title>My First Servlet</title></head>");
    out.println("<body>");
    out.println("<h1 style='color:blue;'>Welcome to Servlet World!</h1>");
    out.println("<p>This page is generated by DemoWebServlet.</p>");
    out.println("<p>Current Time: " + new Date() + "</p>");
    out.println("<p>Servlet Info: " + getServletInfo() + "</p>");

    // Add a simple form for POST demo
    out.println("<h3>Test POST Request:</h3>");
    out.println("<form method='POST' action='welcome'>");
    out.println("Your Name: <input type='text' name='username'><br>");
    out.println("<input type='submit' value='Send POST'>");
    out.println("</form>");

    out.println("</body></html>");
    out.close();
}

// Handler for POST requests
@Override
public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    // Get parameter from request body
    String username = request.getParameter("username");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
```

```
        out.println("<html><body>");
        out.println("<h1 style='color:green;'>POST Request Received!</h1>");
        if (username != null && !username.isEmpty()) {
            out.println("<p>Hello, <strong>" + username + "</strong>! Welcome to our
servlet.</p>");
        } else {
            out.println("<p>No name provided in POST request.</p>");
        }
        out.println("<p><a href='welcome'>Back to Main Page</a></p>");
        out.println("</body></html>");
        out.close();
    }

    @Override
    public String getServletInfo() {
        return "DemoWebServlet version 1.0 - Created for CS311";
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `DemoWebServlet` class `HttpServlet` extend kar rahi hai — yeh sab se common tareeqa hai servlet banane ka. `javax.servlet.http.*` imports use kiye hain.
- **Step 2:** `doGet()` method override kiya hai — yeh GET requests handle karta hai (default request type). `response.setContentType("text/html")` batata hai ke response HTML hai.
- **Step 3:** `PrintWriter` object get karte hain `response.getWriter()` se — is se hum HTML write kar sakte hain client ke browser par. Dynamic content generate kiya hai jaise current time.
- **Step 4:** `doPost()` method bhi implement ki hai — POST requests handle karne ke liye. Is mein hum `request.getParameter("username")` se form data read karte hain jo body mein aata hai.

- **Step 5:** HTML form bhi add ki hai jo POST request bhejti hai — is se student GET aur POST dono ka behavior dekh sakta hai.
- **Step 6:** Har method mein `out.close()` call kiya hai — stream close karna memory management ke liye zaroori hai.

Complete Deployment Structure:

```
text
MyWebApp/                          # Project root folder
│
├── WEB-INF/                       # WEB-INF folder (private)
│   │
│   ├── web.xml                    # Deployment descriptor
│   │
│   ├── classes/                   # Compiled classes go here
│   │   └── DemoWebServlet.class
│   │
│   └── lib/                       # JAR files (optional)
│       └── (any external JARs)
│
└── (HTML/JSP files can be here or in subfolders)
```

Web.xml (Deployment Descriptor):

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
         http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
         version="4.0">
```

```
    <!-- Application display name -->
    <display-name>My First Web Application</display-name>

    <!-- Servlet declaration -->
    <servlet>
        <!-- Internal name used within web.xml -->
        <servlet-name>demoServlet</servlet-name>

        <!-- Fully qualified class name of the servlet -->
        <servlet-class>DemoWebServlet</servlet-class>

        <!-- Optional: Load on startup (1 means load when server starts) -->
        <load-on-startup>1</load-on-startup>
    </servlet>

    <!-- Servlet mapping (URL pattern) -->
    <servlet-mapping>
        <!-- Must match the servlet-name above -->
        <servlet-name>demoServlet</servlet-name>

        <!-- URL pattern - client will use /welcome to access this servlet -->
        <url-pattern>/welcome</url-pattern>
    </servlet-mapping>

    <!-- Welcome file list (default page) -->
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

</web-app>
```

How to Compile (Using Command Line):

```
bash
# 1. Set classpath to include servlet-api.jar
# For Windows:
set CLASSPATH=%CLASSPATH%;C:\apache-tomcat-9.0.65\lib\servlet-api.jar
```

```
# For Linux/Mac:
export CLASSPATH=$CLASSPATH:/opt/tomcat/lib/servlet-api.jar

# 2. Compile the servlet
javac -d WEB-INF/classes DemoWebServlet.java

# 3. Create WAR file (optional)
jar -cvf MyWebApp.war *

# 4. Deploy: Copy WAR to Tomcat's webapps folder
# Or copy the whole MyWebApp folder to webapps directory
```

Accessing the Servlet:

```
text
# After deployment, access via browser:
http://localhost:8080/MyWebApp/welcome

# If using different port (Tomcat default is 8080):
http://localhost:8080/MyWebApp/welcome?username=Ali

# The URL pattern /welcome matches the web.xml mapping
```

PART B — Expected Output:

```
text
// Browser output when accessing http://localhost:8080/MyWebApp/welcome:

Welcome to Servlet World!

This page is generated by DemoWebServlet.

Current Time: Wed Jun 28 15:30:45 PKT 2026

Servlet Info: DemoWebServlet version 1.0 - Created for CS311

Test POST Request:
[Your Name: _____] [Send POST]

// After submitting POST form:
```

```
POST Request Received!

Hello, [username]! Welcome to our servlet.

Back to Main Page
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke servlet successfully deploy ho gayi hai — GET request par dynamic HTML page generate hua jis mein current time aur form hai, POST request par username display ho raha hai. Yeh servlet ka real-world behavior hai jo browser mein visible hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Wrong Import Statements

Buggy Code:

```
java
// Wrong imports
import java.servlet.http.*;
import java.servlet.*;

public class MyServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) {
        // Code
    }
}
```

What's Wrong (Roman Urdu): Student ne `java.servlet` import kiya hai, lekin servlet API `javax.servlet` package mein hai. Java aur javax mein farq hai — sahi package `javax` hai.

Fixed Code:

```
java
// Correct imports
import javax.servlet.http.*;
import javax.servlet.*;

public class MyServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("Hello");
    }
}
```

Fix Explained (Roman Urdu): `javax.servlet` import kiya kyunke servlet API `javax` package mein available hai, `java` mein nahi.

Mistake 2: Servlet Class Not in Correct Directory

Buggy Code:

```
text
MyWebApp/
├── WEB-INF/
│   └── DemoWebServlet.class   # Wrong: class file directly in WEB-INF
└── web.xml
```

What's Wrong (Roman Urdu): Student ne servlet class file WEB-INF folder mein directly rakhi hai, lekin container classes folder mein files search karta hai. Agar classes folder mein nahi hai toh servlet load nahi hogi.

Fixed Code:

```
text
```

```
MyWebApp/
├── WEB-INF/
│   ├── classes/
│   │   └── DemoWebServlet.class   # Correct: classes folder mein
│   └── web.xml
└── (other files)
```

Fix Explained (Roman Urdu): Class file ko `WEB-INF/classes/` folder mein rakha taake container usay find kar sake aur servlet successfully load ho jaye.

Mistake 3: Missing or Incorrect web.xml Configuration

Buggy Code:

```xml
<web-app>
    <servlet>
        <servlet-name>demo</servlet-name>
        <servlet-class>DemoWebServlet</servlet-class>
    </servlet>
    <!-- Missing servlet-mapping -->
</web-app>
```

What's Wrong (Roman Urdu): Student ne servlet declare toh kar diya lekin `servlet-mapping` nahi likhi. Agar mapping nahi hai toh container ko nahi pata ke kaun sa URL pattern kaun si servlet ko call karega, is liye servlet access nahi ho payegi.

Fixed Code:

```xml
<web-app>
    <servlet>
        <servlet-name>demo</servlet-name>
        <servlet-class>DemoWebServlet</servlet-class>
    </servlet>
```

```
    <!-- Added servlet-mapping -->
    <servlet-mapping>
        <servlet-name>demo</servlet-name>
        <url-pattern>/welcome</url-pattern>
    </servlet-mapping>
</web-app>
```

Fix Explained (Roman Urdu): `servlet-mapping` add ki jis mein servlet-name ko url-pattern ke saath map kiya, taake client `/welcome` URL par request bhej sakay.

Mistake 4: Forgetting to Set Content Type in Response

Buggy Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    PrintWriter out = res.getWriter();
    out.println("<html><body><h1>Hello</h1></body></html>");
    // Missing: res.setContentType("text/html");
}
```

What's Wrong (Roman Urdu): Student ne content type set nahi kiya, is liye browser HTML ko plain text ki tarah display karega, proper render nahi hoga. Content type batana zaroori hai taake browser samajh sake ke response mein kya type ka data hai.

Fixed Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    // Set content type BEFORE writing
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
```

```
    out.println("<html><body><h1 style='color:red;'>Hello</h1></body></html>");
}
```

Fix Explained (Roman Urdu): `setContentType("text/html")` pehle call kiya taake browser ko pata chal sake ke response HTML hai, is liye woh properly render karega.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if you place the servlet class file in the WEB-INF folder instead of WEB-INF/classes folder, and why does this cause an error?

Answer: Agar servlet class file WEB-INF folder mein direct rakhi hai (WEB-INF/classes mein nahi), toh servlet container usay find nahi kar payega aur "ClassNotFoundException" aayegi. Container specifically `WEB-INF/classes/` folder mein compiled classes search karta hai. Agar class vahan nahi hai toh container assume karega ke servlet deploy hi nahi hui. Is liye package structure bhi maintain karna zaroori hai — agar package hai toh `WEB-INF/classes/com/example/MyServlet.class` ki tarah.

Jaise agar servlet package `com.cs311` mein hai toh path hoga `WEB-INF/classes/com/cs311/DemoServlet.class` aur web.xml mein `com.cs311.DemoServlet` likhna hoga.

Q: In the web.xml file, what is the relationship between `<servlet-name>` in the `<servlet>` element and `<servlet-name>` in the `<servlet-mapping>` element? What happens if they don't match?

Answer: `<servlet>` element mein `<servlet-name>` servlet ki internal identity hai, aur `<servlet-mapping>` mein `<servlet-name>` usi identity ko refer karta hai taake URL pattern ko servlet ke saath map kare. Agar dono match nahi karte toh container ko nahi pata ke kaun si servlet ko kaun se URL pattern par call karna hai, aur servlet HTTP 404 error return karegi.

Jaise agar servlet element mein name "demo1" hai aur mapping mein "demo2" likha hai toh container mapping ko ignore kar dega aur servlet access nahi ho payegi.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which folder must contain the compiled servlet class files in a web application? (WEB-INF/classes/)
2. What is the purpose of `<url-pattern>/welcome</url-pattern>` in web.xml? (Maps the servlet to the URL pattern /welcome so clients can access it)
3. What will happen if you access a servlet without proper web.xml mapping? (HTTP 404 - Servlet not found error)
4. Which file is known as the deployment descriptor in a servlet application? (web.xml)
5. What is the correct import statement for HttpServlet class? (import javax.servlet.http.HttpServlet;)

SECTION 1 — LESSON HEADING

Lesson 7: Request Dispatcher – Forward and Include

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:

Yeh bilkul waisi baat hai jaise tum kisi office mein receptionist ke paas jaate ho aur woh tumhe sahi department mein bhejti hai (forward) ya tumhara kaam kisi aur department se karwati hai aur tumhe wapas response deti hai (include). RequestDispatcher servlet collaboration ka yahi kaam karta hai — ek servlet doosri servlet ko request forward kar sakta hai ya uska response include kar sakta hai.

PART B — Key Content List:

Key Terms:

1. **RequestDispatcher**: Interface hai jo request ko doosri resource (servlet, JSP, HTML) par forward ya include karne ki facility provide karta hai.
2. **forward()**: Request ko ek servlet se doosri resource par bhej deta hai — client ko sirf second resource ka response dikhta hai, first servlet ka response nahi.
3. **include()**: Doosri resource ke content ko current servlet ke response mein include kar deta hai — client ko dono resources ka combined response dikhta hai.
4. **getRequestDispatcher()**: ServletRequest interface ka method hai jo RequestDispatcher object return karta hai.
5. **Resource**: Koi bhi web component (servlet, JSP, HTML file) jo server par available ho.
6. **Servlet Collaboration**: Multiple servlets aapas mein milkar ek request process karte hain — RequestDispatcher iska ek tareeqa hai.

Key Points:

1. **Forward Behavior**: Request aur response objects transfer ho jatay hain — original servlet ka output discard ho jata hai.
2. **Include Behavior**: Response combined hota hai — dono servlets ka output client tak pahunchta hai.
3. **URL Pattern**: getRequestDispatcher() mein resource ka URL pattern path dete hain — forward/include karne ke liye.

4. **Server-Side**: RequestDispatcher server-side redirection hai — client ko pata nahi chalta ke request forward hui.

SECTION 3 — EXAMPLE

Example:

Socho tum online shopping website par login kar rahe ho. Tum username/password daalte ho aur submit karte ho. Login servlet check karti hai ke credentials sahi hain ya nahi. Agar sahi hain toh RequestDispatcher use karke request Welcome servlet par forward kar deti hai jo user ko welcome page dikhati hai. Agar galat hain toh error message dikhati hai. Yahi RequestDispatcher ka practical use hai — ek servlet doosri servlet ko request forward karta hai

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
// ============================================
// 1. LoginServlet.java - Processes login request
// ============================================
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;

public class LoginServlet extends HttpServlet {

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // Get parameters from form
```

```
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Hardcoded validation (in real app, check database)
        if (username != null && password != null &&
            username.equals("admin") && password.equals("12345")) {

            // SUCCESS: Forward to WelcomeServlet
            // Create RequestDispatcher object
            RequestDispatcher rd = request.getRequestDispatcher("welcome");

            // Forward the request (response of this servlet is discarded)
            rd.forward(request, response);

        } else {
            // FAILURE: Show error message
            out.println("<html><body>");
            out.println("<h2 style='color:red;'>Login Failed!</h2>");
            out.println("<p>Invalid username or password.</p>");
            out.println("<a href='index.html'>Try Again</a>");
            out.println("</body></html>");
        }
        out.close();
    }

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Redirect to login page for GET requests
        response.sendRedirect("index.html");
    }
}

// ============================================
// 2. WelcomeServlet.java - Displays welcome message
// ============================================
```

```java
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;

public class WelcomeServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get username from request (passed from LoginServlet)
        String username = request.getParameter("username");

        out.println("<html><body>");
        out.println("<h1 style='color:green;'>Welcome to Dashboard!</h1>");
        out.println("<h2>Hello, " + username + "!</h2>");
        out.println("<p>You have successfully logged in.</p>");
        out.println("<p>This is the WelcomeServlet response.</p>");
        out.println("</body></html>");
        out.close();
    }
}

// ============================================
// 3. IncludeDemoServlet.java - Demonstrates include()
// ============================================
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;

public class IncludeDemoServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

```
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // First part of response
        out.println("<html><body>");
        out.println("<h1>RequestDispatcher Include Demo</h1>");
        out.println("<p>This is from IncludeDemoServlet (Before include)</p>");
        out.println("<hr>");

        // Include content from another resource (e.g., FooterServlet)
        RequestDispatcher rd = request.getRequestDispatcher("footer");
        rd.include(request, response);  // Content of footer will be included here

        // Third part of response
        out.println("<hr>");
        out.println("<p>This is from IncludeDemoServlet (After include)</p>");
        out.println("</body></html>");
        out.close();
    }
}

// ============================================
// 4. FooterServlet.java - Content to be included
// ============================================
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;
import java.util.Date;

public class FooterServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        PrintWriter out = response.getWriter();

        // This content will be included in the calling servlet's response
        out.println("<div style='background-color:#f0f0f0; padding:10px;'>");
```

```
        out.println("<p><strong>Footer Content:</strong> This is from
FooterServlet</p>");
        out.println("<p>Current Time: " + new Date() + "</p>");
        out.println("<p>&copy; 2026 My Web Application</p>");
        out.println("</div>");
    }
}
```

web.xml (Deployment Descriptor):

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0">

    <!-- Login Servlet Configuration -->
    <servlet>
        <servlet-name>loginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>loginServlet</servlet-name>
        <url-pattern>/login</url-pattern>
    </servlet-mapping>

    <!-- Welcome Servlet Configuration -->
    <servlet>
        <servlet-name>welcomeServlet</servlet-name>
        <servlet-class>WelcomeServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>welcomeServlet</servlet-name>
        <url-pattern>/welcome</url-pattern>
    </servlet-mapping>

    <!-- Include Demo Servlet -->
    <servlet>
        <servlet-name>includeDemo</servlet-name>
        <servlet-class>IncludeDemoServlet</servlet-class>
    </servlet>
```

```
    <servlet-mapping>
        <servlet-name>includeDemo</servlet-name>
        <url-pattern>/includedemo</url-pattern>
    </servlet-mapping>

    <!-- Footer Servlet -->
    <servlet>
        <servlet-name>footerServlet</servlet-name>
        <servlet-class>FooterServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>footerServlet</servlet-name>
        <url-pattern>/footer</url-pattern>
    </servlet-mapping>

</web-app>
```

index.html (Login Page):

```
html
<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
    <style>
        body { font-family: Arial; margin: 50px; }
        .login-box { width: 300px; padding: 20px; border: 1px solid #ccc; }
        input { width: 100%; padding: 8px; margin: 5px 0; }
        .btn { background: blue; color: white; border: none; padding: 10px; }
    </style>
</head>
<body>
    <div class="login-box">
        <h2>Login Form</h2>
        <form method="POST" action="login">
            <label>Username:</label>
            <input type="text" name="username" required><br>
            <label>Password:</label>
            <input type="password" name="password" required><br>
```

```
            <input type="submit" value="Login" class="btn">
        </form>
        <p style="font-size:12px; color:gray;">Try: admin / 12345</p>
    </div>
</body>
</html>
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `LoginServlet` mein `doPost()` method POST request handle karta hai — username aur password parameters read karta hai. Hardcoded validation check karta hai — admin / 12345.
- **Step 2:** Agar login successful hai toh `RequestDispatcher` object create karta hai `request.getRequestDispatcher("welcome")` se — yahan "welcome" URL pattern hai WelcomeServlet ka.
- **Step 3:** `rd.forward(request, response)` call karta hai — yeh request aur response objects WelcomeServlet par forward kar deta hai. LoginServlet ka koi bhi output discard ho jata hai.
- **Step 4:** `WelcomeServlet` mein `doGet()` method request handle karta hai — username parameter read karta hai (jo LoginServlet se aaya) aur welcome page display karta hai.
- **Step 5:** `IncludeDemoServlet` mein `rd.include(request, response)` call karta hai — yeh FooterServlet ke content ko include karta hai. IncludeDemoServlet ka pehla aur aakhri output bhi client tak pahunchta hai.
- **Step 6:** `FooterServlet` sirf footer HTML content generate karta hai — yeh include hone par IncludeDemoServlet ke response mein merge ho jata hai.

PART B — Expected Output:

```
text
// Case 1: Successful Login (Forward)
// Browser URL: http://localhost:8080/demo/login
// After submitting form with admin/12345:

Welcome to Dashboard!
```

```
Hello, admin!
You have successfully logged in.
This is the WelcomeServlet response.

// Case 2: Failed Login
// Browser URL: http://localhost:8080/demo/login

Login Failed!
Invalid username or password.
Try Again

// Case 3: Include Demo
// Browser URL: http://localhost:8080/demo/includedemo

RequestDispatcher Include Demo
This is from IncludeDemoServlet (Before include)

-------------------
Footer Content: This is from FooterServlet
Current Time: Wed Jun 28 16:00:00 PKT 2026
© 2026 My Web Application
-------------------

This is from IncludeDemoServlet (After include)
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke forward mein LoginServlet ka output nahi aaya, sirf WelcomeServlet ka output aaya — is liye client ko login page ka response nahi dikha. Include mein dono servlets ka combined output aaya — IncludeDemoServlet ka pehla aur aakhri part aur FooterServlet ka beech wala part. Yeh forward aur include ka practical difference hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Calling forward() After Writing Output

Buggy Code:

```
java
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    PrintWriter out = res.getWriter();
    out.println("Processing...");  // Written before forward
    RequestDispatcher rd = req.getRequestDispatcher("welcome");
    rd.forward(req, res);  // This will throw IllegalStateException
}
```

What's Wrong (Roman Urdu): Student ne forward() call karne se pehle response mein output write kar diya hai. forward() ke baad response already committed ho chuka hai, is liye IllegalStateException aayegi. Forward se pehle koi output nahi write karna chahiye.

Fixed Code:

```
java
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    // Don't write any output before forward
    RequestDispatcher rd = req.getRequestDispatcher("welcome");
    rd.forward(req, res);  // This works fine
    // Code after forward is not executed
}
```

Fix Explained (Roman Urdu): Output write karna hataya forward se pehle, taake IllegalStateException na aaye. Forward ke baad ka code execute nahi hota.

Mistake 2: Using Wrong Path in getRequestDispatcher()

Buggy Code:

```
java
// File: LoginServlet.java
```

```
// Incorrect path - using full URL instead of resource path
RequestDispatcher rd =
request.getRequestDispatcher("http://localhost:8080/demo/welcome");
rd.forward(request, response);
```

What's Wrong (Roman Urdu): Student ne getRequestDispatcher() mein full URL diya hai, lekin yeh method server-side resource path expect karta hai, absolute URL nahi. Is se error aayegi kyunke container local resource find nahi kar payega.

Fixed Code:

```
java
// Correct - relative path based on web.xml url-pattern
RequestDispatcher rd = request.getRequestDispatcher("/welcome");
// OR
RequestDispatcher rd = request.getRequestDispatcher("welcome");
// Both work if the path is correct relative to the current context
```

Fix Explained (Roman Urdu): Absolute URL ki jagah relative resource path diya jo web.xml mein defined hai. Container local resource search karta hai.

Mistake 3: Forgetting to Return After Forward

Buggy Code:

```
java
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    if (validUser) {
        RequestDispatcher rd = req.getRequestDispatcher("welcome");
        rd.forward(req, res);
    }
    // Code continues even after forward!
    res.getWriter().println("This will still execute");
}
```

What's Wrong (Roman Urdu): Student ne forward ke baad code continue kar diya hai. Forward ke baad bhi method ka baqi code execute hota hai, jo response mein extra output likh sakta hai aur IllegalStateException ka cause ban sakta hai.

Fixed Code:

```
java
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    if (validUser) {
        RequestDispatcher rd = req.getRequestDispatcher("welcome");
        rd.forward(req, res);
        return;  // IMPORTANT: Stop execution after forward
    }
    // This code runs only if forward didn't happen
    res.getWriter().println("Invalid user");
}
```

Fix Explained (Roman Urdu): Forward ke baad `return` statement add kiya taake method execution stop ho jaye aur extra output na likhe.

Mistake 4: Confusing forward() with sendRedirect()

Buggy Code:

```
java
// Trying to use forward for cross-context navigation
RequestDispatcher rd = request.getRequestDispatcher("http://google.com");
rd.forward(request, response);  // Won't work
```

What's Wrong (Roman Urdu): Student forward() use kar raha hai doosre application ya external URL par, lekin forward() sirf same web application ke andar kaam karta hai. Cross-context ya external URL ke liye sendRedirect() use karna chahiye.

Fixed Code:

```java
// For same application internal navigation:
RequestDispatcher rd = request.getRequestDispatcher("/welcome");
rd.forward(request, response);

// For external or cross-context navigation:
response.sendRedirect("http://google.com");
// OR
response.sendRedirect("/otherApp/someServlet");
```

Fix Explained (Roman Urdu): Internal navigation ke liye forward, external/cross-context navigation ke liye sendRedirect use kiya. sendRedirect browser ko naya request bhejne ke liye kehta hai.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if the following code is executed and why?

```java
public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    PrintWriter out = res.getWriter();
    out.println("Header from LoginServlet");

    RequestDispatcher rd = req.getRequestDispatcher("welcome");
    rd.forward(req, res);

    out.println("Footer from LoginServlet");
}
```

Answer: Is code mein IllegalStateException aayegi kyunke forward call karne se pehle response mein output write kar diya gaya hai. Agar response already committed nahi hai toh exception aati hai. Agar response committed hai toh aur bhi badi problem hogi. Is

liye forward se pehle kabhi bhi response output nahi likhna chahiye. Agar hum forward karna chahte hain toh pehle saara output hatana hoga.

Q: What is the difference between forward() and include() methods of RequestDispatcher in terms of how they handle the response and when would you use each?

Answer: forward() method response ko discard kar deta hai aur request ko completely new servlet par transfer kar deta hai — client sirf target servlet ka response dekhta hai. include() method target servlet ke content ko current response mein add kar deta hai — client dono servlets ka combined response dekhta hai.

forward use karte hain jab humein request complete transfer karni ho doosri servlet par, jaise login validation ke baad welcome page par bhejna. include use karte hain jab humein common content (jaise header, footer, navigation) multiple pages mein reuse karna ho — ek hi footer servlet sab jagah include kar sakte hain.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method of RequestDispatcher forwards the request to another resource and discards the current servlet's response? (forward())
2. What happens if you call forward() after writing output to the response? (IllegalStateException is thrown)
3. What will this code output if the password is correct?

```
java
if(isValid) {
    RequestDispatcher rd = req.getRequestDispatcher("welcome");
    rd.forward(req, res);
    res.getWriter().println("After forward");
```

```
}
```

(Prints the welcome page output, "After forward" is not printed because forward transfers control)

4. Which method would you use to include a common footer on multiple pages? (include())
5. What is the difference between forward() and sendRedirect()? (forward is server-side and doesn't change URL, sendRedirect is client-side and changes URL)

SECTION 1 — LESSON HEADING

Lesson 8: ServletContext Interface and Configuration

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tumhari university ka ek common notice board hota hai jahan administration sabhi departments ke liye common announcements lagati hai — jaise exam schedule, holiday list, etc. ServletContext bhi yahi hai — web application ka common area jahan se sabhi servlets shared information access kar sakte hain, jaise database URL, admin email, etc.

PART B — Key Content List:

Key Terms:

1. **ServletContext**: Web container ka object hai jo web application deploy hotay waqt create hota hai — ek web application ke liye sirf ek ServletContext object hota hai.

2. **Context Parameter**: web.xml mein `<context-param>` element ke zariye define kiye gaye parameters jo sabhi servlets ke liye available hotay hain.
3. **getServletContext()**: HttpServlet class ka method hai jo ServletContext object return karta hai.
4. **getInitParameter()**: ServletContext ka method hai jo context parameter ki value return karta hai.
5. **setAttribute()**: ServletContext mein attribute set karne ke liye — isay sabhi servlets access kar sakte hain.
6. **getAttribute()**: ServletContext se attribute ki value retrieve karne ke liye.
7. **removeAttribute()**: ServletContext se attribute delete karne ke liye.

Key Points:

1. **One Per Application**: Har web application ke liye sirf ek ServletContext object exist karta hai — sabhi servlets isay share karte hain.
2. **Easy Maintenance**: Agar koi shared information change hoti hai toh sirf web.xml mein change karna padta hai, servlet code mein nahi — maintenance easy hota hai.
3. **Inter-Application Communication**: ServletContext different servlets ke darmiyaan communication provide karta hai.
4. **Interface Between Container and Servlet**: ServletContext container aur servlet ke darmiyaan bridge ka kaam karta hai.

SECTION 3 — EXAMPLE

Example:

Socho tum ek online store run kar rahe ho. Tumhe website ka name, admin email, aur database connection string sabhi servlets mein use karni hai. Tum yeh information har servlet mein hardcode nahi karte balkay web.xml mein context-param define karte ho. Har servlet ServletContext se yeh values read karti hai. Agar website ka name change ho

toh sirf web.xml change karna hai, saari servlets automatically updated ho jati hain. Yahi ServletContext ka faida hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
// ============================================
// 1. Login.java - Complete Login Servlet with RequestDispatcher
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Login extends HttpServlet {

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get parameters from HTML form
        String name = request.getParameter("userName");
        String password = request.getParameter("userPass");

        // Check if password matches
        if (password != null && password.equals("servlet")) {
            // SUCCESS: Forward to WelcomeServlet
            RequestDispatcher rd = request.getRequestDispatcher("servlet2");
            rd.forward(request, response);
```

```
        } else {
            // FAILURE: Show error and include login form again
            out.print("Sorry UserName or Password Error!");
            RequestDispatcher rd = request.getRequestDispatcher("/index.html");
            rd.include(request, response);
        }
        out.close();
    }
}

// ============================================
// 2. WelcomeServlet.java - Welcome page after login
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class WelcomeServlet extends HttpServlet {

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get username from request (passed from Login servlet)
        String name = request.getParameter("userName");
        out.print("Welcome " + name);
        out.close();
    }
}

// ============================================
// 3. ServletContextDemo.java - Demonstrating ServletContext usage
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
```

```java
public class ServletContextDemo extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get ServletContext object
        ServletContext context = getServletContext();

        out.println("<html><body>");
        out.println("<h1>ServletContext Demo</h1>");

        // 1. Reading context parameters from web.xml
        String appName = context.getInitParameter("applicationName");
        String adminEmail = context.getInitParameter("adminEmail");
        String dbUrl = context.getInitParameter("databaseUrl");

        out.println("<h3>Context Parameters (from web.xml):</h3>");
        out.println("<ul>");
        out.println("<li><strong>Application Name:</strong> " + appName + "</li>");
        out.println("<li><strong>Admin Email:</strong> " + adminEmail + "</li>");
        out.println("<li><strong>Database URL:</strong> " + dbUrl + "</li>");
        out.println("</ul>");

        // 2. Setting and getting attributes
        // Check if counter attribute exists
        Integer counter = (Integer) context.getAttribute("visitCounter");
        if (counter == null) {
            counter = 1;
        } else {
            counter = counter + 1;
        }
        // Store updated counter in context
        context.setAttribute("visitCounter", counter);
```

```
        out.println("<h3>Shared Attributes:</h3>");
        out.println("<p>This page has been visited <strong>" + counter + "</strong>
times.</p>");

        // 3. Getting server information
        out.println("<h3>Server Information:</h3>");
        out.println("<ul>");
        out.println("<li><strong>Server Info:</strong> " + context.getServerInfo() +
"</li>");
        out.println("<li><strong>Servlet Context Path:</strong> " +
context.getContextPath() + "</li>");
        out.println("</ul>");

        // 4. Getting resource paths
        String realPath = context.getRealPath("/WEB-INF/web.xml");
        out.println("<p><strong>web.xml real path:</strong> " + realPath + "</p>");

        out.println("</body></html>");
        out.close();
    }
}

// ============================================
// 4. AnotherServlet.java - Showing attribute sharing
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class AnotherServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        ServletContext context = getServletContext();
```

```
        // Read the shared counter attribute
        Integer counter = (Integer) context.getAttribute("visitCounter");
        String appName = context.getInitParameter("applicationName");

        out.println("<html><body>");
        out.println("<h1>Another Servlet - Shared Data Access</h1>");
        out.println("<p><strong>Application:</strong> " + appName + "</p>");
        out.println("<p><strong>Shared Visit Counter:</strong> " + counter + "</p>");
        out.println("<p><a href='contextdemo'>Go to ContextDemo</a></p>");
        out.println("</body></html>");
        out.close();
    }
}
```

web.xml with Context Parameters:

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0">

    <!-- Context Parameters - Available to ALL servlets -->
    <context-param>
        <param-name>applicationName</param-name>
        <param-value>CS311 Web Services Demo Application</param-value>
    </context-param>

    <context-param>
        <param-name>adminEmail</param-name>
        <param-value>admin@cs311.edu.pk</param-value>
    </context-param>

    <context-param>
        <param-name>databaseUrl</param-name>
        <param-value>jdbc:mysql://localhost:3306/cs311db</param-value>
    </context-param>

    <!-- Login Servlet -->
    <servlet>
```

```
    <servlet-name>Login</servlet-name>
    <servlet-class>Login</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>Login</servlet-name>
    <url-pattern>/servlet1</url-pattern>
</servlet-mapping>

<!-- Welcome Servlet -->
<servlet>
    <servlet-name>WelcomeServlet</servlet-name>
    <servlet-class>WelcomeServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>WelcomeServlet</servlet-name>
    <url-pattern>/servlet2</url-pattern>
</servlet-mapping>

<!-- Context Demo Servlet -->
<servlet>
    <servlet-name>ContextDemo</servlet-name>
    <servlet-class>ServletContextDemo</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ContextDemo</servlet-name>
    <url-pattern>/contextdemo</url-pattern>
</servlet-mapping>

<!-- Another Servlet -->
<servlet>
    <servlet-name>AnotherServlet</servlet-name>
    <servlet-class>AnotherServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>AnotherServlet</servlet-name>
    <url-pattern>/another</url-pattern>
</servlet-mapping>

<!-- Welcome File -->
```

```
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>

</web-app>
```

index.html (Login Form):

```
html
<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h2>Login Form</h2>
    <form action="servlet1" method="post">
        Name: <input type="text" name="userName"/><br/>
        Password: <input type="password" name="userPass"/><br/>
        <input type="submit" value="login"/>
    </form>
    <p style="color:gray;font-size:12px;">Try password: servlet</p>
</body>
</html>
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `Login` servlet HTML form se username aur password read karta hai. Agar password "servlet" ke barabar hai toh forward `servlet2` (WelcomeServlet) par karta hai — client welcome page dekhta hai.
- **Step 2:** Agar password galat hai toh error message print karta hai aur `index.html` ko include karta hai — client error message ke saath login form wapas dekhta hai.
- **Step 3:** `ServletContextDemo` servlet mein `getServletContext()` method se ServletContext object obtain karte hain — yeh shared resources access karne ke liye zaroori hai.

- **Step 4:** Context parameters `getInitParameter()` se read karte hain — yeh values web.xml ke `<context-param>` se aati hain aur sabhi servlets ke liye available hain.
- **Step 5:** Visit counter attribute use kar rahe hain — `getAttribute()` se current value read karte hain, increment karte hain, aur `setAttribute()` se wapas store karte hain. Yehi attribute `AnotherServlet` bhi access kar sakta hai.
- **Step 6:** `getServerInfo()`, `getContextPath()`, aur `getRealPath()` methods server aur path information provide karte hain.

PART B — Expected Output:

```
text
// Case 1: Successful Login (password = "servlet")
// Browser URL: http://localhost:8080/demo/servlet1

Welcome admin

// Case 2: Failed Login (wrong password)
// Browser URL: http://localhost:8080/demo/servlet1

Sorry UserName or Password Error!
Login Form
Name: [___________]
Password: [___________]
[login]

// Case 3: Context Demo - First visit
// Browser URL: http://localhost:8080/demo/contextdemo

ServletContext Demo

Context Parameters (from web.xml):
- Application Name: CS311 Web Services Demo Application
- Admin Email: admin@cs311.edu.pk
- Database URL: jdbc:mysql://localhost:3306/cs311db

Shared Attributes:
This page has been visited 1 times.
```

```
Server Information:
- Server Info: Apache Tomcat/9.0.65
- Servlet Context Path: /demo

web.xml real path: C:\apache-tomcat\webapps\demo\WEB-INF\web.xml

// Case 4: Context Demo - Second visit (after refresh)

This page has been visited 2 times.   // Counter has increased

// Case 5: Another Servlet - Reading shared data
// Browser URL: http://localhost:8080/demo/another

Another Servlet - Shared Data Access
Application: CS311 Web Services Demo Application
Shared Visit Counter: 2
Go to ContextDemo
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke ServletContext ek application-level storage provide karta hai — context parameters web.xml se read hue, visit counter multiple visits mein persist hua (kyunke attribute context mein store hai), aur AnotherServlet ne same counter access kiya. Yeh sabhi servlets ke liye shared data ka concept hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Using getInitParameter() on ServletConfig Instead of ServletContext

Buggy Code:

```
java
public class MyServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
```

```
        // Wrong: Using ServletConfig for context parameter
        String appName = getServletConfig().getInitParameter("applicationName");
        // This returns null because context-param is not in servlet config
    }
}
```

What's Wrong (Roman Urdu): Student ne context parameter read karne ke liye `ServletConfig.getInitParameter()` use kiya hai, lekin context parameters `<context-param>` ServletConfig mein available nahi hain, ServletContext mein hain. Is liye null return ho ga.

Fixed Code:

```
java
public class MyServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        // Correct: Using ServletContext for context-param
        ServletContext context = getServletContext();
        String appName = context.getInitParameter("applicationName");
        // Now appName will have the correct value from web.xml
    }
}
```

Fix Explained (Roman Urdu): ServletConfig ki jagah ServletContext use kiya kyunke context-param application-level hain, servlet-level nahi.

Mistake 2: Forgetting to Cast Attribute Values When Retrieving

Buggy Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    ServletContext context = getServletContext();
```

```
    // Wrong: No casting
    Integer counter = context.getAttribute("visitCounter");
    // Error: Incompatible types - Object cannot be converted to Integer
}
```

What's Wrong (Roman Urdu): Student ne `getAttribute()` ko direct Integer variable mein assign kar diya hai, lekin yeh method Object return karta hai. Casting kiye baghair compilation error aayegi.

Fixed Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    ServletContext context = getServletContext();
    // Correct: Cast to appropriate type
    Integer counter = (Integer) context.getAttribute("visitCounter");
    // Also check for null
    if (counter == null) {
        counter = 0;
    }
}
```

Fix Explained (Roman Urdu): `getAttribute()` Object return karta hai, is liye appropriate type mein cast karna zaroori hai aur null check bhi karna chahiye.

Mistake 3: Using ServletContext.setAttribute() with Non-Serializable Objects

Buggy Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    ServletContext context = getServletContext();
    // Non-serializable custom object
```

```
    MyCustomObject obj = new MyCustomObject();
    context.setAttribute("myData", obj);  // May cause issues in distributed
environment
}
```

What's Wrong (Roman Urdu): Student ne ServletContext mein non-serializable object store kiya hai. Agar web application distributed environment (cluster) mein deploy hai toh objects serializable hone chahiye taake server nodes ke darmiyaan share ho sakein.

Fixed Code:

```
java
// Make the class implement Serializable
import java.io.Serializable;

class MyCustomObject implements Serializable {
    private String name;
    private int value;
    // getters and setters
}

public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    ServletContext context = getServletContext();
    MyCustomObject obj = new MyCustomObject();
    context.setAttribute("myData", obj);  // Now safe for distributed env
}
```

Fix Explained (Roman Urdu): Custom class ko `Serializable` implement karwaya taake object cluster environment mein safely share ho sake.

Mistake 4: Modifying Context Attributes Without Synchronization

Buggy Code:

```java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    ServletContext context = getServletContext();
    // No synchronization - multiple threads can corrupt data
    Integer count = (Integer) context.getAttribute("counter");
    if (count == null) count = 0;
    count = count + 1;
    context.setAttribute("counter", count);  // Thread unsafe!
}
```

What's Wrong (Roman Urdu): Student ne counter increment kiya hai lekin synchronization nahi use ki. Multiple threads simultaneously is code ko execute kar sakti hain, is liye counter ki value corrupt ho sakti hai (race condition).

Fixed Code:

```java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    ServletContext context = getServletContext();

    // Synchronize on the context object
    synchronized (context) {
        Integer count = (Integer) context.getAttribute("counter");
        if (count == null) count = 0;
        count = count + 1;
        context.setAttribute("counter", count);
    }  // Only one thread at a time can execute this block
}
```

Fix Explained (Roman Urdu): `synchronized` block use kiya taake ek waqt mein sirf ek thread counter modify kar sake aur data consistency maintain rahe.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if two different servlets try to access the same ServletContext attribute simultaneously, and how would you make it thread-safe?

Answer: Agar do servlets (ya same servlet ke multiple threads) ek hi time mein ServletContext attribute ko access karein, toh race condition ho sakti hai — data corrupt ho sakta hai. Example: Agar counter attribute increment kar rahe hain aur do threads simultaneously get aur set karein toh counter ki value galat ho sakti hai (lost update problem).

Is problem ko solve karne ke liye hum `synchronized` block use karte hain ServletContext object par:

```
java
synchronized (getServletContext()) {
    Integer count = (Integer) getServletContext().getAttribute("counter");
    // Safe operations
}
```

Is tarah ek waqt mein sirf ek thread context attribute modify kar sakta hai.

Q: What is the difference between ServletContext and ServletConfig in terms of scope and usage? When would you use each?

Answer: ServletContext application-level scope hai — ek web application mein sirf ek ServletContext object hota hai, jo sabhi servlets share karte hain. ServletConfig servlet-level scope hai — har servlet ka apna ServletConfig object hota hai, jismein usi servlet ke initialization parameters hote hain.

ServletContext use karte hain jab data sabhi servlets ke liye common ho, jaise database URL, application name, admin email, etc. ServletConfig use karte hain jab data sirf ek specific servlet ke liye ho, jaise us servlet ka file path, timeout value, etc.

ServletConfig web.xml mein `<servlet>` ke andar `<init-param>` se define hota hai, jabke ServletContext `<context-param>` se define hota hai.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which object is created once per web application and shared among all servlets? (ServletContext)
2. What will this code return if "dbUrl" is defined as context-param?

```java
String url = getServletContext().getInitParameter("dbUrl");
```

(The value of dbUrl from web.xml context-param)

3. What is the difference between ServletContext and ServletConfig? (ServletContext is application-wide, ServletConfig is per-servlet)
4. What will happen in this code?

```java
Integer count = (Integer) getServletContext().getAttribute("count");
count = count + 1;
getServletContext().setAttribute("count", count);
```

(If multiple threads access, race condition may cause incorrect count values)

SECTION 1 — LESSON HEADING

Lesson 9: Session Tracking and Cookies in Servlets

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum kisi store mein jaate ho aur store wala tumhe ek membership card deta hai. Har baar jab tum store mein aate ho, tum woh card dikhate ho aur store wala pehchan leta hai ke tum kaun ho. Cookies bhi yahi karte hain — browser mein ek small piece of information store hoti hai, aur har request ke saath server ko bheji jati hai taake server user ko recognize kar sake.

PART B — Key Content List:

Key Terms:

1. **Session Tracking**: User ki state (data) maintain karne ka tareeqa hai kyunke HTTP stateless protocol hai — har request nayi maani jaati hai.
2. **Cookie**: Chhoti si information hai jo browser ke cache mein store hoti hai aur multiple requests ke darmiyaan persist karti hai.
3. **Non-persistent Cookie**: Sirf ek session ke liye valid hoti hai — browser band hotay hi delete ho jati hai.
4. **Persistent Cookie**: Multiple sessions ke liye valid hoti hai — browser band honay par bhi delete nahi hoti, logout ya signout par delete hoti hai.
5. **Session Tracking Techniques**: Chaar techniques hain — Cookies, Hidden Form Field, URL Rewriting, aur HttpSession.
6. **addCookie()**: HttpServletResponse ka method hai jo cookie ko response mein add karta hai.
7. **getCookies()**: HttpServletRequest ka method hai jo browser se saari cookies return karta hai.

Session Tracking Techniques:

1. **Cookies**: Client side par data store hota hai — simplest technique hai.
2. **Hidden Form Field**: HTML form mein hidden field use karte hain — server side par maintain hota hai.

3. **URL Rewriting**: URL mein data append karte hain — cookie disable honay par kaam karta hai.
4. **HttpSession**: Server side par session object maintain karta hai — sab se advanced technique hai.

Superstructure:

1. **Cookie Class**: javax.servlet.http.Cookie package mein hai — cookie create aur manage karne ke liye.
2. **setMaxAge()**: Cookie ki life time set karta hai seconds mein — 0 means delete, positive means persistent.
3. **getName() / getValue()**: Cookie ka name aur value retrieve karne ke liye.
4. **setName() / setValue()**: Cookie ka name aur value change karne ke liye.

SECTION 3 — EXAMPLE

Example:

Socho tum online shopping website par login karte ho aur apni cart mein products add karte ho. Jab tum doosri page par jate ho toh website tumhe pehchan leti hai aur tumhari cart waisi hi rehti hai — yeh cookies ki wajah se hota hai. Website tumhare browser mein ek cookie store karti hai jis mein tumhari session ID hoti hai, aur har request ke saath woh cookie server par bheji jati hai. Agar tum logout karte ho toh cookie delete ho jati hai. Yahi session tracking hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```java
// ============================================
// 1. CookieLoginServlet.java - Creates cookie on login
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieLoginServlet extends HttpServlet {

    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get username from form
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // Simple validation
        if (username != null && !username.isEmpty() &&
            password != null && password.equals("123")) {

            // Create a cookie with username
            Cookie userCookie = new Cookie("userName", username);
            // Set cookie to live for 30 minutes (1800 seconds)
            userCookie.setMaxAge(1800);
            // Add cookie to response
            response.addCookie(userCookie);

            // Create another cookie for login timestamp
            Cookie timeCookie = new Cookie("loginTime",
String.valueOf(System.currentTimeMillis()));
            timeCookie.setMaxAge(1800);
            response.addCookie(timeCookie);

            // Forward to welcome page
```

```
            out.println("<html><body>");
            out.println("<h2 style='color:green;'>Login Successful!</h2>");
            out.println("<p>Cookie has been created and stored in your
browser.</p>");
            out.println("<a href='welcomeCookie'>Go to Welcome Page</a>");
            out.println("</body></html>");

        } else {
            out.println("<html><body>");
            out.println("<h2 style='color:red;'>Login Failed!</h2>");
            out.println("<p>Invalid username or password (try password: 123)</p>");
            out.println("<a href='index.html'>Try Again</a>");
            out.println("</body></html>");
        }
        out.close();
    }
}

// ============================================
// 2. WelcomeCookieServlet.java - Reads cookies
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.text.SimpleDateFormat;
import java.util.Date;

public class WelcomeCookieServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h1>Welcome Page - Cookie Demo</h1>");
```

```
        // Get all cookies from request
        Cookie[] cookies = request.getCookies();
        String userName = null;
        String loginTime = null;

        if (cookies != null) {
            // Loop through all cookies
            for (Cookie cookie : cookies) {
                String name = cookie.getName();
                String value = cookie.getValue();

                if ("userName".equals(name)) {
                    userName = value;
                } else if ("loginTime".equals(name)) {
                    loginTime = value;
                }
            }
        }

        if (userName != null) {
            out.println("<h2 style='color:blue;'>Welcome back, " + userName +
"!</h2>");

            if (loginTime != null) {
                long timeInMillis = Long.parseLong(loginTime);
                Date date = new Date(timeInMillis);
                SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
                out.println("<p><strong>Login Time:</strong> " + sdf.format(date) +
"</p>");
            }

            out.println("<h3>All Cookies in your browser:</h3>");
            out.println("<table border='1' cellpadding='5'>");
            out.println("<tr><th>Cookie Name</th><th>Cookie Value</th><th>Max Age
(seconds)</th></tr>");

            for (Cookie cookie : cookies) {
                out.println("<tr>");
                out.println("<td>" + cookie.getName() + "</td>");
```

```
                out.println("<td>" + cookie.getValue() + "</td>");
                out.println("<td>" + cookie.getMaxAge() + "</td>");
                out.println("</tr>");
            }
            out.println("</table>");

            // Logout button
            out.println("<br><a href='logoutCookie'>Logout (Delete Cookie)</a>");

        } else {
            out.println("<h2 style='color:red;'>No cookie found!</h2>");
            out.println("<p>Please <a href='index.html'>login</a> first.</p>");
        }

        out.println("</body></html>");
        out.close();
    }
}

// ============================================
// 3. LogoutCookieServlet.java - Deletes cookies
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LogoutCookieServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get all cookies
        Cookie[] cookies = request.getCookies();

        if (cookies != null) {
```

```
            for (Cookie cookie : cookies) {
                // Delete each cookie by setting max age to 0
                cookie.setMaxAge(0);
                response.addCookie(cookie);
                System.out.println("Deleted cookie: " + cookie.getName());
            }
        }

        out.println("<html><body>");
        out.println("<h2 style='color:blue;'>Logged Out Successfully!</h2>");
        out.println("<p>All cookies have been deleted from your browser.</p>");
        out.println("<a href='index.html'>Login Again</a>");
        out.println("</body></html>");
        out.close();
    }
}

// ============================================
// 4. HiddenFieldDemo.java - Hidden form field example
// ============================================
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HiddenFieldDemo extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get hidden field value
        String userId = request.getParameter("userId");
        String userName = request.getParameter("userName");
        String pageId = request.getParameter("pageId");

        out.println("<html><body>");
```

```
        out.println("<h1>Hidden Field Demo</h1>");

        if (userId != null && !userId.isEmpty()) {
            out.println("<h3>Data received from hidden fields:</h3>");
            out.println("<ul>");
            out.println("<li><strong>User ID:</strong> " + userId + "</li>");
            out.println("<li><strong>User Name:</strong> " + userName + "</li>");
            out.println("<li><strong>Page ID:</strong> " + pageId + "</li>");
            out.println("</ul>");
        } else {
            // Show form with hidden fields
            out.println("<h3>Submit data using hidden fields:</h3>");
            out.println("<form method='GET' action='hiddenform'>");

            // Hidden fields - not visible to user
            out.println("<input type='hidden' name='userId' value='101'>");
            out.println("<input type='hidden' name='userName' value='Ahmed Khan'>");
            out.println("<input type='hidden' name='pageId' value='dashboard_001'>");

            out.println("<p>Hidden fields are in the form. Click submit to see
them.</p>");
            out.println("<input type='submit' value='Submit with Hidden Fields'>");
            out.println("</form>");
        }

        out.println("</body></html>");
        out.close();
    }
}
```

index.html (Login Form):

```
html
<!DOCTYPE html>
<html>
<head>
    <title>Login - Cookie Demo</title>
</head>
<body>
```

```
    <h2>Login Form</h2>
    <form action="cookieLogin" method="post">
        Username: <input type="text" name="username" required/><br/><br/>
        Password: <input type="password" name="password" required/><br/><br/>
        <input type="submit" value="Login"/>
    </form>
    <p style="color:gray;font-size:12px;">Try password: 123</p>
</body>
</html>
```

web.xml (Deployment Descriptor):

```
xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0">

    <!-- Cookie Login -->
    <servlet>
        <servlet-name>CookieLogin</servlet-name>
        <servlet-class>CookieLoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>CookieLogin</servlet-name>
        <url-pattern>/cookieLogin</url-pattern>
    </servlet-mapping>

    <!-- Welcome Cookie -->
    <servlet>
        <servlet-name>WelcomeCookie</servlet-name>
        <servlet-class>WelcomeCookieServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>WelcomeCookie</servlet-name>
        <url-pattern>/welcomeCookie</url-pattern>
    </servlet-mapping>

    <!-- Logout Cookie -->
    <servlet>
        <servlet-name>LogoutCookie</servlet-name>
```

```
        <servlet-class>LogoutCookieServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LogoutCookie</servlet-name>
        <url-pattern>/logoutCookie</url-pattern>
    </servlet-mapping>

    <!-- Hidden Field Demo -->
    <servlet>
        <servlet-name>HiddenField</servlet-name>
        <servlet-class>HiddenFieldDemo</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HiddenField</servlet-name>
        <url-pattern>/hiddenform</url-pattern>
    </servlet-mapping>

    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `CookieLoginServlet` login form se username aur password read karta hai. Agar password "123" hai toh `Cookie` object create karta hai — `new Cookie("userName", username)` — aur `response.addCookie()` se browser mein store kar deta hai.
- **Step 2:** `setMaxAge(1800)` cookie ko 30 minutes ke liye valid banata hai — yeh persistent cookie hai. Agar setMaxAge nahi karte toh non-persistent hoti (browser close hotay hi delete).
- **Step 3:** `WelcomeCookieServlet` mein `request.getCookies()` se saari cookies read karte hain — yeh array return karta hai. Loop mein jaakar userName aur loginTime cookies ko extract karte hain.

- **Step 4:** Agar userName cookie milti hai toh welcome message show karta hai aur saari cookies ki table display karta hai — cookie ka name, value, aur max age dikhata hai.
- **Step 5:** `LogoutCookieServlet` logout functionality provide karta hai — saari cookies ko `setMaxAge(0)` set karke delete kar deta hai. `response.addCookie()` call karna zaroori hai taake browser update ho jaye.
- **Step 6:** `HiddenFieldDemo` hidden form fields demonstrate karta hai — `input type='hidden'` fields user ko dikhti nahi lekin form submit hotay waqt server par bheji jati hain. Yeh cookie disable honay par bhi kaam karti hain.

PART B — Expected Output:

```
text
// Case 1: Successful Login
// Browser URL: http://localhost:8080/demo/cookieLogin

Login Successful!
Cookie has been created and stored in your browser.
Go to Welcome Page

// Case 2: Welcome Page (after login)
// Browser URL: http://localhost:8080/demo/welcomeCookie

Welcome Page - Cookie Demo

Welcome back, admin!

Login Time: 2026-06-28 16:30:15

All Cookies in your browser:
Cookie Name    | Cookie Value           | Max Age (seconds)
-----------------------------------------------------------
userName       | admin                  | 1800
loginTime      | 1719583815000          | 1800
JSESSIONID     | ABCD1234...            | -1

Logout (Delete Cookie)
```

```
// Case 3: After clicking Logout
// Browser URL: http://localhost:8080/demo/logoutCookie

Logged Out Successfully!
All cookies have been deleted from your browser.
Login Again

// Case 4: Hidden Form Field Demo
// Browser URL: http://localhost:8080/demo/hiddenform

Hidden Field Demo
Submit data using hidden fields:
[Hidden fields are in the form. Click submit to see them.]
[Submit with Hidden Fields]

// After clicking submit:
Hidden Field Demo
Data received from hidden fields:
- User ID: 101
- User Name: Ahmed Khan
- Page ID: dashboard_001
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke cookie successfully create hui, welcome page par username aur login time display hua, logout par cookies delete hui, aur hidden form fields ne data successfully transfer kiya. Yeh session tracking techniques ka practical implementation hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Forgetting Null Check on getCookies()

Buggy Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    Cookie[] cookies = req.getCookies();
    // No null check - will cause NullPointerException
    for (Cookie cookie : cookies) {
        out.println(cookie.getName());
    }
}
```

What's Wrong (Roman Urdu): Student ne `getCookies()` par null check nahi kiya. Agar browser mein koi cookie nahi hai toh `getCookies()` null return karegi, aur loop par NullPointerException aayegi.

Fixed Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    Cookie[] cookies = req.getCookies();
    // Always check for null
    if (cookies != null) {
        for (Cookie cookie : cookies) {
            out.println(cookie.getName() + ": " + cookie.getValue());
        }
    } else {
        out.println("No cookies found!");
    }
}
```

Fix Explained (Roman Urdu): `if (cookies != null)` check add kiya taake null pointer exception na aaye aur handle ho jaye.

Mistake 2: Setting Max Age to -1 Instead of 0 for Deletion

Buggy Code:

```
java
// Trying to delete cookie
Cookie ck = new Cookie("userName", "");
ck.setMaxAge(-1);  // Wrong: -1 means session cookie, not delete
response.addCookie(ck);
```

What's Wrong (Roman Urdu): Student ne cookie delete karne ke liye `setMaxAge(-1)` use kiya hai, lekin -1 ka matlab hai "session cookie" — browser close honay tak valid. Cookie delete nahi hogi. Delete karne ke liye 0 set karna chahiye.

Fixed Code:

```
java
// Correct way to delete cookie
Cookie ck = new Cookie("userName", "");
ck.setMaxAge(0);  // Correct: 0 means immediate deletion
response.addCookie(ck);
```

Fix Explained (Roman Urdu): `setMaxAge(0)` use kiya taake cookie immediately delete ho jaye. 0 ka matlab hai "expire immediately".

Mistake 3: Trying to Get Cookie Without Matching Name Exactly

Buggy Code:

```
java
Cookie[] cookies = req.getCookies();
// Case sensitive - "Username" vs "userName"
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if (cookie.getName().equals("Username")) {  // Wrong case
```

```
            String value = cookie.getValue();
        }
    }
}
```

What's Wrong (Roman Urdu): Student ne cookie name mein case mismatch kiya hai — server ne "userName" store kiya hai lekin search "Username" kar raha hai. Java case-sensitive hai, is liye cookie find nahi hogi.

Fixed Code:

```
java
Cookie[] cookies = req.getCookies();
// Use exact same case as stored
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if (cookie.getName().equals("userName")) {  // Correct case
            String value = cookie.getValue();
            out.println("Found: " + value);
        }
    }
}
```

Fix Explained (Roman Urdu): Cookie name ka exact case match kiya — "userName" as stored, "Username" nahi.

Mistake 4: Adding Cookie to Response After Content is Written

Buggy Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    PrintWriter out = res.getWriter();
    out.println("<html><body>");
```

```
    out.println("Hello World");
    // Wrong: Adding cookie after writing content
    Cookie ck = new Cookie("user", "value");
    res.addCookie(ck);  // May not work if response already committed
    out.println("</body></html>");
}
```

What's Wrong (Roman Urdu): Student ne response content write karne ke baad cookie add ki hai. Agar response already committed ho chuka hai toh cookie add nahi hogi. Cookies headers mein bheji jati hain, jo body se pehle bhejni chahiye.

Fixed Code:

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    // Add cookie BEFORE writing any content
    Cookie ck = new Cookie("user", "value");
    res.addCookie(ck);

    // Then write content
    PrintWriter out = res.getWriter();
    out.println("<html><body>");
    out.println("Hello World");
    out.println("</body></html>");
}
```

Fix Explained (Roman Urdu): Cookie add karne ka code response body se pehle move kiya taake headers body se pehle bheje jayein.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if a user logs in, receives a cookie, and then closes the browser without logging out? What if they reopen the browser and visit the welcome page?

Answer: Agar cookie persistent hai (setMaxAge positive value set ki hai), toh browser close karne par bhi cookie browser ke cache mein store rahegi. Jab user wapas browser open kare aur welcome page visit kare, toh server ko cookie milegi aur user automatically logged in state mein ho ga.

Agar cookie non-persistent hai (setMaxAge nahi set ki ya -1 set ki), toh browser close hotay hi cookie delete ho jati hai. Wapas browser open karne par cookie nahi milegi aur user ko login karna parega. Is liye persistent cookies "Remember Me" feature ke liye use hoti hain aur non-persistent cookies sensitive data (like session ID) ke liye.

Q: What is the difference between Cookies and Hidden Form Fields for session tracking? When would you prefer one over the other?

Answer: Cookies client-side (browser) par store hoti hain aur automatically har request ke saath bheji jati hain — user ko kuch nahi karna padta. Hidden form fields server-side par form ke andar store hoti hain aur sirf form submit karne par bheji jati hain — har page par form submit karna padta hai.

Cookies prefer karte hain jab user identification automatic ho, jaise login state maintain karna. Hidden form fields prefer karte hain jab cookie disabled ho browser mein ya jab humein specific page-to-page data transfer karna ho (jaise multi-step forms). Hidden form fields ka disadvantage yeh hai ke har page par form submit karna padta hai aur data URL ya body mein visible ho sakta hai.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method is used to add a cookie to the response? (response.addCookie(Cookie))
2. What will this code do?

```
java
Cookie ck = new Cookie("user", "Ali");
ck.setMaxAge(0);
response.addCookie(ck);
```

(Deletes the cookie immediately by setting its expiry to 0)

3. What is returned by request.getCookies() if no cookies exist? (null)
4. Which session tracking technique works even when cookies are disabled? (Hidden Form Field or URL Rewriting)
5. What is the difference between setMaxAge(-1) and setMaxAge(0)? (-1 means session cookie, 0 means delete immediately)

SECTION 1 — LESSON HEADING

Lesson 10: URL Rewriting, Http Session, and send Redirect

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum college mein apna ID card dikhaate ho har entry par, lekin agar tum card bhool jaate ho toh tum apna roll number zaban se bata dete ho (URL rewriting). Kabhi kabhi guard tumhe directly principal ke office bhej deta hai (sendRedirect), aur kabhi tumhara data office record mein store ho jata hai (HttpSession). Session tracking ke yeh saare tareeqay hain — user ko pehchanne aur data maintain karne ke liye.

PART B — Key Content List:

Key Terms:

1. **URL Rewriting**: Token ya identifier ko URL ke saath append karna — `url?name=value` format mein — taake data next resource tak pahunchaye.
2. **HttpSession**: Server-side object hai jo user-specific data store karta hai — container har user ke liye unique session ID create karta hai.
3. **sendRedirect()**: HttpServletResponse ka method hai jo client ko new URL par bhejta hai — browser ke URL bar mein change aata hai.
4. **getSession()**: HttpServletRequest ka method hai jo current session return karta hai, agar nahi hai toh naya create karta hai.
5. **Session ID**: Unique identifier hai jo container har user ke liye create karta hai — user ko recognize karne ke liye.
6. **invalidate()**: HttpSession ka method hai jo session ko end karta hai aur saari bound objects unbind kar deta hai.

Key Points:

1. **URL Rewriting Advantage**: Cookie disabled honay par bhi kaam karta hai — browser independent hai.
2. **URL Rewriting Disadvantage**: Sirf links ke saath kaam karta hai, aur sirf textual information bhej sakta hai.
3. **HttpSession vs Cookies**: HttpSession server-side data store karta hai (zyada secure), cookies client-side store karti hain.
4. **sendRedirect vs forward**: sendRedirect client-side hai (naya request), forward server-side hai (same request).
5. **sendRedirect Use Cases**: External URLs par bhejna, cross-application navigation, aur POST-Redirect-GET pattern.

Session Tracking Techniques:

1. **Cookies**: Client-side storage, automatic, but can be disabled.

2. **URL Rewriting**: URL mein data append, works when cookies disabled.
3. **Hidden Form Fields**: Form mein hidden fields, works when cookies disabled but needs form submission.
4. **HttpSession**: Server-side storage, most secure and feature-rich.

SECTION 3 — EXAMPLE

Example:

Socho tum online shopping kar rahe ho aur tumne products cart mein daale hain. Tum profile page par jaana chahte ho — agar cookie enabled hai toh server session ID se tumhe pehchan leta hai. Agar cookie disabled hai toh URL mein session ID append ho jati hai — `profile?jsessionid=ABC123`. Agar tum logout karte ho toh session invalidate ho jati hai. Agar tum Google search karte ho toh sendRedirect use hoti hai — tumhari query Google par bheji jati hai. Yahi URL rewriting, HttpSession, aur sendRedirect ka practical use hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
// ==========================================
// 1. LoginServlet.java - Login with Cookie and Session
// ==========================================
package com.cs311;
```

```java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Include navigation links
        request.getRequestDispatcher("link.html").include(request, response);

        // Get parameters from form
        String name = request.getParameter("name");
        String password = request.getParameter("password");

        // Validate credentials (hardcoded)
        if (password != null && password.equals("admin123")) {
            out.print("<p style='color:green;'>You are successfully logged in!</p>");
            out.print("<p>Welcome, " + name + "</p>");

            // 1. Create Cookie (for client-side storage)
            Cookie ck = new Cookie("name", name);
            ck.setMaxAge(30 * 60); // 30 minutes
            response.addCookie(ck);

            // 2. Create HttpSession (for server-side storage)
            HttpSession session = request.getSession();
            session.setAttribute("userName", name);
            session.setAttribute("loginTime", new java.util.Date().toString());

            out.print("<p>Session ID: " + session.getId() + "</p>");

        } else {
```

```
            out.print("<p style='color:red;'>Sorry, username or password
error!</p>");
            request.getRequestDispatcher("login.html").include(request, response);
        }

        out.close();
    }
}

// ============================================
// 2. ProfileServlet.java - Access protected page
// ============================================
package com.cs311;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ProfileServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Include navigation links
        request.getRequestDispatcher("link.html").include(request, response);

        // Try to get user from Cookie
        Cookie[] cookies = request.getCookies();
        String cookieName = null;

        if (cookies != null) {
            for (Cookie ck : cookies) {
                if ("name".equals(ck.getName())) {
                    cookieName = ck.getValue();
                    break;
```

```
                }
            }
        }

        // Try to get user from Session
        HttpSession session = request.getSession(false); // Don't create new
        String sessionName = null;
        String loginTime = null;

        if (session != null) {
            sessionName = (String) session.getAttribute("userName");
            loginTime = (String) session.getAttribute("loginTime");
        }

        // Check if user is logged in via cookie OR session
        if ((cookieName != null && !cookieName.isEmpty()) ||
            (sessionName != null && !sessionName.isEmpty())) {

            String displayName = (cookieName != null) ? cookieName : sessionName;

            out.print("<h2 style='color:blue;'>Welcome to Profile</h2>");
            out.print("<p><strong>User:</strong> " + displayName + "</p>");

            if (session != null && sessionName != null) {
                out.print("<p><strong>Session ID:</strong> " + session.getId() +
"</p>");
                out.print("<p><strong>Login Time:</strong> " + loginTime + "</p>");

                // Session creation time
                long creationTime = session.getCreationTime();
                out.print("<p><strong>Session Created:</strong> " +
                          new java.util.Date(creationTime) + "</p>");
            }

            // URL Rewriting Example - passing data via URL
            out.print("<h3>URL Rewriting Demo:</h3>");
            out.print("<a
href='AnotherServlet?userId=101&action=view&source=profile'>");
            out.print("Go to Another Servlet (with URL parameters)</a>");
```

```
        } else {
            out.print("<p style='color:red;'>Please login first</p>");
            request.getRequestDispatcher("login.html").include(request, response);
        }

        out.close();
    }
}

// ============================================
// 3. LogoutServlet.java - Logout and cleanup
// ============================================
package com.cs311;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        request.getRequestDispatcher("link.html").include(request, response);

        // 1. Delete Cookie
        Cookie ck = new Cookie("name", "");
        ck.setMaxAge(0);
        response.addCookie(ck);

        // 2. Invalidate Session
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
```

```
            out.print("<p>Session invalidated successfully!</p>");
        }

        out.print("<p style='color:blue;'>You are successfully logged out!</p>");
        out.close();
    }
}

// ============================================
// 4. AnotherServlet.java - URL Rewriting Demo
// ============================================
package com.cs311;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class AnotherServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Read parameters from URL (URL Rewriting)
        String userId = request.getParameter("userId");
        String action = request.getParameter("action");
        String source = request.getParameter("source");

        out.println("<html><body>");
        out.println("<h1>URL Rewriting Demo</h1>");
        out.println("<p><strong>Data received from URL parameters:</strong></p>");
        out.println("<ul>");
        out.println("<li><strong>User ID:</strong> " + userId + "</li>");
        out.println("<li><strong>Action:</strong> " + action + "</li>");
        out.println("<li><strong>Source:</strong> " + source + "</li>");
        out.println("</ul>");
```

```
        // Show how URL rewriting works with links
        out.println("<h3>URL Rewriting with Links:</h3>");
        out.println("<a href='AnotherServlet?userId=202&action=edit&source=link1'>");
        out.println("Edit User 202 (Link 1)</a><br>");
        out.println("<a
href='AnotherServlet?userId=303&action=delete&source=link2'>");
        out.println("Delete User 303 (Link 2)</a><br>");

        out.println("<a href='ProfileServlet'>Back to Profile</a>");
        out.println("</body></html>");
        out.close();
    }
}

// ============================================
// 5. GoogleSearchServlet.java - sendRedirect Example
// ============================================
package com.cs311;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class GoogleSearchServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // Get search query from form
        String query = request.getParameter("name");

        if (query != null && !query.isEmpty()) {
            // sendRedirect to Google with the query
            // This works because sendRedirect works at client side
            String googleUrl = "https://www.google.co.in/search?q=" + query;
            response.sendRedirect(googleUrl);
```

```
        } else {
            // If no query, show form
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            out.println("<html><body>");
            out.println("<h2>Google Search</h2>");
            out.println("<form action='googleSearch' method='get'>");
            out.println("Search: <input type='text' name='name' required>");
            out.println("<input type='submit' value='Search'>");
            out.println("</form>");
            out.println("</body></html>");
            out.close();
        }
    }
}
```

HTML Files:

index.html:

```
html
<!DOCTYPE html>
<html>
<head>
    <title>Login App</title>
</head>
<body>
    <h1>Welcome to Login App</h1>
    <a href="login.html">Login</a> |
    <a href="LogoutServlet">Logout</a> |
    <a href="ProfileServlet">Profile</a> |
    <a href="googleSearch.html">Google Search</a>
</body>
</html>
```

link.html:

```
html
<a href="login.html">Login</a> |
<a href="LogoutServlet">Logout</a> |
```

```
<a href="ProfileServlet">Profile</a> |
<a href="googleSearch.html">Google Search</a>
<hr>
```

login.html:

```
html
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <h2>Login Form</h2>
    <form action="LoginServlet" method="post">
        Name: <input type="text" name="name" required><br><br>
        Password: <input type="password" name="password" required><br><br>
        <input type="submit" value="Login">
    </form>
    <p style="color:gray;">Try password: admin123</p>
</body>
</html>
```

googleSearch.html:

```
html
<!DOCTYPE html>
<html>
<head>
    <title>Google Search</title>
</head>
<body>
    <h2>Google Search using sendRedirect</h2>
    <form action="googleSearch" method="get">
        <input type="text" name="name" size="40" placeholder="Enter search term..."
required>
        <input type="submit" value="Google Search">
    </form>
    <p>This uses response.sendRedirect() to send you to Google</p>
</body>
```

```
</html>
```

web.xml:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0">

    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>com.cs311.LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>ProfileServlet</servlet-name>
        <servlet-class>com.cs311.ProfileServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>ProfileServlet</servlet-name>
        <url-pattern>/ProfileServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>LogoutServlet</servlet-name>
        <servlet-class>com.cs311.LogoutServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LogoutServlet</servlet-name>
        <url-pattern>/LogoutServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>AnotherServlet</servlet-name>
        <servlet-class>com.cs311.AnotherServlet</servlet-class>
    </servlet>
```

```
    <servlet-mapping>
        <servlet-name>AnotherServlet</servlet-name>
        <url-pattern>/AnotherServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>GoogleSearch</servlet-name>
        <servlet-class>com.cs311.GoogleSearchServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>GoogleSearch</servlet-name>
        <url-pattern>/googleSearch</url-pattern>
    </servlet-mapping>

    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `LoginServlet` doPost method mein username aur password read karta hai. Agar password "admin123" hai toh user login hota hai — cookie create karta hai (client-side) aur HttpSession create karta hai (server-side). Session mein username aur login time store karta hai.
- **Step 2:** Cookie `setMaxAge(30*60)` se 30 minutes ke liye persistent banayi gayi hai. Session `getSession()` se create hui hai — container automatically session ID generate karta hai.
- **Step 3:** `ProfileServlet` mein hum dono tareekon se user check karte hain — pehle cookies array se "name" cookie search karte hain, phir session se "userName" attribute retrieve karte hain. Agar koi bhi milta hai toh profile page show hota hai.
- **Step 4:** Profile page mein URL Rewriting ka demo hai — `AnotherServlet?userId=101&action=view&source=profile` — yeh parameters URL mein append ho gaye hain. `AnotherServlet` mein `request.getParameter()` se yeh values read karte hain.

- **Step 5:** `LogoutServlet` mein cookie delete karne ke liye `setMaxAge(0)` use kiya hai aur session invalidate karne ke liye `session.invalidate()` call kiya hai. Dono tareekon se user data clear ho jata hai.
- **Step 6:** `GoogleSearchServlet` mein `response.sendRedirect()` use kiya hai — yeh browser ko Google par bhej deta hai. URL mein query parameter append kiya gaya hai. Yeh external URL par redirect hai jo forward() se nahi ho sakta.

PART B — Expected Output:

```
text
// Case 1: Successful Login (POST)
// After submitting login form with admin/admin123:

[link.html navigation bar]

You are successfully logged in!
Welcome, admin
Session ID: 0A1B2C3D4E5F6789...

// Case 2: Profile Page (GET)
// After clicking Profile link:

[link.html navigation bar]

Welcome to Profile
User: admin
Session ID: 0A1B2C3D4E5F6789...
Login Time: Wed Jun 28 16:30:15 PKT 2026
Session Created: Wed Jun 28 16:30:15 PKT 2026

URL Rewriting Demo:
Go to Another Servlet (with URL parameters)

// Case 3: URL Rewriting - Another Servlet
// After clicking "Go to Another Servlet":

URL Rewriting Demo
Data received from URL parameters:
```

```
- User ID: 101
- Action: view
- Source: profile

URL Rewriting with Links:
Edit User 202 (Link 1)
Delete User 303 (Link 2)
Back to Profile

// Case 4: Logout (GET)
// After clicking Logout:

[link.html navigation bar]
Session invalidated successfully!
You are successfully logged out!

// Case 5: Google Search (sendRedirect)
// After searching "servlet tutorial":

Browser redirects to:
https://www.google.co.in/search?q=servlet+tutorial
(Google search results page)

// Case 6: Access Profile After Logout (GET)
// After clicking Profile after logout:

[link.html navigation bar]
Please login first
[login.html form appears]
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke login successful hua — cookie aur session dono create hue. Profile page ne cookie aur session se user data read kiya. URL rewriting ne parameters successfully transfer kiye. Logout ne cookie delete aur session invalidate kar di. sendRedirect ne browser ko Google par redirect kar diya. Yeh saari session tracking techniques ka complete demonstration hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Not Checking for Null When Getting Session

Buggy Code:

```
java
HttpSession session = request.getSession(false);
String name = (String) session.getAttribute("userName"); // NullPointerException
// No null check on session
```

What's Wrong (Roman Urdu): Student ne `getSession(false)` call kiya hai lekin agar session exist nahi karta toh null return hota hai. Phir session par method call karte hain toh NullPointerException aayegi.

Fixed Code:

```
java
HttpSession session = request.getSession(false);
String name = null;
if (session != null) {  // Always check for null
    name = (String) session.getAttribute("userName");
}
if (name == null) {
    // User not logged in
}
```

Fix Explained (Roman Urdu): `if (session != null)` check add kiya taake null pointer exception na aaye aur gracefully handle ho jaye.

Mistake 2: Confusing sendRedirect and forward with External URLs

Buggy Code:

```
java
```

```java
// Trying to forward to external URL - WILL NOT WORK
RequestDispatcher rd = request.getRequestDispatcher("https://www.google.com");
rd.forward(request, response);  // This will throw exception
```

What's Wrong (Roman Urdu): Student forward() use kar raha hai external URL par, lekin forward() sirf same server ke internal resources ke liye kaam karta hai. External URL ke liye sendRedirect() use karna chahiye.

Fixed Code:

```java
// Correct: Use sendRedirect for external URLs
response.sendRedirect("https://www.google.com");
// Also works for internal resources with full URL
response.sendRedirect("/demo/ProfileServlet");
```

Fix Explained (Roman Urdu): External URL par bhejne ke liye sendRedirect use kiya — yeh client-side redirection hai jo kahin bhi bhej sakta hai.

Mistake 3: Storing Non-Serializable Objects in Session

Buggy Code:

```java
// Custom class not Serializable
class UserData {
    String name;
    String email;
}

// Storing in session
UserData data = new UserData();
session.setAttribute("userData", data);  // May cause issues in distributed env
```

What's Wrong (Roman Urdu): Student ne session mein non-serializable object store kiya hai. Agar application distributed environment (cluster) mein deploy hai toh session objects serializable hone chahiye taake server nodes ke darmiyaan share ho sakein.

Fixed Code:

```
java
import java.io.Serializable;

class UserData implements Serializable {  // Implement Serializable
    String name;
    String email;
}

// Now safe to store in session
UserData data = new UserData();
session.setAttribute("userData", data);
```

Fix Explained (Roman Urdu): `Serializable` interface implement karwaya taake object cluster environment mein safely share ho sake.

Mistake 4: Forgetting to Include Cookies When Creating URL for Rewriting

Buggy Code:

```
java
// In welcome page, trying to create link with session info
// But not encoding the URL
out.println("<a href='ProfileServlet'>Profile</a>");
// If cookies disabled, session ID won't be in URL
```

What's Wrong (Roman Urdu): Student ne URL rewriting use kiya hai lekin `HttpServletResponse.encodeURL()` method use nahi kiya. Agar cookies disabled hain

toh container automatically session ID ko URL mein append nahi karega jab tak hum encodeURL use nahi karte.

Fixed Code:

```
java
// Correct way: Use encodeURL for URL rewriting
String url = response.encodeURL("ProfileServlet");
out.println("<a href='" + url + "'>Profile</a>");
// If cookies disabled, URL becomes "ProfileServlet;jsessionid=ABC123"
// If cookies enabled, URL remains "ProfileServlet"
```

Fix Explained (Roman Urdu): `response.encodeURL()` use kiya taake container automatically decide kare ke session ID append karna hai ya nahi (cookies disabled honay par append karega).

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if the following code is executed when cookies are disabled in the browser, and why?

```
java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    HttpSession session = req.getSession();
    session.setAttribute("user", "Ali");
    res.sendRedirect("welcome");
}
```

Answer: Agar cookies disabled hain toh session ID cookie browser mein store nahi hogi, lekin container URL rewriting ka use karega. Jab hum `sendRedirect("welcome")` karte hain toh container automatically session ID ko URL mein append karega agar hum `encodeRedirectURL()` use karein. Agar hum sirf plain URL use karein toh session ID nahi append hogi aur user ki session lost ho jayegi. Is liye

humesha `response.encodeRedirectURL("welcome")` use karna chahiye taake session ID URL mein include ho.

Q: What is the difference between `getSession()` and `getSession(false)` in terms of behavior, and when would you use each?

Answer: `getSession()` returns current session, agar session exist nahi karta toh naya create kar deta hai. `getSession(false)` returns current session agar exist karta hai, warna null return karta hai — naya session nahi create karta.

`getSession()` use karte hain jab humein session create karna ho, jaise login page par user authenticate ho raha ho aur humein session start karna ho. `getSession(false)` use karte hain jab humein check karna ho ke user already logged in hai ya nahi, jaise profile page par — agar session null hai toh user ko login page par bhej dena chahiye.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. What is the output of `request.getSession(false)` if no session exists? (null)
2. Which method would you use to redirect to an external website like Google? (response.sendRedirect())
3. What will happen in this code if cookies are disabled?

```
java
String url = response.encodeURL("ProfileServlet");
out.println("<a href='" + url + "'>Profile</a>");
```

(The URL will include jsessionid parameter for URL rewriting)

4. Which session tracking technique appends data to the URL? (URL Rewriting)

5. What is the difference between forward() and sendRedirect()? (forward is server-side with same request, sendRedirect is client-side with new request)

SECTION 1 — LESSON HEADING

Lesson 11: HttpSession Login/Logout Application

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum hotel mein check-in karte ho aur receptionist tumhe ek room key deta hai. Jab tum hotel ke andar kisi bhi jagah jaate ho, tum woh key dikhate ho aur woh tumhe pehchan lete hain. HttpSession bhi yahi hai — server tumhe ek unique session ID deta hai, aur har request ke saath woh ID check hoti hai taake server tumhe pehchan sake aur tumhara data maintain kar sake.

PART B — Key Content List:

Key Terms:

1. **HttpSession**: Server-side object hai jo user-specific data store karta hai — container har user ke liye unique session ID create karta hai.
2. **Session ID**: Unique identifier hai jo container create karta hai — user ko recognize karne ke liye.
3. **getSession()**: HttpServletRequest ka method hai jo current session return karta hai, agar nahi hai toh naya create kar deta hai.
4. **getSession(false)**: HttpServletRequest ka method hai jo current session return karta hai agar exist karta hai, warna null return karta hai — naya session nahi create karta.

5. **setAttribute()**: HttpSession ka method hai jo session mein object store karta hai — key-value pair ki tarah.
6. **getAttribute()**: HttpSession ka method hai jo session se stored object retrieve karta hai.
7. **invalidate()**: HttpSession ka method hai jo session ko end karta hai — saari stored objects unbind ho jati hain.
8. **getCreationTime()**: Session kab create hui thi, yeh time milliseconds mein return karta hai.
9. **getLastAccessedTime()**: Client ne session ke saath last request kab bheji, yeh time return karta hai.

Key Points:

1. **Server-Side Storage**: HttpSession data server par store hota hai — cookies ki tarah client par nahi, is liye zyada secure hai.
2. **Automatic Session Management**: Container automatically session ID manage karta hai — humein sirf data store/retrieve karna hota hai.
3. **Session vs Cookie**: Session server-side data store karta hai, cookie client-side — session zyada secure aur zyada data store kar sakta hai.
4. **Login/Logout Pattern**: Login par session create karte hain, profile page par session check karte hain, logout par session invalidate karte hain.

SECTION 3 — EXAMPLE

Example:

Socho tum bank mein account kholte ho. Bank tumhe ek account number deta hai (session ID). Jab bhi tum bank mein aate ho, tum woh account number batate ho aur bank tumhara balance, transactions, etc. check karta hai (session data). Jab tum account close karte ho (logout), toh bank tumhara record archive kar deta hai (session invalidate). HttpSession bhi yahi hai — user login karta hai, session create hoti hai, user data session mein store hota hai, aur logout par session invalidate ho jati hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

java

```java
// ============================================
// 1. LoginServlet.java - User Login with HttpSession
// ============================================
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Include navigation links
        request.getRequestDispatcher("link.html").include(request, response);

        // Get parameters from login form
        String name = request.getParameter("name");
        String password = request.getParameter("password");

        // Validate credentials (hardcoded for demo)
        if (password != null && password.equals("admin123")) {
```

```
            // Login successful - show welcome message
            out.print("<p style='color:green;'>Welcome, " + name + "!</p>");

            // Create or get session and store user data
            HttpSession session = request.getSession();
            session.setAttribute("name", name);

            // Optional: Store login time
            session.setAttribute("loginTime", new java.util.Date().toString());

            out.print("<p>Session created! Session ID: " + session.getId() + "</p>");
            out.print("<p><a href='ProfileServlet'>Go to Profile</a></p>");

        } else {
            // Login failed
            out.print("<p style='color:red;'>Sorry, username or password
error!</p>");
            request.getRequestDispatcher("login.html").include(request, response);
        }

        out.close();
    }
}

// ============================================
// 2. ProfileServlet.java - Access protected page
// ============================================
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class ProfileServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Include navigation links
        request.getRequestDispatcher("link.html").include(request, response);

        // Get session WITHOUT creating a new one
        HttpSession session = request.getSession(false);

        if (session != null) {
            // Session exists - get user data
            String name = (String) session.getAttribute("name");
            String loginTime = (String) session.getAttribute("loginTime");

            out.print("<h2 style='color:blue;'>Welcome to Profile</h2>");
            out.print("<p><strong>User:</strong> " + name + "</p>");
            out.print("<p><strong>Session ID:</strong> " + session.getId() + "</p>");
            out.print("<p><strong>Login Time:</strong> " + loginTime + "</p>");

            // Session creation and last access times
            long creationTime = session.getCreationTime();
            long lastAccessTime = session.getLastAccessedTime();

            out.print("<p><strong>Session Created:</strong> " +
                    new java.util.Date(creationTime) + "</p>");
            out.print("<p><strong>Last Access:</strong> " +
                    new java.util.Date(lastAccessTime) + "</p>");

        } else {
            // No session - user not logged in
            out.print("<p style='color:red;'>Please login first</p>");
            request.getRequestDispatcher("login.html").include(request, response);
        }

        out.close();
    }
}
```

```java
// ============================================
// 3. LogoutServlet.java - User Logout
// ============================================
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Include navigation links
        request.getRequestDispatcher("link.html").include(request, response);

        // Get session WITHOUT creating a new one
        HttpSession session = request.getSession(false);

        if (session != null) {
            // Invalidate the session - removes all data
            session.invalidate();
            out.print("<p style='color:blue;'>You are successfully logged out!</p>");
            out.print("<p>Session has been invalidated.</p>");
        } else {
            out.print("<p style='color:orange;'>You are already logged out.</p>");
        }

        out.close();
    }
}
```

```java
// ==========================================
// 4. SessionInfoServlet.java - Display all session info
// ==========================================
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Enumeration;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class SessionInfoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        request.getRequestDispatcher("link.html").include(request, response);

        out.println("<h2>Session Information</h2>");

        HttpSession session = request.getSession(false);

        if (session != null) {
            out.println("<h3>Session Details:</h3>");
            out.println("<ul>");
            out.println("<li><strong>Session ID:</strong> " + session.getId() +
"</li>");
            out.println("<li><strong>Creation Time:</strong> " +
                    new java.util.Date(session.getCreationTime()) + "</li>");
            out.println("<li><strong>Last Access Time:</strong> " +
                    new java.util.Date(session.getLastAccessedTime()) + "</li>");
            out.println("<li><strong>Max Inactive Interval:</strong> " +
                    session.getMaxInactiveInterval() + " seconds</li>");
```

```
            out.println("<li><strong>Is New?</strong> " + session.isNew() + "</li>");
            out.println("</ul>");

            // Show all session attributes
            out.println("<h3>Session Attributes:</h3>");
            Enumeration<String> attributeNames = session.getAttributeNames();

            if (attributeNames.hasMoreElements()) {
                out.println("<ul>");
                while (attributeNames.hasMoreElements()) {
                    String attrName = attributeNames.nextElement();
                    Object attrValue = session.getAttribute(attrName);
                    out.println("<li><strong>" + attrName + ":</strong> " +
                                attrValue + "</li>");
                }
                out.println("</ul>");
            } else {
                out.println("<p>No attributes stored in session.</p>");
            }

        } else {
            out.println("<p style='color:red;'>No active session. Please login
first.</p>");
        }

        out.close();
    }
}
```

HTML Files:

index.html:

```
html
<!DOCTYPE html>
<html>
<head>
    <meta charset="ISO-8859-1">
    <title>Servlet Login Example</title>
```

```
</head>
<body>
    <h1>Login App using HttpSession</h1>
    <a href="login.html">Login</a> |
    <a href="LogoutServlet">Logout</a> |
    <a href="ProfileServlet">Profile</a> |
    <a href="SessionInfoServlet">Session Info</a>
</body>
</html>
```

link.html:

```
html
<a href="login.html">Login</a> |
<a href="LogoutServlet">Logout</a> |
<a href="ProfileServlet">Profile</a> |
<a href="SessionInfoServlet">Session Info</a>
<hr>
```

login.html:

```
html
<!DOCTYPE html>
<html>
<head>
    <meta charset="ISO-8859-1">
    <title>Login</title>
</head>
<body>
    <h2>Login Form</h2>
    <form action="LoginServlet" method="post">
        Name: <input type="text" name="name" required><br><br>
        Password: <input type="password" name="password" required><br><br>
        <input type="submit" value="Login">
    </form>
    <p style="color:gray;">Try password: admin123</p>
</body>
</html>
```

web.xml:

xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0">

    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>ProfileServlet</servlet-name>
        <servlet-class>ProfileServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>ProfileServlet</servlet-name>
        <url-pattern>/ProfileServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>LogoutServlet</servlet-name>
        <servlet-class>LogoutServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LogoutServlet</servlet-name>
        <url-pattern>/LogoutServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>SessionInfoServlet</servlet-name>
        <servlet-class>SessionInfoServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>SessionInfoServlet</servlet-name>
        <url-pattern>/SessionInfoServlet</url-pattern>
    </servlet-mapping>
```

```
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `LoginServlet` doPost method mein username aur password read karta hai. Agar password "admin123" hai toh `request.getSession()` call karta hai — yeh session create karta hai agar nahi hai toh existing return karta hai. `session.setAttribute("name", name)` se user data session mein store karta hai.
- **Step 2:** `ProfileServlet` mein `request.getSession(false)` use kiya hai — yeh sirf existing session return karta hai, naya nahi create karta. Agar session null hai toh user login nahi hai, is liye login page dikhata hai.
- **Step 3:** Agar session exist karta hai toh `session.getAttribute("name")` se username retrieve karta hai aur profile page display karta hai. Session ID, creation time, aur last access time bhi display karta hai.
- **Step 4:** `LogoutServlet` mein `request.getSession(false)` se session retrieve karta hai. Agar session exist karta hai toh `session.invalidate()` call karta hai — yeh session ko immediately end kar deta hai aur saari attributes unbind ho jati hain.
- **Step 5:** `SessionInfoServlet` session ki complete information display karta hai — ID, creation time, last access time, max inactive interval, isNew status, aur saari attributes. Enumeration use karke saari attribute names retrieve karta hai.
- **Step 6:** Session tracking ka complete flow — login par session create hoti hai, profile par session check hoti hai, logout par session invalidate hoti hai, aur session info page par details dikhti hain.

PART B — Expected Output:

```
text
// Case 1: Login Page (GET)
// Browser URL: http://localhost:8080/demo/login.html
```

```
Login Form
Name: [_____]
Password: [_____]
[Login Button]
Try password: admin123

// Case 2: After Successful Login (POST)
// After submitting login form with admin/admin123:

Login | Logout | Profile | Session Info
---------------------------------------

Welcome, admin!
Session created! Session ID: 0A1B2C3D4E5F6789...
Go to Profile

// Case 3: Profile Page (GET)
// After clicking Profile link:

Login | Logout | Profile | Session Info
---------------------------------------

Welcome to Profile
User: admin
Session ID: 0A1B2C3D4E5F6789...
Login Time: Wed Jun 28 16:30:15 PKT 2026
Session Created: Wed Jun 28 16:30:15 PKT 2026
Last Access: Wed Jun 28 16:30:15 PKT 2026

// Case 4: Session Info Page (GET)
// After clicking Session Info link:

Login | Logout | Profile | Session Info
---------------------------------------

Session Information
Session Details:
- Session ID: 0A1B2C3D4E5F6789...
- Creation Time: Wed Jun 28 16:30:15 PKT 2026
```

```
- Last Access Time: Wed Jun 28 16:31:00 PKT 2026
- Max Inactive Interval: 1800 seconds
- Is New? false

Session Attributes:
- name: admin
- loginTime: Wed Jun 28 16:30:15 PKT 2026

// Case 5: After Logout (GET)
// After clicking Logout link:

Login | Logout | Profile | Session Info
---------------------------------------

You are successfully logged out!
Session has been invalidated.

// Case 6: Profile After Logout (GET)
// After clicking Profile link after logout:

Login | Logout | Profile | Session Info
---------------------------------------

Please login first
[Login Form appears]
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke session successfully create hui, user data session mein store hua, profile page ne session se data read kiya, session info page ne saari details dikhayi, logout ne session invalidate kar di, aur logout ke baad profile page par user ko login karna parha. Yeh complete session management lifecycle hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Not Checking for Null When Using getSession(false)

Buggy Code:

```
java
HttpSession session = request.getSession(false);
String name = (String) session.getAttribute("name"); // NullPointerException
// If session is null, this line will throw exception
```

What's Wrong (Roman Urdu): Student ne `getSession(false)` call kiya hai jo null return kar sakta hai agar session exist nahi karti. Phir session par getAttribute call karte hain toh NullPointerException aayegi. Session check karna zaroori hai.

Fixed Code:

```
java
HttpSession session = request.getSession(false);
if (session != null) {  // Always check for null
    String name = (String) session.getAttribute("name");
    // Process user data
} else {
    // User not logged in - redirect to login
    response.sendRedirect("login.html");
}
```

Fix Explained (Roman Urdu): `if (session != null)` check add kiya taake null pointer exception na aaye. Agar session null hai toh user ko login page par bhej dena chahiye.

Mistake 2: Forgetting to Close PrintWriter

Buggy Code:

```
java
```

```
protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    PrintWriter out = res.getWriter();
    out.println("<h1>Hello</h1>");
    // Missing out.close()
}
```

What's Wrong (Roman Urdu): Student ne PrintWriter close nahi kiya. Agar output stream close nahi karte toh response properly complete nahi ho sakti aur buffer flush nahi ho ga, is liye client ko incomplete response mil sakta hai.

Fixed Code:

java

```
protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    PrintWriter out = res.getWriter();
    try {
        out.println("<h1>Hello</h1>");
    } finally {
        out.close();  // Always close the writer
    }
}
```

Fix Explained (Roman Urdu): `out.close()` call kiya finally block mein taake output stream properly close ho jaye aur response complete ho.

Mistake 3: Using getSession() in Profile Without Checking

Buggy Code:

java

```
// In ProfileServlet
HttpSession session = request.getSession();  // Creates new session if none exists
// This will create a session for unauthorized users too
```

What's Wrong (Roman Urdu): Student ne `getSession()` use kiya hai jo naya session create kar deta hai agar exist nahi karti. Isliye unauthorized user bhi session create kar lete hain, jo server resources waste karta hai aur security risk hai.

Fixed Code:

```
java
// Correct: Use getSession(false) for checking login status
HttpSession session = request.getSession(false);
if (session != null) {
    String name = (String) session.getAttribute("name");
    // User is logged in
} else {
    // User not logged in - show login page
}
```

Fix Explained (Roman Urdu): `getSession(false)` use kiya taake sirf existing session check ho, naya session na bane. Is se unauthorized users ke liye session create nahi hogi.

Mistake 4: Invalidating Session But Still Using it Later

Buggy Code:

```
java
HttpSession session = request.getSession();
session.setAttribute("user", "Ali");
session.invalidate();  // Session is now invalid
String name = (String) session.getAttribute("user");  // IllegalStateException
```

What's Wrong (Roman Urdu): Student ne session invalidate kar di hai, phir bhi usi session par getAttribute call kar raha hai. invalidate ke baad session unusable ho jati hai, aur us par koi bhi method call IllegalStateException throw karegi.

Fixed Code:

```java
HttpSession session = request.getSession();
session.setAttribute("user", "Ali");
session.invalidate();
// Don't use session after invalidate
// If you need data, get it before invalidating
```

Fix Explained (Roman Urdu): invalidate ke baad session use nahi karna chahiye. Agar data chahiye toh invalidate se pehle retrieve kar lena chahiye.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen when the following code is executed and why?

```java
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    HttpSession session = req.getSession(false);
    if (session == null) {
        session = req.getSession();
        session.setAttribute("user", "Guest");
    }
    PrintWriter out = res.getWriter();
    out.println("User: " + session.getAttribute("user"));
}
```

Answer: Pehli baar jab yeh code execute hogi, `getSession(false)` null return karegi kyunke session exist nahi karti. Phir `getSession()` nayi session create karegi aur "Guest" set karega. Output "User: Guest" aayega. Doosri baar jab yeh code execute hogi, `getSession(false)` existing session return karegi, "user" attribute already set hai toh output "User: Guest" hi aayega. Is logic mein issue yeh hai ke har user ko "Guest" hi

milega chahe woh login ho ya nahi, kyunke session attribute sirf pehli visit par set hota hai.

Q: What is the difference between using `setAttribute()` on HttpSession vs using Cookies for storing user data, and when would you prefer one over the other?

Answer: HttpSession server-side data store karta hai — data server par secure hai, client use nahi dekh sakta, aur hum objects bhi store kar sakte hain (strings, numbers, lists, custom objects). Cookies client-side store karti hain — data browser mein visible hai, sirf textual data store kar sakte hain, aur size limit 4KB hai.

HttpSession prefer karte hain jab sensitive data store karna ho (passwords, personal info, shopping cart objects) ya jab complex data structures chahiyein. Cookies prefer karte hain jab humein client-side data chahiye (user preferences, theme selection, "Remember Me" flag) ya jab server memory save karna ho.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method would you use to get an existing session without creating a new one? (getSession(false))
2. What will this code do?

```java
HttpSession session = request.getSession();
session.invalidate();
```

(Invalidates the session and unbinds all attributes)

3. What is returned by `getSession(false)` if no session exists? (null)
4. What happens if you try to call `getAttribute()` on a session after `invalidate()` is called? (IllegalStateException is thrown)
5. What is the difference between `getSession()` and `getSession(false)`? (getSession() creates new session if none exists, getSession(false) returns null if no session exists)

SECTION 1 — LESSON HEADING

Lesson 12: Introduction to JDBC and Database Connectivity

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum kisi restaurant mein jaate ho aur waiter tumhara order leta hai, kitchen mein chef ko order pahunchata hai, aur phir tumhe khana serve karta hai. JDBC bhi yahi hai — Java application (tum) database (kitchen) se baat karti hai, JDBC driver (waiter) unke darmiyaan communication handle karta hai, aur data (khana) lekar aata hai.

PART B — Key Content List:

Key Terms:

1. **JDBC**: Java API hai jo Java applications ko databases se connect karne aur queries execute karne ki facility provide karta hai.
2. **JDBC Driver**: Software component hai jo Java application aur database ke darmiyaan communication provide karta hai.
3. **ODBC**: Purani technology hai jo C language mein likhi gayi thi — platform dependent aur unsecured thi, is liye JDBC banaya gaya.

4. **DriverManager**: Class hai jo database connection establish karne ke liye appropriate driver manage karti hai.
5. **Connection**: Interface hai jo database ke saath established connection represent karti hai.
6. **Statement**: Interface hai jo SQL queries execute karne ke liye use hoti hai.
7. **ResultSet**: Interface hai jo query results ko table format mein represent karti hai.

Types:

1. **JDBC-ODBC Bridge Driver**: ODBC driver use karta hai, C language mein likha hai — performance slow hai, ab discouraged hai.
2. **Native-API Driver**: Database ki client-side libraries use karta hai, partially Java mein likha hai — client machine par installation required hai.
3. **Network Protocol Driver**: Middleware (application server) use karta hai, fully Java mein likha hai — network support required hai.
4. **Thin Driver**: Directly database vendor-specific protocol use karta hai, fully Java mein likha hai — sab se fast aur best performance hai.

Steps:

1. **Register Driver Class**: `Class.forName()` method use karte hain driver class load karne ke liye.
2. **Create Connection**: `DriverManager.getConnection()` method use karte hain database connection establish karne ke liye.
3. **Create Statement**: `Connection.createStatement()` method use karte hain statement object banane ke liye.
4. **Execute Query**: `Statement.executeQuery()` ya `executeUpdate()` method use karte hain SQL queries execute karne ke liye.
5. **Close Connection**: `Connection.close()` method use karte hain resources release karne ke liye.

SECTION 3 — EXAMPLE

Example:

Socho tum library management system bana rahe ho. Java program mein students ki list database se fetch karni hai. Pehle tum JDBC driver load karte ho (jaise library ka gate open karna), phir database connection establish karte ho (library mein entry lena), phir statement create karte ho (librarian se book list maangna), query execute karte ho (list mil jati hai), aur akhir mein connection close karte ho (library se bahar aana). Yahi JDBC connectivity ka 5-step process hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```java
// ============================================
// Complete JDBC Example - MySQL Database
// ============================================
import java.sql.*;

public class JDBCDemo {

    // Database credentials
    static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
    static final String DB_URL = "jdbc:mysql://localhost:3306/sonoo";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;
```

```
try {
    // Step 1: Register the driver class (load JDBC driver)
    System.out.println("Step 1: Loading JDBC Driver...");
    Class.forName(JDBC_DRIVER);
    System.out.println("Driver loaded successfully!");

    // Step 2: Create the connection object
    System.out.println("\nStep 2: Connecting to database...");
    conn = DriverManager.getConnection(DB_URL, USER, PASS);
    System.out.println("Connected to database successfully!");

    // Step 3: Create the statement object
    System.out.println("\nStep 3: Creating statement...");
    stmt = conn.createStatement();

    // Step 4: Execute queries

    // 4a: CREATE TABLE (DDL operation)
    System.out.println("\nStep 4a: Creating table...");
    String createTableSQL = "CREATE TABLE IF NOT EXISTS emp (" +
                            "id INT(10), " +
                            "name VARCHAR(40), " +
                            "age INT(3))";
    stmt.executeUpdate(createTableSQL);
    System.out.println("Table created successfully!");

    // 4b: INSERT data (DML operation)
    System.out.println("\nStep 4b: Inserting data...");
    String insertSQL = "INSERT INTO emp (id, name, age) VALUES " +
                       "(1, 'Ahmed Khan', 25), " +
                       "(2, 'Fatima Noor', 30), " +
                       "(3, 'Usman Ali', 28)";
    int rowsInserted = stmt.executeUpdate(insertSQL);
    System.out.println(rowsInserted + " rows inserted successfully!");

    // 4c: SELECT data (DQL operation)
    System.out.println("\nStep 4c: Reading data...");
    String selectSQL = "SELECT * FROM emp";
    rs = stmt.executeQuery(selectSQL);
```

```java
// Display results
System.out.println("Employee Data:");
System.out.println("+----+------------------+-----+");
System.out.println("| ID | Name             | Age |");
System.out.println("+----+------------------+-----+");

while (rs.next()) {
    int id = rs.getInt("id");           // Get by column name
    String name = rs.getString("name"); // Get by column name
    int age = rs.getInt("age");         // Get by column name

    // Alternative: using column index (1-based)
    // int id = rs.getInt(1);
    // String name = rs.getString(2);
    // int age = rs.getInt(3);

    System.out.printf("| %-2d | %-16s | %-3d |\n", id, name, age);
}
System.out.println("+----+------------------+-----+");

// 4d: UPDATE data
System.out.println("\nStep 4d: Updating data...");
String updateSQL = "UPDATE emp SET age = 26 WHERE id = 1";
int rowsUpdated = stmt.executeUpdate(updateSQL);
System.out.println(rowsUpdated + " row(s) updated!");

// 4e: DELETE data
System.out.println("\nStep 4e: Deleting data...");
String deleteSQL = "DELETE FROM emp WHERE id = 3";
int rowsDeleted = stmt.executeUpdate(deleteSQL);
System.out.println(rowsDeleted + " row(s) deleted!");

// Show final data
System.out.println("\nFinal Data after Update and Delete:");
rs = stmt.executeQuery("SELECT * FROM emp");
System.out.println("+----+------------------+-----+");
System.out.println("| ID | Name             | Age |");
System.out.println("+----+------------------+-----+");
```

```
            while (rs.next()) {
                System.out.printf("| %-2d | %-16s | %-3d |\n",
                                  rs.getInt(1), rs.getString(2), rs.getInt(3));
            }
            System.out.println("+----+------------------+-----+");

        } catch (ClassNotFoundException e) {
            System.err.println("JDBC Driver not found! Check your classpath.");
            System.err.println("Error: " + e.getMessage());

        } catch (SQLException e) {
            System.err.println("SQL Error occurred!");
            System.err.println("Error Code: " + e.getErrorCode());
            System.err.println("SQL State: " + e.getSQLState());
            System.err.println("Message: " + e.getMessage());

        } finally {
            // Step 5: Close connection (cleanup)
            System.out.println("\nStep 5: Closing resources...");
            try {
                if (rs != null) {
                    rs.close();
                    System.out.println("ResultSet closed.");
                }
                if (stmt != null) {
                    stmt.close();
                    System.out.println("Statement closed.");
                }
                if (conn != null) {
                    conn.close();
                    System.out.println("Connection closed.");
                }
                System.out.println("All resources cleaned up successfully!");

            } catch (SQLException e) {
                System.err.println("Error while closing resources: " +
e.getMessage());
            }
        }
```

```
    }
}

// ==========================================
// JDBC with PreparedStatement (Parameterized Query)
// ==========================================
class JDBCPreparedStatementDemo {

    public static void main(String[] args) {
        String DB_URL = "jdbc:mysql://localhost:3306/sonoo";
        String USER = "root";
        String PASS = "root";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS)) {

            // PreparedStatement prevents SQL Injection
            String insertSQL = "INSERT INTO emp (id, name, age) VALUES (?, ?, ?)";
            PreparedStatement pstmt = conn.prepareStatement(insertSQL);

            // Set parameters
            pstmt.setInt(1, 4);
            pstmt.setString(2, "Sara Ahmed");
            pstmt.setInt(3, 22);

            int rows = pstmt.executeUpdate();
            System.out.println(rows + " row inserted using PreparedStatement!");

            // Select with parameter
            String selectSQL = "SELECT * FROM emp WHERE age > ?";
            PreparedStatement selectStmt = conn.prepareStatement(selectSQL);
            selectStmt.setInt(1, 24);

            ResultSet rs = selectStmt.executeQuery();
            System.out.println("\nEmployees with age > 24:");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
```

```
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `Class.forName(JDBC_DRIVER)` se driver class load karte hain — yeh dynamic loading hai. Agar driver nahi milta toh ClassNotFoundException aati hai. MySQL driver `com.mysql.jdbc.Driver` hai.
- **Step 2:** `DriverManager.getConnection(DB_URL, USER, PASS)` se database connection establish karte hain — URL mein database name, host, port specify karte hain. Connection object return hota hai.
- **Step 3:** `conn.createStatement()` se Statement object create karte hain — yeh SQL queries execute karne ke liye use hota hai.
- **Step 4:** Queries execute karte hain — `executeUpdate()` DDL/DML queries ke liye (CREATE, INSERT, UPDATE, DELETE), `executeQuery()` SELECT queries ke liye jo ResultSet return karti hai.
- **Step 5:** `finally` block mein resources close karte hain — pehle ResultSet, phir Statement, phir Connection. Close karna zaroori hai taake memory leak na ho.
- **PreparedStatement:** Parameters ke liye `?` placeholders use karte hain — SQL Injection se bachata hai aur performance better hoti hai.

PART B — Expected Output:

```
text
Step 1: Loading JDBC Driver...
Driver loaded successfully!

Step 2: Connecting to database...
Connected to database successfully!

Step 3: Creating statement...
```

```
Step 4a: Creating table...
Table created successfully!

Step 4b: Inserting data...
3 rows inserted successfully!

Step 4c: Reading data...
Employee Data:
+----+------------------+-----+
| ID | Name             | Age |
+----+------------------+-----+
| 1  | Ahmed Khan       | 25  |
| 2  | Fatima Noor      | 30  |
| 3  | Usman Ali        | 28  |
+----+------------------+-----+

Step 4d: Updating data...
1 row(s) updated!

Step 4e: Deleting data...
1 row(s) deleted!

Final Data after Update and Delete:
+----+------------------+-----+
| ID | Name             | Age |
+----+------------------+-----+
| 1  | Ahmed Khan       | 26  |
| 2  | Fatima Noor      | 30  |
+----+------------------+-----+

Step 5: Closing resources...
ResultSet closed.
Statement closed.
Connection closed.
All resources cleaned up successfully!
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke 5 steps successfully complete hue — driver load hua, connection establish hui, table create hui, data insert

hua, select query ne data display kiya, update aur delete operations bhi successful hue, aur akhir mein resources properly close hue. Yeh complete JDBC workflow hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Missing JDBC Driver JAR in Classpath

Buggy Code:

```
java
try {
    Class.forName("com.mysql.jdbc.Driver");
    Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/test",
"root", "root");
} catch (ClassNotFoundException e) {
    System.out.println("Driver not found");
}
```

What's Wrong (Roman Urdu): Student ne mysql-connector-java.jar file classpath mein add nahi ki. Agar JAR file nahi hai toh `Class.forName()` ClassNotFoundException throw karegi aur connection establish nahi ho payegi.

Fixed Code:

```
java
// Make sure to add mysql-connector-java.jar to classpath
// For Eclipse: Right-click project -> Build Path -> Configure Build Path -> Add
External JARs
// For command line: java -cp ".;mysql-connector-java-8.0.28.jar" JDBCDemo

try {
    Class.forName("com.mysql.cj.jdbc.Driver");  // For newer versions
    Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test?useSSL=false", "root", "root");
} catch (ClassNotFoundException e) {
    System.err.println("JDBC Driver not found in classpath!");
```

```
}
```

Fix Explained (Roman Urdu): Driver JAR file classpath mein add karwaya aur driver class name update kiya (newer versions mein `com.mysql.cj.jdbc.Driver` use hota hai).

Mistake 2: Not Closing Resources Properly

Buggy Code:

```
java
Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM emp");
// Missing try-catch-finally, resources not closed
```

What's Wrong (Roman Urdu): Student ne resources close nahi kiye — connection, statement, aur resultset open rehte hain. Is se memory leak hoti hai, database connections exhaust ho sakte hain, aur application slow ho jati hai.

Fixed Code:

```
java
Connection conn = null;
Statement stmt = null;
ResultSet rs = null;
try {
    conn = DriverManager.getConnection(url, user, pass);
    stmt = conn.createStatement();
    rs = stmt.executeQuery("SELECT * FROM emp");
    // Process results
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    // Always close in reverse order
    try { if (rs != null) rs.close(); } catch (SQLException e) {}
```

```
    try { if (stmt != null) stmt.close(); } catch (SQLException e) {}
    try { if (conn != null) conn.close(); } catch (SQLException e) {}
}
```

Fix Explained (Roman Urdu): finally block mein resources close kiye — pehle ResultSet, phir Statement, phir Connection. Is se memory leak nahi hoti.

Mistake 3: SQL Injection Vulnerability

Buggy Code:

```
java
String userId = request.getParameter("id");
// Vulnerable to SQL Injection
String sql = "SELECT * FROM users WHERE id = " + userId;
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

What's Wrong (Roman Urdu): Student ne user input ko directly SQL query mein concatenate kiya hai. Agar user input mein `1 OR 1=1` bhejta hai toh saari rows return ho jati hain, ya `1; DROP TABLE users` bhejta hai toh table delete ho jati hai. Yeh SQL Injection vulnerability hai.

Fixed Code:

```
java
String userId = request.getParameter("id");
// Use PreparedStatement to prevent SQL Injection
String sql = "SELECT * FROM users WHERE id = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, userId);
ResultSet rs = pstmt.executeQuery();
// Now user input is treated as data, not SQL code
```

Fix Explained (Roman Urdu): PreparedStatement use kiya — yeh parameters ko escape karta hai aur SQL Injection attacks se bachata hai.

Mistake 4: Wrong URL Format

Buggy Code:

```
java
// Wrong URL format
String url = "jdbc:mysql://localhost/test";  // Missing port
// OR
String url = "jdbc:mysql://localhost:3306:test";  // Wrong separator
// OR for Oracle
String url = "jdbc:oracle:thin:localhost:1521:xe";  // Missing @
```

What's Wrong (Roman Urdu): Student ne database URL ka format galat likha hai — MySQL mein port aur database name ke darmiyaan "/" aata hai, ":" nahi. Oracle mein "thin:@" format use hota hai. Galat URL se connection establish nahi hoti.

Fixed Code:

```
java
// MySQL - Correct format
String url = "jdbc:mysql://localhost:3306/test";
// Or with SSL disabled for local
String url = "jdbc:mysql://localhost:3306/test?useSSL=false";

// Oracle - Correct format
String url = "jdbc:oracle:thin:@localhost:1521:xe";

// PostgreSQL - Correct format
String url = "jdbc:postgresql://localhost:5432/test";
```

Fix Explained (Roman Urdu): Sahi URL format use kiya — MySQL ke liye `host:port/database` format, Oracle ke liye `thin:@host:port:database` format.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What is the difference between Statement and PreparedStatement in JDBC, and when would you use each?

Answer: Statement simple queries ke liye use hoti hai jab parameters nahi hote, PreparedStatement parameterized queries ke liye use hoti hai jab humein dynamic values pass karni hain. PreparedStatement SQL Injection se bachata hai kyunke parameters ko escape kar deta hai, aur performance better hoti hai kyunke query pre-compiled hoti hai aur database cache hoti hai.

PreparedStatement use karein jab user input query mein aata hai (login, search, forms), Statement use karein jab query fixed hai aur koi user input nahi hai (jaise SELECT * FROM static_table).

Q: What will happen if you execute the following code and the table doesn't exist?

```
java
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM non_existent_table");
```

Answer: Is code mein SQLException aayegi kyunke table exist nahi karti. Exception ka SQLState "42S02" (table not found) hoga aur message "Table 'database.non_existent_table' doesn't exist" aayega. Is exception ko properly handle karna chahiye taake program crash na ho. Hum try-catch block mein exception catch kar sakte hain aur user ko friendly message dikha sakte hain.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method is used to load the JDBC driver class? (Class.forName())
2. What will happen if `Class.forName("com.mysql.jdbc.Driver")` fails? (ClassNotFoundException is thrown)
3. Which JDBC interface represents the result of a query? (ResultSet)
4. What is the difference between `executeQuery()` and `executeUpdate()`? (executeQuery() returns ResultSet for SELECT, executeUpdate() returns row count for INSERT/UPDATE/DELETE)
5. What will happen if you forget to close the Connection object? (Database connections remain open, causing memory leak and connection exhaustion)

SECTION 1 — LESSON HEADING

Lesson 13: JDBC Connection, Statement, and Introduction to Web Services

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum bank mein jaate ho aur teller se paise nikalwate ho. Pehle tum apna account number dete ho (Connection establish), phir teller tumhara transaction karta hai (Statement execute), aur agar tum koi galati karte ho toh transaction rollback kar dete ho (rollback). Web services uss bank ki tarah hain jo doosre banks ke saath bhi baat kar sakta hai — chahe woh different software use karte hon, woh aapas mein standard language (XML) mein baat karte hain.

PART B — Key Content List:

Key Terms:

1. **Connection Interface**: Java application aur database ke darmiyaan session hai — Statement, PreparedStatement, aur DatabaseMetaData ka factory hai.
2. **setAutoCommit()**: Connection ka method hai jo auto-commit mode set karta hai — default true hota hai, har query ke baad automatic commit hota hai.
3. **commit()**: Connection ka method hai jo changes ko permanent save karta hai — transaction complete hotay waqt use karte hain.
4. **rollback()**: Connection ka method hai jo previous commit/rollback ke baad ki saari changes ko undo kar deta hai.
5. **Statement Interface**: SQL queries execute karne ke liye methods provide karti hai — ResultSet ka factory hai.
6. **executeQuery()**: Statement ka method hai SELECT queries execute karne ke liye — ResultSet return karta hai.
7. **executeUpdate()**: Statement ka method hai INSERT, UPDATE, DELETE, CREATE, DROP queries execute karne ke liye — affected rows count return karta hai.
8. **Web Service**: Open standards (XML, SOAP, HTTP) based web application hai jo doosri applications ke saath data exchange karti hai.
9. **WSDL**: Web Services Description Language — XML-based language hai jo web service ko describe karti hai.
10. **SOAP**: Simple Object Access Protocol — XML-based protocol hai jo web services access karne ke liye use hota hai.
11. **RDF**: Resource Description Framework — web par resources describe karne ka framework hai.
12. **RSS**: Really Simple Syndication — site content syndicate karne ka easy tareeqa hai.

Key Points:

1. **Interoperability**: Web services different platforms aur technologies ke applications ko aapas mein baat karne dete hain — VB, .NET, Java sab ek saath kaam kar sakte hain.

2. **Loosely Coupled**: Web services loosely coupled hain — ek service mein change doosre service ko affect nahi karta.
3. **XML-Based**: Web services XML use karte hain data representation aur transportation ke liye — platform independent hai.
4. **Standardized Protocol**: Web services industry standard protocols use karte hain — Service Transport, XML Messaging, Service Description, Service Discovery.
5. **Low Cost**: Web services HTTP over SOAP use karte hain — existing low-cost internet par implement ho sakte hain.

SECTION 3 — EXAMPLE

Example:

Socho tum online shopping website ho aur tumhe customer ka order process karna hai. Java application database mein order save karti hai — pehle connection establish hota hai, phir statement create hoti hai, insert query execute hoti hai, aur agar sab theek hai toh commit hota hai, agar koi error aati hai toh rollback hota hai (jaise payment fail ho jaye). Phir yeh order information web service ke zariye courier company ko bheji jati hai — SOAP message mein XML data pack karke HTTP par bheja jata hai, aur courier company ka system (chahe woh .NET ho) usay receive kar leta hai. Yahi web services ka interoperability hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```java
// ============================================
// Complete JDBC Demo with Connection, Statement, and Transaction
// ============================================
import java.sql.*;

public class JDBCConnectionStatementDemo {

    // Database credentials
    static final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
    static final String DB_URL = "jdbc:mysql://localhost:3306/sonoo?useSSL=false";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            // Step 1: Register the driver class
            System.out.println("=== Step 1: Loading Driver ===");
            Class.forName(JDBC_DRIVER);
            System.out.println("Driver loaded successfully!");

            // Step 2: Create connection
            System.out.println("\n=== Step 2: Creating Connection ===");
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            System.out.println("Connected to database successfully!");

            // Disable auto-commit for transaction management
            conn.setAutoCommit(false);
            System.out.println("Auto-commit disabled - manual transaction mode");

            // Step 3: Create statement
            System.out.println("\n=== Step 3: Creating Statement ===");
            stmt = conn.createStatement();
            System.out.println("Statement created successfully!");
```

```
            // Step 4: Execute queries

            // 4a: Create table
            System.out.println("\n=== Step 4a: Creating Table ===");
            String createTableSQL = "CREATE TABLE IF NOT EXISTS employees (" +
                                    "id INT PRIMARY KEY, " +
                                    "name VARCHAR(50), " +
                                    "salary DECIMAL(10,2), " +
                                    "dept VARCHAR(30))";
            int createResult = stmt.executeUpdate(createTableSQL);
            System.out.println("Table created (or already exists)!");

            // 4b: Insert records
            System.out.println("\n=== Step 4b: Inserting Records ===");
            String insertSQL = "INSERT INTO employees (id, name, salary, dept) VALUES
" +
                               "(101, 'Ahmed Khan', 55000.00, 'IT'), " +
                               "(102, 'Fatima Noor', 62000.00, 'HR'), " +
                               "(103, 'Usman Ali', 48000.00, 'Finance')";
            int insertCount = stmt.executeUpdate(insertSQL);
            System.out.println(insertCount + " records inserted successfully!");

            // 4c: Query using executeQuery
            System.out.println("\n=== Step 4c: Querying Data (executeQuery) ===");
            String selectSQL = "SELECT * FROM employees ORDER BY id";
            rs = stmt.executeQuery(selectSQL);

            System.out.println("Employee Data:");
            System.out.println("+----+------------------+----------+----------+");
            System.out.println("| ID | Name             | Salary   | Dept     |");
            System.out.println("+----+------------------+----------+----------+");
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                double salary = rs.getDouble("salary");
                String dept = rs.getString("dept");
                System.out.printf("| %-2d | %-16s | %8.2f | %-8s |\n",
                                  id, name, salary, dept);
            }
```

```
            System.out.println("+----+-----------------+----------+----------+");

            // 4d: Update record
            System.out.println("\n=== Step 4d: Updating Record ===");
            String updateSQL = "UPDATE employees SET salary = 58000 WHERE id = 101";
            int updateCount = stmt.executeUpdate(updateSQL);
            System.out.println(updateCount + " record updated!");

            // 4e: Delete record
            System.out.println("\n=== Step 4e: Deleting Record ===");
            String deleteSQL = "DELETE FROM employees WHERE id = 103";
            int deleteCount = stmt.executeUpdate(deleteSQL);
            System.out.println(deleteCount + " record deleted!");

            // 4f: Batch execution
            System.out.println("\n=== Step 4f: Batch Execution ===");
            stmt.addBatch("INSERT INTO employees VALUES (104, 'Sara Ahmed', 45000,
'IT')");
            stmt.addBatch("INSERT INTO employees VALUES (105, 'Ali Raza', 52000,
'HR')");
            stmt.addBatch("UPDATE employees SET salary = salary * 1.10 WHERE dept =
'IT'");

            int[] batchResults = stmt.executeBatch();
            System.out.println("Batch executed! " + batchResults.length + "
statements processed.");

            // Step 4g: Show final data after commit
            System.out.println("\n=== Step 4g: Final Data (After all operations)
===");
            rs = stmt.executeQuery("SELECT * FROM employees ORDER BY id");
            System.out.println("+----+-----------------+----------+----------+");
            System.out.println("| ID | Name            | Salary   | Dept     |");
            System.out.println("+----+-----------------+----------+----------+");
            while (rs.next()) {
                System.out.printf("| %-2d | %-16s | %8.2f | %-8s |\n",
                                  rs.getInt(1), rs.getString(2),
                                  rs.getDouble(3), rs.getString(4));
            }
```

```
            System.out.println("+----+------------------+----------+----------+");

            // COMMIT: Save all changes
            System.out.println("\n=== COMMITTING Transactions ===");
            conn.commit();
            System.out.println("All changes committed successfully!");

        } catch (ClassNotFoundException e) {
            System.err.println("Error: JDBC Driver not found!");
            System.err.println("Message: " + e.getMessage());

        } catch (SQLException e) {
            System.err.println("\n=== SQL ERROR OCCURRED ===");
            System.err.println("Error Code: " + e.getErrorCode());
            System.err.println("SQL State: " + e.getSQLState());
            System.err.println("Message: " + e.getMessage());

            // ROLLBACK: Undo all changes if any error
            try {
                if (conn != null) {
                    System.err.println("\nROLLING BACK transactions...");
                    conn.rollback();
                    System.err.println("Rollback successful! All changes undone.");
                }
            } catch (SQLException ex) {
                System.err.println("Rollback failed: " + ex.getMessage());
            }

        } finally {
            // Step 5: Close resources
            System.out.println("\n=== Step 5: Closing Resources ===");
            try {
                if (rs != null) {
                    rs.close();
                    System.out.println("ResultSet closed.");
                }
                if (stmt != null) {
                    stmt.close();
                    System.out.println("Statement closed.");
```

```
                }
                if (conn != null) {
                    // Restore auto-commit before closing
                    conn.setAutoCommit(true);
                    conn.close();
                    System.out.println("Connection closed.");
                }
                System.out.println("All resources cleaned up successfully!");
            } catch (SQLException e) {
                System.err.println("Error closing resources: " + e.getMessage());
            }
        }
    }
}

// ============================================
// Web Services - Simple SOAP Message Structure Demo
// ============================================
class WebServicesDemo {

    public static void main(String[] args) {
        System.out.println("\n=== WEB SERVICES INTRODUCTION ===\n");

        // Example SOAP message
        System.out.println("SOAP Message Example (XML):");
        System.out.println("<?xml version=\"1.0\"?>");
        System.out.println("<soap:Envelope");
        System.out.println("  xmlns:soap=\"http://www.w3.org/2001/12/soap-
envelope\"");
        System.out.println("  soap:encodingStyle=\"http://www.w3.org/2001/12/soap-
encoding\">");
        System.out.println("");
        System.out.println("  <soap:Header>");
        System.out.println("    <m:Transaction
xmlns:m=\"http://example.com/transaction\">");
        System.out.println("      <m:ID>TX12345</m:ID>");
        System.out.println("    </m:Transaction>");
        System.out.println("  </soap:Header>");
        System.out.println("");
```

```
        System.out.println("  <soap:Body>");
        System.out.println("    <m:GetAccountBalance
xmlns:m=\"http://example.com/account\">");
        System.out.println("      <m:AccountNumber>ACC-001</m:AccountNumber>");
        System.out.println("      <m:CustomerName>Ahmed Khan</m:CustomerName>");
        System.out.println("    </m:GetAccountBalance>");
        System.out.println("  </soap:Body>");
        System.out.println("");
        System.out.println("</soap:Envelope>");

        System.out.println("\nWeb Services Protocol Stack:");
        System.out.println("+---------------------------+");
        System.out.println("| Service Discovery (UDDI)   |");
        System.out.println("+---------------------------+");
        System.out.println("| Service Description (WSDL) |");
        System.out.println("+---------------------------+");
        System.out.println("| XML Messaging (SOAP)      |");
        System.out.println("+---------------------------+");
        System.out.println("| Service Transport (HTTP)   |");
        System.out.println("+---------------------------+");

        System.out.println("\nKey Web Services Technologies:");
        System.out.println("1. XML - Data representation and transportation");
        System.out.println("2. SOAP - XML-based protocol for accessing web
services");
        System.out.println("3. WSDL - Describes web service availability");
        System.out.println("4. UDDI - Service discovery and registration");
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `Class.forName()` se MySQL driver load karte hain — `com.mysql.cj.jdbc.Driver` (newer versions ke liye). Driver successful load hona zaroori hai.

- **Step 2:** `DriverManager.getConnection()` se Connection object create karte hain — URL, username, password pass karte hain. `conn.setAutoCommit(false)` se auto-commit disable karte hain taake hum manually transaction manage kar sakein.
- **Step 3:** `conn.createStatement()` se Statement object create karte hain — yeh SQL queries execute karne ke liye use hoti hai. Statement ResultSet ka factory hai.
- **Step 4:** Queries execute karte hain — `executeUpdate()` DML queries ke liye (CREATE, INSERT, UPDATE, DELETE) jo affected rows count return karti hai. `executeQuery()` SELECT queries ke liye jo ResultSet return karti hai.
- **Step 5:** Batch execution use karte hain — `addBatch()` se multiple statements add karte hain aur `executeBatch()` se ek saath execute karte hain. Is se performance improve hoti hai.
- **Step 6:** Transaction management — agar sab queries successful hain toh `conn.commit()` call karte hain jo changes permanent save karta hai. Agar koi exception aati hai toh `conn.rollback()` call karte hain jo saari changes undo kar deta hai.
- **Step 7:** `finally` block mein resources close karte hain — first ResultSet, then Statement, then Connection. Reverse order mein close karna best practice hai.

PART B — Expected Output:

```
text
=== Step 1: Loading Driver ===
Driver loaded successfully!

=== Step 2: Creating Connection ===
Connected to database successfully!
Auto-commit disabled - manual transaction mode

=== Step 3: Creating Statement ===
Statement created successfully!

=== Step 4a: Creating Table ===
Table created (or already exists)!

=== Step 4b: Inserting Records ===
```

```
3 records inserted successfully!

=== Step 4c: Querying Data (executeQuery) ===
Employee Data:
+----+-----------------+----------+----------+
| ID | Name            | Salary   | Dept     |
+----+-----------------+----------+----------+
| 101| Ahmed Khan      | 55000.00 | IT       |
| 102| Fatima Noor     | 62000.00 | HR       |
| 103| Usman Ali       | 48000.00 | Finance  |
+----+-----------------+----------+----------+

=== Step 4d: Updating Record ===
1 record updated!

=== Step 4e: Deleting Record ===
1 record deleted!

=== Step 4f: Batch Execution ===
Batch executed! 3 statements processed.

=== Step 4g: Final Data (After all operations) ===
+----+-----------------+----------+----------+
| ID | Name            | Salary   | Dept     |
+----+-----------------+----------+----------+
| 101| Ahmed Khan      | 58000.00 | IT       |
| 102| Fatima Noor     | 62000.00 | HR       |
| 104| Sara Ahmed      | 45000.00 | IT       |
| 105| Ali Raza        | 52000.00 | HR       |
+----+-----------------+----------+----------+

=== COMMITTING Transactions ===
All changes committed successfully!

=== Step 5: Closing Resources ===
ResultSet closed.
Statement closed.
Connection closed.
All resources cleaned up successfully!
```

```
=== WEB SERVICES INTRODUCTION ===

SOAP Message Example (XML):
<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">

  <soap:Header>
    <m:Transaction xmlns:m="http://example.com/transaction">
      <m:ID>TX12345</m:ID>
    </m:Transaction>
  </soap:Header>

  <soap:Body>
    <m:GetAccountBalance xmlns:m="http://example.com/account">
      <m:AccountNumber>ACC-001</m:AccountNumber>
      <m:CustomerName>Ahmed Khan</m:CustomerName>
    </m:GetAccountBalance>
  </soap:Body>

</soap:Envelope>

Web Services Protocol Stack:
+---------------------------+
| Service Discovery (UDDI)  |
+---------------------------+
| Service Description (WSDL) |
+---------------------------+
| XML Messaging (SOAP)      |
+---------------------------+
| Service Transport (HTTP)  |
+---------------------------+

Key Web Services Technologies:
1. XML - Data representation and transportation
2. SOAP - XML-based protocol for accessing web services
3. WSDL - Describes web service availability
```

```
4. UDDI - Service discovery and registration
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke JDBC ke 5 steps successful hue — driver load hua, connection establish hui, statement create hui, queries execute hui (insert, update, delete, batch), aur transaction commit hua. Web services section mein SOAP message ka XML structure dikhaya gaya hai aur web services protocol stack explain kiya gaya hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Not Handling Commit/Rollback Properly

Buggy Code:

```
java
Connection conn = DriverManager.getConnection(url, user, pass);
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
stmt.executeUpdate("INSERT INTO emp VALUES (1, 'Ali')");
// Error occurs here - connection closes without commit/rollback
conn.close();
```

What's Wrong (Roman Urdu): Student ne auto-commit disable ki hai lekin commit ya rollback nahi ki. Agar error aati hai toh connection close ho jati hai aur transactions pending reh jati hain, is se database inconsistent state mein aa sakta hai.

Fixed Code:

```
java
Connection conn = null;
try {
    conn = DriverManager.getConnection(url, user, pass);
```

```
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();
    stmt.executeUpdate("INSERT INTO emp VALUES (1, 'Ali')");
    conn.commit();  // Commit on success
} catch (SQLException e) {
    if (conn != null) {
        conn.rollback();  // Rollback on error
    }
} finally {
    if (conn != null) conn.close();
}
```

Fix Explained (Roman Urdu): Try-catch block mein commit aur rollback added — success par commit, error par rollback, is se data consistency maintain rehti hai.

Mistake 2: Using executeQuery() for UPDATE/INSERT Queries

Buggy Code:

```
java
Statement stmt = conn.createStatement();
// Wrong: executeQuery for INSERT
ResultSet rs = stmt.executeQuery("INSERT INTO emp VALUES (1, 'Ali')");
```

What's Wrong (Roman Urdu): Student ne INSERT query ke liye `executeQuery()` use kiya hai, lekin yeh method sirf SELECT queries ke liye hai. INSERT query ResultSet return nahi karti, is liye SQLException aayegi.

Fixed Code:

```
java
Statement stmt = conn.createStatement();
// Correct: executeUpdate for INSERT/UPDATE/DELETE
int rows = stmt.executeUpdate("INSERT INTO emp VALUES (1, 'Ali')");
System.out.println(rows + " row(s) inserted.");
```

Fix Explained (Roman Urdu): INSERT query ke liye `executeUpdate()` use kiya, jo affected rows count return karta hai.

Mistake 3: Forgetting to Set Auto-Commit Back to True Before Closing

Buggy Code:

```
java
conn.setAutoCommit(false);
// ... perform operations
conn.commit();
conn.close();  // Connection closes while auto-commit is false
```

What's Wrong (Roman Urdu): Student ne auto-commit false set kiya aur commit kiya, lekin connection close karne se pehle auto-commit true nahi kiya. Is se connection pool mein connection wapas aane par auto-commit false state mein rehti hai, jo future operations ko affect kar sakti hai.

Fixed Code:

```
java
conn.setAutoCommit(false);
try {
    // ... perform operations
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
} finally {
    // Restore auto-commit before closing
    conn.setAutoCommit(true);
    conn.close();
}
```

Fix Explained (Roman Urdu): Connection close karne se pehle `setAutoCommit(true)` call kiya taake connection pool mein wapas jane par default state mein ho.

Mistake 4: SQL Injection in Statement

Buggy Code:

```
java
String userInput = request.getParameter("dept");
// Vulnerable to SQL Injection
String sql = "SELECT * FROM employees WHERE dept = '" + userInput + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

What's Wrong (Roman Urdu): Student ne user input ko directly SQL query mein concatenate kiya hai. Agar user input `' OR '1'='1` bhejta hai toh saari rows return ho jati hain, data leak ho sakta hai.

Fixed Code:

```
java
String userInput = request.getParameter("dept");
// Use PreparedStatement to prevent SQL Injection
String sql = "SELECT * FROM employees WHERE dept = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, userInput);
ResultSet rs = pstmt.executeQuery();
// Parameters are properly escaped - safe from SQL Injection
```

Fix Explained (Roman Urdu): PreparedStatement use kiya jo parameters ko escape karta hai, SQL Injection attacks se bachata hai.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if the following code is executed and an SQLException occurs after the first update? Explain the transaction behavior.

```
java
Connection conn = DriverManager.getConnection(url, user, pass);
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
stmt.executeUpdate("UPDATE accounts SET balance = balance - 100 WHERE id = 1");
// SQLException occurs here (e.g., constraint violation)
stmt.executeUpdate("UPDATE accounts SET balance = balance + 100 WHERE id = 2");
conn.commit();
```

Answer: Agar second update mein SQLException aati hai (kyunke foreign key constraint violate ho, ya column not found, etc.), toh transaction commit nahi hogi. Kyunke auto-commit false hai, first update bhi pending hai. Exception catch block mein agar rollback karte hain toh first update bhi undo ho jayegi. Agar rollback nahi karte toh connection close honay par automatic rollback ho jayegi (connection close hotay waqt pending transactions rollback ho jati hain). Is liye data consistency maintain rehti hai — ya toh dono updates apply honge, ya koi nahi.

Q: What is the difference between Statement and PreparedStatement in terms of performance and security, and when would you choose one over the other?

Answer: Statement simple queries ke liye use hoti hai jab query static hai aur koi user input nahi hai. PreparedStatement parameterized queries ke liye use hoti hai jab dynamic values pass karni hain.

PreparedStatement ke faiday:

1. **Security**: SQL Injection se bachata hai kyunke parameters escape ho jate hain
2. **Performance**: Query pre-compiled hoti hai aur database cache hoti hai, repeated execution faster hoti hai
3. **Readability**: Code clean aur maintainable hota hai

Statement use karein jab query fixed hai (jaise `SELECT * FROM static_table`). PreparedStatement use karein jab user input query mein aata hai (login, search, forms) — security aur performance dono better hain.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which method of Statement is used to execute a SELECT query? (executeQuery())
2. What will happen if `conn.setAutoCommit(false)` is set and `conn.commit()` is not called? (Changes remain pending and are rolled back when connection closes)
3. Which method is used to undo all changes since the last commit in a transaction? (rollback())
4. What is the difference between `executeQuery()` and `executeUpdate()`? (executeQuery() returns ResultSet for SELECT, executeUpdate() returns row count for INSERT/UPDATE/DELETE)
5. What will this code do?

```
java
stmt.addBatch("INSERT INTO emp VALUES (1, 'Ali')");
stmt.addBatch("INSERT INTO emp VALUES (2, 'Ahmed')");
int[] results = stmt.executeBatch();
```

(Executes both insert statements as a batch and returns array of row counts)

SECTION 1 — LESSON HEADING

Lesson 14: Web Services Architecture, Protocol Stack, and RESTful Services

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:
Yeh bilkul waisi baat hai jaise tum kisi market mein jaate ho aur kisi shop se samaan khareedna chahte ho. Shopkeeper (Service Provider) apna samaan rakhta hai, tum (Service Requestor) us se samaan khareedte ho, aur market ka manager (Service Registry) batata hai ke kaun si shop mein kya milta hai. Web services architecture bhi yahi hai — provider service deta hai, requestor use karta hai, aur registry dono ko ek jagah laati hai.

PART B — Key Content List:

Key Terms:

1. **Service Provider**: Web service implement karta hai aur use Internet par available karta hai — service ka owner hai.
2. **Service Requestor**: Web service ka consumer hai — existing service ko use karta hai XML request bhej kar.
3. **Service Registry**: Services ka centralized directory hai — developers naye services publish kar sakte hain aur existing dhoondh sakte hain.
4. **Service Transport Layer**: Messages ko applications ke darmiyaan transport karne ki responsibility hai — HTTP, SMTP, FTP, BEEP.
5. **XML Messaging Layer**: Messages ko common XML format mein encode karta hai — SOAP aur XML-RPC.
6. **Service Description Layer**: Web service ka public interface describe karta hai — WSDL.
7. **Service Discovery Layer**: Services ko common registry mein centralize karta hai — UDDI.
8. **SOAP**: Simple Object Access Protocol — XML-based protocol hai web services access karne ke liye.
9. **WSDL**: Web Services Description Language — XML-based language hai web services describe karne ke liye.
10. **UDDI**: Universal Description, Discovery, and Integration — XML-based standard hai web services publish aur find karne ke liye.
11. **REST**: REpresentational State Transfer — web standards based architecture hai jo HTTP protocol use karti hai.

12. **RESTful Web Services**: REST architecture par based web services — HTTP methods (GET, POST, PUT, DELETE) use karte hain.

Key Points:

1. **Loose Coupling**: Web service consumer directly tied nahi hai — interface change ho sakta hai without compromising client interaction.
2. **Coarse-Grained**: Web services coarse-grained services provide karte hain — fine-grained methods ko compose kar ke useful service banate hain.
3. **Synchronous/Asynchronous**: Synchronous mein client blocks aur wait karta hai, Asynchronous mein client service invoke karke doosre functions execute kar sakta hai.
4. **RPC Support**: Web services remote procedures, functions, aur methods ko invoke karne allow karte hain XML-based protocol ke zariye.
5. **Document Exchange**: XML documents exchange kar sakte hain — simple address se complex RFQ (Request for Quotation) tak.

Superstructure:

1. **XML-RPC**: Simplest XML-based protocol hai RPCs perform karne ke liye — platform-independent, diverse applications communicate kar sakti hain.
2. **SOAP**: XML-based protocol hai information exchange ke liye — communication protocol, platform-independent, language-independent, firewall-friendly.
3. **WSDL**: XML-based language hai web services aur unhe access karne ka tareeqa describe karne ke liye — standard format, UDDI ka integral part.
4. **UDDI**: XML-based standard hai web services describe, publish, aur find karne ke liye — distributed registry, platform-independent, SOAP/ CORBA/ RMI communicate kar sakta hai.

SECTION 3 — EXAMPLE

Example:

Socho tum online taxi booking app use kar rahe ho. App (Service Requestor) taxi company ke server (Service Provider) se taxi availability check karta hai. Taxi company apni services ek registry (Service Registry) par publish karti hai jahan doosri companies bhi services dhoondh sakti hain. Jab tum taxi book karte ho, request HTTP par SOAP message mein bheji jati hai, server WSDL ke mutabiq service identify karta hai, aur response wapas aata hai. Agar tum asynchronous use karte ho toh app background

mein request bhej kar tumhe wait nahi karwana padta. Yahi web services architecture hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
// ============================================
// Web Services Architecture - Simple SOAP Client Simulation
// ============================================
import java.io.*;
import java.net.*;
import java.util.*;

// Service Provider interface
interface WebServiceProvider {
    String processRequest(String soapRequest);
}

// Service Registry - UDDI Simulation
class ServiceRegistry {
    private Map<String, WebServiceProvider> services = new HashMap<>();

    // Publish service
    public void publish(String serviceName, WebServiceProvider provider) {
        services.put(serviceName, provider);
        System.out.println("Service published: " + serviceName);
    }

    // Find service
    public WebServiceProvider find(String serviceName) {
        WebServiceProvider provider = services.get(serviceName);
        if (provider == null) {
            System.out.println("Service not found: " + serviceName);
        } else {
            System.out.println("Service found: " + serviceName);
```

```
        }
        return provider;
    }

    // List all services
    public void listServices() {
        System.out.println("\n=== Available Services in Registry ===");
        for (String name : services.keySet()) {
            System.out.println(" - " + name);
        }
    }
}

// Service Provider implementation - Weather Service
class WeatherServiceProvider implements WebServiceProvider {
    @Override
    public String processRequest(String soapRequest) {
        System.out.println("WeatherService: Processing SOAP request...");
        // Simulate parsing SOAP request and generating response
        String city = extractCityFromSOAP(soapRequest);
        String temperature = generateTemperature(city);

        // Build SOAP response
        return "<?xml version=\"1.0\"?>" +
               "<soap:Envelope xmlns:soap=\"http://www.w3.org/2001/12/soap-
envelope\">" +
               "  <soap:Body>" +
               "    <WeatherResponse>" +
               "      <City>" + city + "</City>" +
               "      <Temperature>" + temperature + "</Temperature>" +
               "      <Status>Success</Status>" +
               "    </WeatherResponse>" +
               "  </soap:Body>" +
               "</soap:Envelope>";
    }

    private String extractCityFromSOAP(String soapRequest) {
        // Simulate parsing - in real app use proper XML parser
        if (soapRequest.contains("Karachi")) return "Karachi";
```

```
        if (soapRequest.contains("Lahore")) return "Lahore";
        if (soapRequest.contains("Islamabad")) return "Islamabad";
        return "Unknown";
    }

    private String generateTemperature(String city) {
        // Simulate temperature based on city
        Random rand = new Random();
        int temp = 20 + rand.nextInt(20);
        return temp + "°C";
    }
}

// Service Provider implementation - Currency Converter Service
class CurrencyConverterProvider implements WebServiceProvider {
    @Override
    public String processRequest(String soapRequest) {
        System.out.println("CurrencyConverter: Processing SOAP request...");
        // Simulate conversion
        String fromCurrency = extractFromCurrency(soapRequest);
        String toCurrency = extractToCurrency(soapRequest);
        double amount = extractAmount(soapRequest);
        double convertedAmount = convertCurrency(amount, fromCurrency, toCurrency);

        // Build SOAP response
        return "<?xml version=\"1.0\"?>" +
               "<soap:Envelope xmlns:soap=\"http://www.w3.org/2001/12/soap-
envelope\">" +
               "  <soap:Body>" +
               "    <CurrencyResponse>" +
               "      <FromCurrency>" + fromCurrency + "</FromCurrency>" +
               "      <ToCurrency>" + toCurrency + "</ToCurrency>" +
               "      <Amount>" + amount + "</Amount>" +
               "      <ConvertedAmount>" + convertedAmount + "</ConvertedAmount>" +
               "      <Status>Success</Status>" +
               "    </CurrencyResponse>" +
               "  </soap:Body>" +
               "</soap:Envelope>";
    }
```

```
    private String extractFromCurrency(String soapRequest) {
        return soapRequest.contains("USD") ? "USD" : "PKR";
    }

    private String extractToCurrency(String soapRequest) {
        return soapRequest.contains("PKR") ? "PKR" : "USD";
    }

    private double extractAmount(String soapRequest) {
        // Simulate parsing amount
        return 100.0;
    }

    private double convertCurrency(double amount, String from, String to) {
        // Simulated exchange rates
        if (from.equals("USD") && to.equals("PKR")) return amount * 280;
        if (from.equals("PKR") && to.equals("USD")) return amount / 280;
        return amount;
    }
}

// RESTful Web Service - Simple Resource Representation
class RESTResource {
    private String id;
    private String name;
    private String data;

    public RESTResource(String id, String name, String data) {
        this.id = id;
        this.name = name;
        this.data = data;
    }

    // GET - Retrieve resource
    public String getRepresentation() {
        return "{\"id\": \"" + id + "\", \"name\": \"" + name + "\", \"data\": \"" +
data + "\"}";
    }
```

```
    // PUT - Update resource
    public void update(RESTResource newData) {
        this.name = newData.name;
        this.data = newData.data;
    }
}

// RESTful Service Implementation
class RESTfulService {
    private Map<String, RESTResource> resources = new HashMap<>();

    // GET method - Read resource
    public String getResource(String id) {
        RESTResource resource = resources.get(id);
        if (resource != null) {
            return resource.getRepresentation();
        }
        return "{\"error\": \"Resource not found\"}";
    }

    // POST method - Create resource
    public String createResource(String id, String name, String data) {
        if (resources.containsKey(id)) {
            return "{\"error\": \"Resource already exists\"}";
        }
        RESTResource resource = new RESTResource(id, name, data);
        resources.put(id, resource);
        return "{\"status\": \"created\", \"id\": \"" + id + "\"}";
    }

    // PUT method - Update resource
    public String updateResource(String id, String name, String data) {
        RESTResource existing = resources.get(id);
        if (existing == null) {
            return "{\"error\": \"Resource not found\"}";
        }
        RESTResource newData = new RESTResource(id, name, data);
        existing.update(newData);
```

```
        return "{\"status\": \"updated\", \"id\": \"" + id + "\"}";
    }

    // DELETE method - Remove resource
    public String deleteResource(String id) {
        RESTResource removed = resources.remove(id);
        if (removed != null) {
            return "{\"status\": \"deleted\", \"id\": \"" + id + "\"}";
        }
        return "{\"error\": \"Resource not found\"}";
    }

    // List all resources
    public String listResources() {
        StringBuilder sb = new StringBuilder();
        sb.append("[");
        for (RESTResource resource : resources.values()) {
            sb.append(resource.getRepresentation()).append(",");
        }
        if (sb.length() > 1) sb.deleteCharAt(sb.length() - 1);
        sb.append("]");
        return sb.toString();
    }
}

// Main Demo Class
public class WebServicesArchitectureDemo {

    public static void main(String[] args) {
        System.out.println("=== WEB SERVICES ARCHITECTURE DEMO ===\n");

        // Part 1: Service Registry (UDDI simulation)
        System.out.println("1. Service Registry (UDDI)");
        System.out.println("   Creating registry and publishing services...\n");

        ServiceRegistry registry = new ServiceRegistry();
        WeatherServiceProvider weatherService = new WeatherServiceProvider();
        CurrencyConverterProvider currencyService = new CurrencyConverterProvider();
```

```
registry.publish("WeatherService", weatherService);
registry.publish("CurrencyConverter", currencyService);
registry.listServices();

// Part 2: Service Discovery and Invocation
System.out.println("\n2. Service Discovery and Invocation");
System.out.println("   Finding and invoking services...\n");

// Find and invoke Weather Service
System.out.println("--- Weather Service (SOAP) ---");
WebServiceProvider foundService = registry.find("WeatherService");
if (foundService != null) {
    String soapRequest = "<?xml version=\"1.0\"?>" +
                         "<soap:Envelope>" +
                         "  <soap:Body>" +
                         "    <GetWeather>" +
                         "      <City>Karachi</City>" +
                         "    </GetWeather>" +
                         "  </soap:Body>" +
                         "</soap:Envelope>";

    String soapResponse = foundService.processRequest(soapRequest);
    System.out.println("SOAP Response received!");
    System.out.println(soapResponse);
}

// Find and invoke Currency Converter Service
System.out.println("\n--- Currency Converter Service (SOAP) ---");
foundService = registry.find("CurrencyConverter");
if (foundService != null) {
    String soapRequest = "<?xml version=\"1.0\"?>" +
                         "<soap:Envelope>" +
                         "  <soap:Body>" +
                         "    <ConvertCurrency>" +
                         "      <From>USD</From>" +
                         "      <To>PKR</To>" +
                         "      <Amount>100</Amount>" +
                         "    </ConvertCurrency>" +
                         "  </soap:Body>" +
```

```
                                "</soap:Envelope>";

            String soapResponse = foundService.processRequest(soapRequest);
            System.out.println("SOAP Response received!");
            System.out.println(soapResponse);
        }

        // Part 3: RESTful Web Services
        System.out.println("\n3. RESTful Web Services");
        System.out.println("   Demonstrating REST operations...\n");

        RESTfulService restService = new RESTfulService();

        // POST - Create resource
        System.out.println("--- POST: Create Resource ---");
        String postResponse = restService.createResource("EMP001", "Ahmed Khan",
"Java Developer");
        System.out.println("Response: " + postResponse);

        // GET - Retrieve resource
        System.out.println("\n--- GET: Retrieve Resource ---");
        String getResponse = restService.getResource("EMP001");
        System.out.println("Response: " + getResponse);

        // PUT - Update resource
        System.out.println("\n--- PUT: Update Resource ---");
        String putResponse = restService.updateResource("EMP001", "Ahmed Khan",
"Senior Java Developer");
        System.out.println("Response: " + putResponse);

        // GET again - Verify update
        System.out.println("\n--- GET: Verify Update ---");
        getResponse = restService.getResource("EMP001");
        System.out.println("Response: " + getResponse);

        // LIST - Get all resources
        System.out.println("\n--- LIST: All Resources ---");
        String listResponse = restService.listResources();
        System.out.println("Response: " + listResponse);
```

```
        // DELETE - Remove resource
        System.out.println("\n--- DELETE: Remove Resource ---");
        String deleteResponse = restService.deleteResource("EMP001");
        System.out.println("Response: " + deleteResponse);

        // Part 4: Protocol Stack
        System.out.println("\n4. Web Services Protocol Stack");
        System.out.println("   +---------------------------+");
        System.out.println("   | Service Discovery (UDDI)  |");
        System.out.println("   +---------------------------+");
        System.out.println("   | Service Description (WSDL)|");
        System.out.println("   +---------------------------+");
        System.out.println("   | XML Messaging (SOAP)      |");
        System.out.println("   +---------------------------+");
        System.out.println("   | Service Transport (HTTP)  |");
        System.out.println("   +---------------------------+");

        // Part 5: Loose Coupling and Synchronous/Asynchronous
        System.out.println("\n5. Web Services Characteristics");
        System.out.println("   - Loose Coupled: Services can evolve independently");
        System.out.println("   - Coarse-Grained: Exposes business-level operations");
        System.out.println("   - Synchronous: Client waits for response");
        System.out.println("   - Asynchronous: Client continues working, gets result
later");
        System.out.println("   - Supports RPC: Remote procedure calls");
        System.out.println("   - Supports Document Exchange: XML documents");
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `WebServiceProvider` interface define ki hai — yeh service provider ka contract hai. `processRequest()` method SOAP request leta hai aur SOAP response return karta hai.
- **Step 2:** `ServiceRegistry` class UDDI registry simulate karti hai — `publish()` method services register karta hai aur `find()` method services search karta hai. `listServices()` saari available services dikhati hai.

- **Step 3:** `WeatherServiceProvider` aur `CurrencyConverterProvider` classes service providers hain — yeh `processRequest()` implement karte hain. SOAP request parse karte hain, business logic apply karte hain, aur SOAP response generate karte hain.
- **Step 4:** `RESTResource` class RESTful architecture mein resource represent karti hai — `getRepresentation()` JSON format mein resource return karti hai, `update()` resource modify karti hai.
- **Step 5:** `RESTfulService` class REST operations provide karti hai — `createResource()` (POST), `getResource()` (GET), `updateResource()` (PUT), `deleteResource()` (DELETE), aur `listResources()`.
- **Step 6:** Main method mein pehle registry create ki, services publish ki, SOAP requests bheje aur responses receive kiye, phir REST operations demonstrate kiye. Protocol stack aur characteristics bhi display kiye.

PART B — Expected Output:

```
text
=== WEB SERVICES ARCHITECTURE DEMO ===

1. Service Registry (UDDI)
   Creating registry and publishing services...

Service published: WeatherService
Service published: CurrencyConverter

=== Available Services in Registry ===
 - WeatherService
 - CurrencyConverter

2. Service Discovery and Invocation
   Finding and invoking services...

--- Weather Service (SOAP) ---
Service found: WeatherService
WeatherService: Processing SOAP request...
SOAP Response received!
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope">
  <soap:Body>
    <WeatherResponse>
      <City>Karachi</City>
```

```
      <Temperature>28°C</Temperature>
      <Status>Success</Status>
    </WeatherResponse>
  </soap:Body>
</soap:Envelope>

--- Currency Converter Service (SOAP) ---
Service found: CurrencyConverter
CurrencyConverter: Processing SOAP request...
SOAP Response received!
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope">
  <soap:Body>
    <CurrencyResponse>
      <FromCurrency>USD</FromCurrency>
      <ToCurrency>PKR</ToCurrency>
      <Amount>100.0</Amount>
      <ConvertedAmount>28000.0</ConvertedAmount>
      <Status>Success</Status>
    </CurrencyResponse>
  </soap:Body>
</soap:Envelope>

3. RESTful Web Services
   Demonstrating REST operations...

--- POST: Create Resource ---
Response: {"status": "created", "id": "EMP001"}

--- GET: Retrieve Resource ---
Response: {"id": "EMP001", "name": "Ahmed Khan", "data": "Java Developer"}

--- PUT: Update Resource ---
Response: {"status": "updated", "id": "EMP001"}

--- GET: Verify Update ---
Response: {"id": "EMP001", "name": "Ahmed Khan", "data": "Senior Java Developer"}

--- LIST: All Resources ---
```

```
Response: [{"id": "EMP001", "name": "Ahmed Khan", "data": "Senior Java Developer"}]

--- DELETE: Remove Resource ---
Response: {"status": "deleted", "id": "EMP001"}

4. Web Services Protocol Stack
   +--------------------------+
   | Service Discovery (UDDI)  |
   +--------------------------+
   | Service Description (WSDL)|
   +--------------------------+
   | XML Messaging (SOAP)     |
   +--------------------------+
   | Service Transport (HTTP)  |
   +--------------------------+

5. Web Services Characteristics
   - Loose Coupled: Services can evolve independently
   - Coarse-Grained: Exposes business-level operations
   - Synchronous: Client waits for response
   - Asynchronous: Client continues working, gets result later
   - Supports RPC: Remote procedure calls
   - Supports Document Exchange: XML documents
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke registry services ko publish aur discover kar rahi hai, SOAP requests successfully process ho rahe hain, REST CRUD operations kaam kar rahe hain (POST, GET, PUT, DELETE), protocol stack display ho raha hai, aur web services characteristics explain ho rahi hain. Yeh complete web services architecture demonstration hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Confusing REST with SOAP in Architecture Design

Buggy Code:

```java
// Trying to use REST for a transaction that needs security and reliability
// Wrong approach for banking transaction
public void transferMoney(String fromAccount, String toAccount, double amount) {
    // Using REST GET for money transfer - WRONG
    restClient.get("/transfer?from=" + fromAccount + "&to=" + toAccount + "&amount="
+ amount);
}
```

What's Wrong (Roman Urdu): Student ne GET method use kiya hai money transfer ke liye, lekin GET idempotent aur safe hona chahiye. Money transfer state change karta hai, is liye POST ya PUT use karna chahiye. GET URL mein sensitive data visible ho jata hai aur browser history mein save ho jata hai.

Fixed Code:

```java
// Correct: Use POST for state-changing operations
public void transferMoney(String fromAccount, String toAccount, double amount) {
    // REST POST with request body
    String jsonRequest = "{\"from\": \"" + fromAccount +
                         "\", \"to\": \"" + toAccount +
                         "\", \"amount\": " + amount + "}";
    restClient.post("/transfer", jsonRequest);
}

// For read-only operations use GET
public void getBalance(String account) {
    restClient.get("/balance?account=" + account);
}
```

Fix Explained (Roman Urdu): State-changing operations ke liye POST/PUT use kiya, read-only operations ke liye GET use kiya — REST principles correctly followed.

Mistake 2: Not Understanding Loose Coupling - Hardcoding Service URLs

Buggy Code:

```java
// Wrong: Hardcoded service URL
public class Client {
    public void callService() {
        // If service moves to another server, code breaks
        String serviceUrl = "http://192.168.1.100:8080/weather";
        // Call service...
    }
}
```

What's Wrong (Roman Urdu): Student ne service URL hardcode kar di hai. Agar service provider IP change kare ya port change kare toh client ka code break ho jayega. Loose coupling violate ho raha hai.

Fixed Code:

```java
// Correct: Use configuration or service registry
public class Client {
    private String serviceUrl;

    public Client() {
        // Read from configuration file
        this.serviceUrl = System.getProperty("weather.service.url",
"http://localhost:8080/weather");
        // Or use Service Registry to discover service
    }

    public void callService() {
        // Use the configured URL
        // Call service...
    }
}
```

Fix Explained (Roman Urdu): Configuration se URL read kiya taake service location change ho toh sirf config change karna pare, code mein change na karna pare — loose coupling maintain ki.

Mistake 3: Using Synchronous Calls for Long-Running Operations

Buggy Code:

```
java
// Wrong: Synchronous call for long-running report generation
public void generateReport() {
    // Client blocks for 5 minutes waiting for report
    String report = reportService.generate();
    // User can't do anything while waiting
}
```

What's Wrong (Roman Urdu): Student ne synchronous call use kiya hai report generation ke liye jo 5 minutes leti hai. Client block ho jata hai aur user kuch nahi kar sakta. Is se user experience bad ho jati hai.

Fixed Code:

```
java
// Correct: Asynchronous call for long-running operations
public void generateReportAsync() {
    // Submit request and get correlation ID
    String correlationId = reportService.submitReportRequest();

    // Client can continue working
    // Later, poll for result using correlation ID
    // Or use callback
    System.out.println("Report generation started. Correlation ID: " +
correlationId);
    System.out.println("You will be notified when report is ready.");
}

// Polling mechanism
public void checkReportStatus(String correlationId) {
    String status = reportService.getStatus(correlationId);
    if (status.equals("COMPLETED")) {
        String report = reportService.getReport(correlationId);
        // Display report
    }
}
```

Fix Explained (Roman Urdu): Asynchronous approach use ki — client immediate response leta hai (correlation ID) aur baad mein result check kar sakta hai, user blocking nahi hota.

Mistake 4: Using Wrong HTTP Method for REST Operations

Buggy Code:

```
java
// Creating a new resource using PUT - not recommended
restClient.put("/users/123", userData);
// Updating a resource using POST - not recommended
restClient.post("/users", userData);
```

What's Wrong (Roman Urdu): Student ne REST HTTP methods galat use kiye hain — PUT create ke liye (non-idempotent), POST update ke liye. REST standards ke mutabiq POST create ke liye, PUT update ke liye, aur DELETE delete ke liye use hota hai.

Fixed Code:

```
java
// Correct REST HTTP methods
// GET: Read resource
restClient.get("/users/123");  // Get user with ID 123

// POST: Create new resource
restClient.post("/users", userData);  // Create new user

// PUT: Update existing resource
restClient.put("/users/123", userData);  // Update user with ID 123

// DELETE: Remove resource
restClient.delete("/users/123");  // Delete user with ID 123
```

Fix Explained (Roman Urdu): Correct HTTP methods use kiye — POST create, PUT update, GET read, DELETE remove. Yeh REST standards hain.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What is the difference between synchronous and asynchronous web service invocations, and when would you use each in a real-world application?

Answer: Synchronous invocation mein client service call karta hai aur response aane tak block ho jata hai — user wait karta hai, kuch aur nahi kar sakta. Asynchronous invocation mein client service call karta hai aur immediate return ho jata hai (usually with a transaction ID or correlation ID) — client doosre tasks continue kar sakta hai aur baad mein result check kar sakta hai.

Synchronous use karte hain jab response immediate chahiye aur operation fast hai, jaise currency conversion, weather check. Asynchronous use karte hain jab operation slow hai (jaise report generation, video processing, batch processing) ya jab user blocking acceptable nahi hai. Real-world mein banking transactions synchronous hoti hain (immediate confirmation), lekin monthly statement generation asynchronous hoti hai (email mein bheji jati hai).

Q: What will happen if a RESTful service receives a PUT request for a resource that doesn't exist, and what should the service return?

Answer: Agar RESTful service par PUT request aati hai resource ke liye jo exist nahi karti, toh service ko HTTP 404 Not Found return karna chahiye (agar resource identifier invalid hai) ya HTTP 201 Created agar service resource create kar deti hai (some APIs create resource on PUT). REST standards ke mutabiq PUT idempotent hai — multiple PUT requests ka same effect hona chahiye. Agar resource exist nahi karta toh service can choose to create it (return 201) or return 404.

Best practice is: agar resource path valid hai aur client complete representation bhej raha hai toh service resource create kare (201 Created), otherwise 404 Not Found return kare.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. Which layer of the web services protocol stack is responsible for encoding messages in XML format? (XML Messaging - SOAP/XML-RPC)
2. What does UDDI stand for in web services architecture? (Universal Description, Discovery, and Integration)
3. What will happen if a synchronous client calls a service that takes 10 minutes to complete? (The client will block and wait for 10 minutes before continuing)
4. Which HTTP method is idempotent and should be used for updating resources in REST? (PUT)
5. What is the difference between loosely coupled and tightly coupled systems? (Loosely coupled systems can evolve independently, tightly coupled systems require both sides to update when one changes)

SECTION 1 — LESSON HEADING

Lesson 15: SOAP vs RESTful Web Services, SOA, and JAX-WS

SECTION 2 — DEEP EXPLANATION

Deep Explanation

PART A — Opening Analogy:

Yeh bilkul waisi baat hai jaise tum restaurant mein khana order karte ho. SOAP woh tareeqa hai jahan tum waiter ko proper form mein order dete ho, woh kitchen mein jaata hai, chef ko batata hai, aur phir tumhe khana laata hai (strict protocol). REST woh tareeqa hai jahan tum directly kitchen counter par jaate ho aur chef se kehte ho "mujhe burger do" (simple, direct). Dono kaam karte hain, lekin alag-alag situations ke liye best hain.

PART B — Key Content List:

Key Terms:

1. **SOA (Service-Oriented Architecture)**: Enterprise IT architecture ka naya tareeqa hai — business processes ko IT ke saath associate karta hai aur services ke zariye business applications deliver karta hai.
2. **Service**: Well-defined, self-contained function hai jo unit of functionality represent karti hai — doosri services ke saath information exchange kar sakti hai.
3. **JAX-WS**: Java API for XML Web Services — XML-based web services aur clients build karne ke liye technology hai.
4. **RPC Style Web Service**: Method name aur parameters use karta hai XML structure generate karne ke liye — tightly coupled hai.
5. **Document Style Web Service**: Predefined schema ke against validate ho sakta hai — single document bhejta hai, loosely coupled hai.
6. **XML-RPC**: Simplest XML-based protocol hai computers ke darmiyaan information exchange ke liye — platform-independent, diverse applications communicate kar sakti hain.

Key Points:

1. **SOAP**: Protocol hai — strict standards follow karta hai, XML data format only, more bandwidth required, built-in security, JAX-WS API use karta hai.
2. **REST**: Architectural style hai — flexible, multiple data formats (JSON, XML, Plain Text), less bandwidth, inherits security from transport, JAX-RS API use karti hai.
3. **SOA Benefits**: Business processes ko services mein convert karta hai, reusability increase karta hai, loose coupling provide karta hai.
4. **JAX-WS Development**: Do tareeqay hain — RPC style aur Document style. RPC tightly coupled, Document loosely coupled.
5. **RPC vs Document**: RPC mein method name visible hai, parameters discrete values hain; Document mein operation name lose ho jata hai, parameters XML format mein hain.

SECTION 3 — EXAMPLE

Example:

Socho tum online banking application bana rahe ho. Account balance check karna (GET request) REST ke liye perfect hai — simple, fast, lightweight. Lekin money transfer karna (jo security aur reliability chahta hai) SOAP ke liye better hai — kyunke SOAP built-in security aur transaction support provide karta hai. REST daily operations ke liye, SOAP critical transactions ke liye. Yahi SOAP aur REST ka practical difference hai.

SECTION 4 — CODE SESSION

Code Session

PART A — Working Code Example:

```
java
// ============================================
// JAX-WS SOAP Web Service - RPC Style
// ============================================
import javax.jws.*;
import javax.jws.soap.*;
import javax.xml.ws.*;

// Service Endpoint Interface
@WebService(name = "CalculatorService", targetNamespace = "http://cs311.edu.pk")
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface Calculator {

    @WebMethod(operationName = "add")
    public int add(@WebParam(name = "a") int a, @WebParam(name = "b") int b);

    @WebMethod(operationName = "subtract")
    public int subtract(@WebParam(name = "a") int a, @WebParam(name = "b") int b);

    @WebMethod(operationName = "multiply")
```

```
    public int multiply(@WebParam(name = "a") int a, @WebParam(name = "b") int b);

    @WebMethod(operationName = "divide")
    public double divide(@WebParam(name = "a") int a, @WebParam(name = "b") int b);
}

// Service Implementation
@WebService(endpointInterface = "Calculator",
            serviceName = "CalculatorService",
            portName = "CalculatorPort",
            targetNamespace = "http://cs311.edu.pk")
@SOAPBinding(style = SOAPBinding.Style.RPC)
public class CalculatorImpl implements Calculator {

    @Override
    public int add(int a, int b) {
        System.out.println("Adding: " + a + " + " + b);
        return a + b;
    }

    @Override
    public int subtract(int a, int b) {
        System.out.println("Subtracting: " + a + " - " + b);
        return a - b;
    }

    @Override
    public int multiply(int a, int b) {
        System.out.println("Multiplying: " + a + " * " + b);
        return a * b;
    }

    @Override
    public double divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Division by zero!");
        }
        System.out.println("Dividing: " + a + " / " + b);
        return (double) a / b;
```

```
    }
}

// ============================================
// SOAP Client - Consuming the Web Service
// ============================================
import javax.xml.ws.*;
import java.net.URL;

public class SOAPClient {

    public static void main(String[] args) {
        try {
            System.out.println("=== SOAP Web Service Client (RPC Style) ===\n");

            // Create service URL
            URL url = new URL("http://localhost:8080/CalculatorService?wsdl");

            // Get service
            Service service = Service.create(url,
                new javax.xml.namespace.QName("http://cs311.edu.pk",
"CalculatorService"));

            // Get port (proxy)
            Calculator calculator = service.getPort(Calculator.class);

            // Call operations
            System.out.println("1. Add Operation:");
            System.out.println("   10 + 20 = " + calculator.add(10, 20));

            System.out.println("\n2. Subtract Operation:");
            System.out.println("   50 - 30 = " + calculator.subtract(50, 30));

            System.out.println("\n3. Multiply Operation:");
            System.out.println("   6 * 7 = " + calculator.multiply(6, 7));

            System.out.println("\n4. Divide Operation:");
            System.out.println("   100 / 4 = " + calculator.divide(100, 4));
```

```
            System.out.println("\n5. Divide by Zero (Error):");
            try {
                calculator.divide(10, 0);
            } catch (Exception e) {
                System.out.println("   Error: " + e.getMessage());
            }

        } catch (Exception e) {
            System.err.println("Client error: " + e.getMessage());
        }
    }
}

// ============================================
// Document Style Web Service Example
// ============================================
import javax.jws.*;
import javax.jws.soap.*;
import javax.xml.ws.*;
import javax.xml.bind.annotation.*;

// Request object for Document Style
@XmlRootElement(name = "employeeRequest")
@XmlAccessorType(XmlAccessType.FIELD)
class EmployeeRequest {
    @XmlElement(name = "employeeId")
    private int employeeId;

    @XmlElement(name = "includeDetails")
    private boolean includeDetails;

    public EmployeeRequest() {}

    public EmployeeRequest(int employeeId, boolean includeDetails) {
        this.employeeId = employeeId;
        this.includeDetails = includeDetails;
    }

    public int getEmployeeId() { return employeeId; }
```

```
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }
    public boolean isIncludeDetails() { return includeDetails; }
    public void setIncludeDetails(boolean includeDetails) { this.includeDetails =
includeDetails; }
}

// Response object for Document Style
@XmlRootElement(name = "employeeResponse")
@XmlAccessorType(XmlAccessType.FIELD)
class EmployeeResponse {
    @XmlElement(name = "employeeId")
    private int employeeId;

    @XmlElement(name = "employeeName")
    private String employeeName;

    @XmlElement(name = "department")
    private String department;

    @XmlElement(name = "salary")
    private double salary;

    @XmlElement(name = "status")
    private String status;

    public EmployeeResponse() {}

    public EmployeeResponse(int employeeId, String employeeName, String department,
double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.department = department;
        this.salary = salary;
        this.status = "SUCCESS";
    }

    // Getters and setters
    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }
```

```
    public String getEmployeeName() { return employeeName; }
    public void setEmployeeName(String employeeName) { this.employeeName =
employeeName; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}

// Document Style Web Service
@WebService(name = "EmployeeService", targetNamespace = "http://cs311.edu.pk")
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public class EmployeeService {

    @WebMethod(operationName = "getEmployee")
    public EmployeeResponse getEmployee(@WebParam(name = "request") EmployeeRequest
request) {
        System.out.println("Document Style: Getting employee with ID: " +
request.getEmployeeId());
        System.out.println("Include Details: " + request.isIncludeDetails());

        // Simulate database lookup
        EmployeeResponse response = new EmployeeResponse(
            request.getEmployeeId(),
            "Ahmed Khan",
            "IT Department",
            75000.0
        );

        if (request.isIncludeDetails()) {
            // More details would be added here
            response.setStatus("SUCCESS - Full Details");
        }

        return response;
    }
}
```

```java
// ==========================================
// RESTful Web Service Example (JAX-RS Style)
// ==========================================
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.xml.bind.annotation.XmlRootElement;

// Resource class for REST
@XmlRootElement
class Product {
    private int id;
    private String name;
    private double price;

    public Product() {}

    public Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Getters and setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}

// RESTful Web Service
@Path("/products")
public class ProductService {

    // GET - Retrieve all products
    @GET
    @Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
```

```java
public Response getProducts() {
    // Simulate database
    Product[] products = {
        new Product(1, "Laptop", 75000.0),
        new Product(2, "Mouse", 1500.0),
        new Product(3, "Keyboard", 3000.0)
    };
    return Response.ok(products).build();
}

// GET - Retrieve single product by ID
@GET
@Path("/{id}")
@Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
public Response getProduct(@PathParam("id") int id) {
    Product product = new Product(id, "Product " + id, 1000.0 * id);
    return Response.ok(product).build();
}

// POST - Create new product
@POST
@Consumes({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
@Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
public Response createProduct(Product product) {
    System.out.println("Creating product: " + product.getName());
    product.setId(100); // Simulate generated ID
    return Response.status(Response.Status.CREATED)
                   .entity(product)
                   .build();
}

// PUT - Update existing product
@PUT
@Path("/{id}")
@Consumes({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
@Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
public Response updateProduct(@PathParam("id") int id, Product product) {
    System.out.println("Updating product: " + id);
    product.setId(id);
```

```
        return Response.ok(product).build();
    }

    // DELETE - Remove product
    @DELETE
    @Path("/{id}")
    public Response deleteProduct(@PathParam("id") int id) {
        System.out.println("Deleting product: " + id);
        return Response.status(Response.Status.NO_CONTENT).build();
    }
}

// ============================================
// Comparison Demo - SOAP vs REST
// ============================================
public class SOAPvsRESTDemo {

    public static void main(String[] args) {
        System.out.println("=== SOAP vs RESTful Web Services Comparison ===\n");

        System.out.println("1. SOAP (JAX-WS) - RPC Style:");
        System.out.println("   - Uses SOAP protocol");
        System.out.println("   - WSDL for service description");
        System.out.println("   - Strict standards");
        System.out.println("   - Built-in security (WS-Security)");
        System.out.println("   - XML data format only");
        System.out.println("   - More bandwidth and resource");
        System.out.println("   - Example: Bank transactions, Enterprise systems");

        System.out.println("\n2. REST (JAX-RS) - Representational State Transfer:");
        System.out.println("   - Architectural style");
        System.out.println("   - Uses URI for resources");
        System.out.println("   - Flexible standards");
        System.out.println("   - Inherits security from HTTP");
        System.out.println("   - Multiple formats: JSON, XML, Plain Text");
        System.out.println("   - Less bandwidth and resource");
        System.out.println("   - Example: Social media APIs, Mobile apps");

        System.out.println("\n3. When to use SOAP:");
```

```
        System.out.println("   - When you need built-in security");
        System.out.println("   - When you need transaction support (ACID)");
        System.out.println("   - When you need reliable messaging");
        System.out.println("   - When both sides are Enterprise systems");

        System.out.println("\n4. When to use REST:");
        System.out.println("   - When you want simplicity and flexibility");
        System.out.println("   - When you need JSON (mobile apps)");
        System.out.println("   - When bandwidth is limited");
        System.out.println("   - When you want easy caching");
        System.out.println("   - When you have public APIs");

        System.out.println("\n5. SOA (Service-Oriented Architecture):");
        System.out.println("   - Associates business process with IT");
        System.out.println("   - Services are well-defined, self-contained");
        System.out.println("   - Loose coupling between services");
        System.out.println("   - Promotes reusability");
        System.out.println("   - Can use both SOAP and REST");

        System.out.println("\n6. JAX-WS Options:");
        System.out.println("   - RPC Style: Tightly coupled, method names visible");
        System.out.println("   - Document Style: Loosely coupled, schema
validation");
        System.out.println("   - RPC uses discrete values");
        System.out.println("   - Document uses XML documents");
    }
}
```

Code Walkthrough (Roman Urdu):

- **Step 1:** `Calculator` interface `@WebService` annotation ke saath define ki hai — yeh SOAP web service ka contract hai. `@SOAPBinding(style = SOAPBinding.Style.RPC)` indicates RPC style. `@WebMethod` har method ko web service operation banata hai.
- **Step 2:** `CalculatorImpl` class interface implement karti hai — `@WebService(endpointInterface = "Calculator")` batata hai ke yeh service endpoint hai. Methods mein business logic implement karte hain.

- **Step 3:** `SOAPClient` service consume karta hai — `Service.create()` se WSDL read karta hai, `getPort()` se service proxy leta hai, phir methods call karta hai jaise local methods hon.
- **Step 4:** `EmployeeRequest` aur `EmployeeResponse` classes `@XmlRootElement` annotations ke saath hain — yeh Document style mein XML marshalling/unmarshalling ke liye use hoti hain. `@XmlAccessorType(XmlAccessType.FIELD)` fields ko directly access karta hai.
- **Step 5:** `EmployeeService` Document style service hai — `@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)` indicate karta hai. `getEmployee()` method complex object (EmployeeRequest) leta hai aur complex object (EmployeeResponse) return karta hai.
- **Step 6:** `ProductService` RESTful service hai — `@Path("/products")` base URI define karta hai. `@GET`, `@POST`, `@PUT`, `@DELETE` HTTP methods map karte hain. `@Produces` aur `@Consumes` data formats specify karte hain (JSON, XML).

PART B — Expected Output:

```
text
=== SOAP Web Service Client (RPC Style) ===

1. Add Operation:
   10 + 20 = 30

2. Subtract Operation:
   50 - 30 = 20

3. Multiply Operation:
   6 * 7 = 42

4. Divide Operation:
   100 / 4 = 25.0

5. Divide by Zero (Error):
   Error: Division by zero!
```

=== SOAP vs RESTful Web Services Comparison ===

1. SOAP (JAX-WS) - RPC Style:
   - Uses SOAP protocol
   - WSDL for service description
   - Strict standards
   - Built-in security (WS-Security)
   - XML data format only
   - More bandwidth and resource
   - Example: Bank transactions, Enterprise systems

2. REST (JAX-RS) - Representational State Transfer:
   - Architectural style
   - Uses URI for resources
   - Flexible standards
   - Inherits security from HTTP
   - Multiple formats: JSON, XML, Plain Text
   - Less bandwidth and resource
   - Example: Social media APIs, Mobile apps

3. When to use SOAP:
   - When you need built-in security
   - When you need transaction support (ACID)
   - When you need reliable messaging
   - When both sides are Enterprise systems

4. When to use REST:
   - When you want simplicity and flexibility
   - When you need JSON (mobile apps)
   - When bandwidth is limited
   - When you want easy caching
   - When you have public APIs

5. SOA (Service-Oriented Architecture):
   - Associates business process with IT
   - Services are well-defined, self-contained
   - Loose coupling between services
   - Promotes reusability
   - Can use both SOAP and REST

```
6. JAX-WS Options:
   - RPC Style: Tightly coupled, method names visible
   - Document Style: Loosely coupled, schema validation
   - RPC uses discrete values
   - Document uses XML documents
```

Output Explanation (Roman Urdu): Output se pata chalta hai ke SOAP RPC style service successfully kaam kar rahi hai — add, subtract, multiply, divide operations execute hue. Divide by zero error properly handle hui. Comparison section clearly dikhata hai ke SOAP aur REST mein kya differences hain, kab kaunsa use karna chahiye, aur SOA kya hai.

SECTION 5 — COMMON MISTAKES (Error Finder Session)

Common Mistakes — Error Finder Session

Mistake 1: Using Wrong Style Annotation for JAX-WS

Buggy Code:

```
java
// Wrong: Using DOCUMENT style but expecting RPC behavior
@WebService(name = "MyService")
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public class MyService {
    @WebMethod
    public int add(int a, int b) {
        return a + b;
    }
}
```

What's Wrong (Roman Urdu): Student ne DOCUMENT style use kiya hai lekin RPC style ki tarah method parameters expect kar raha hai. Document style mein parameters XML document mein wrap hone chahiye, discrete values nahi. Is liye generated WSDL aur SOAP messages galat honge.

Fixed Code:

```
java
// Option 1: Use RPC style for simple parameters
@WebService(name = "MyService")
@SOAPBinding(style = SOAPBinding.Style.RPC)
public class MyService {
    @WebMethod
    public int add(@WebParam(name = "a") int a, @WebParam(name = "b") int b) {
        return a + b;
    }
}

// Option 2: Use Document style with wrapper objects
@WebService(name = "MyService")
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public class MyService {
    @WebMethod
    public AddResponse add(AddRequest request) {
        // Process request object
        return new AddResponse(request.getA() + request.getB());
    }
}
```

Fix Explained (Roman Urdu): RPC style simple parameters ke liye use kiya, Document style complex objects ke liye — style service requirements ke mutabiq choose karna chahiye.

Mistake 2: Not Setting JAX-WS Endpoint Correctly

Buggy Code:

```
java
// Wrong: Publishing without endpoint URL
import javax.xml.ws.Endpoint;
```

```
public class Publisher {
    public static void main(String[] args) {
        CalculatorImpl calc = new CalculatorImpl();
        Endpoint.publish("", calc);  // Empty URL - will fail
    }
}
```

What's Wrong (Roman Urdu): Student ne endpoint URL empty pass kar di hai. JAX-WS endpoint publish karne ke liye valid URL chahiye. Empty URL se endpoint start nahi hogi aur service accessible nahi hogi.

Fixed Code:

```
java
import javax.xml.ws.Endpoint;

public class Publisher {
    public static void main(String[] args) {
        CalculatorImpl calc = new CalculatorImpl();
        // Correct URL format
        String endpointUrl = "http://localhost:8080/CalculatorService";
        Endpoint.publish(endpointUrl, calc);
        System.out.println("Service published at: " + endpointUrl + "?wsdl");
        System.out.println("Press any key to stop...");
        System.in.read();
    }
}
```

Fix Explained (Roman Urdu): Correct endpoint URL provide ki — host, port, aur service name specify ki. WSDL automatically generate ho jati hai.

Mistake 3: Forgetting to Add @XmlRootElement for Document Style

Buggy Code:

```java
// Wrong: Missing @XmlRootElement
class MyRequest {
    private String data;
    // getters and setters
}

@WebService
public class MyService {
    @WebMethod
    public MyResponse process(MyRequest request) {  // JAXB error!
        // ...
    }
}
```

What's Wrong (Roman Urdu): Student ne Document style service mein `@XmlRootElement` annotation nahi lagayi request/response classes par. JAX-WS internally JAXB use karta hai XML marshalling/unmarshalling ke liye, aur `@XmlRootElement` zaroori hai root element ke liye.

Fixed Code:

```java
// Correct: Add @XmlRootElement
import javax.xml.bind.annotation.*;

@XmlRootElement(name = "myRequest")
@XmlAccessorType(XmlAccessType.FIELD)
class MyRequest {
    @XmlElement(name = "data")
    private String data;
    // getters and setters
}

@XmlRootElement(name = "myResponse")
@XmlAccessorType(XmlAccessType.FIELD)
class MyResponse {
    @XmlElement(name = "result")
```

```
    private String result;
    // getters and setters
}

@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public class MyService {
    @WebMethod
    public MyResponse process(@WebParam(name = "request") MyRequest request) {
        // ...
    }
}
```

Fix Explained (Roman Urdu): `@XmlRootElement` add kiya taake JAXB classes ko XML mein marshal/unmarshal kar sake.

Mistake 4: Not Handling SOAP Faults Properly

Buggy Code:

```
java
@WebService
public class CalculatorImpl implements Calculator {
    @Override
    public int divide(int a, int b) {
        return a / b;  // If b=0, ArithmeticException goes unhandled
    }
}
```

What's Wrong (Roman Urdu): Student ne division by zero handle nahi kiya. ArithmeticException server par throw hoti hai, lekin SOAP fault mein properly convert nahi hoti. Client ko generic error milega, specific SOAP fault nahi.

Fixed Code:

```java
@WebService
public class CalculatorImpl implements Calculator {
    @Override
    public int divide(int a, int b) {
        if (b == 0) {
            // Throw custom exception that maps to SOAP fault
            throw new DivisionByZeroException("Cannot divide by zero!");
        }
        return a / b;
    }
}

// Custom exception
@WebFault(name = "DivisionByZeroFault",
          faultBean = "DivisionByZeroFaultBean")
public class DivisionByZeroException extends Exception {
    public DivisionByZeroException(String message) {
        super(message);
    }
}
```

Fix Explained (Roman Urdu): Custom exception defined ki with `@WebFault` annotation taake SOAP fault properly generate ho.

SECTION 6 — CONCEPTUAL QUESTIONS

Conceptual Questions

Q: What will happen if a RESTful service is designed to handle state (e.g., shopping cart) and uses GET to add items to the cart? Why is this a problem?

Answer: Agar RESTful service GET method use kare shopping cart mein items add karne ke liye, toh yeh REST principles violate ho jata hai. GET idempotent aur safe hona chahiye — is side effects nahi hone chahiye. GET caching bhi ho jati hai, agar cache mein

request store ho jaye toh user cart mein unwanted items add ho sakte hain. Is liye state-changing operations ke liye POST, PUT, ya DELETE use karna chahiye.

REST mein resources ko URIs se identify karte hain, aur HTTP methods resources par operations define karte hain. GET = read, POST = create, PUT = update, DELETE = delete. Shopping cart mein item add karna create/update hai, is liye POST ya PUT use karna chahiye.

Q: What is the difference between RPC style and Document style web services in JAX-WS, and when would you choose one over the other?

Answer: RPC style web services method name aur parameters use karte hain XML structure generate karne ke liye — SOAP message mein operation name visible hota hai aur parameters discrete values hain. RPC tightly coupled hai aur WSDL schema validation mushkil hai. Document style web services SOAP message ko single document bhejte hain — operation name lose ho jata hai aur parameters XML format mein hain. Document loosely coupled hai aur schema validation possible hai.

RPC style choose karein jab simple operations hon aur method signatures clear hon, jaise calculator operations. Document style choose karein jab complex data structures hon, jab schema validation zaroori ho, aur jab loosely coupled architecture chahiye (enterprise systems, B2B integration). Most modern web services Document style prefer karte hain kyunke yeh more flexible aur interoperable hai.

SECTION 7 — LESSON QUIZ

Lesson Quiz

1. What is the main difference between SOAP and REST in terms of data format? (SOAP supports only XML, REST supports multiple formats like JSON, XML, Plain Text)
2. Which JAX-WS annotation is used to mark a class as a web service endpoint? (@WebService)
3. What will happen if a REST service uses GET to delete a resource? (Violates REST principles - GET should be safe and idempotent, DELETE should be used for deletion)
4. What is the difference between RPC and Document style web services? (RPC tightly coupled with method names visible, Document loosely coupled with XML documents)
5. Which Java API is used for SOAP web services and which for RESTful web services? (JAX-WS for SOAP, JAX-RS for REST)

BEST OF LUCK